MonadBFT: Fast, Responsive, Fork-Resistant Streamlined Consensus

Mohammad Mussadiq Jalalzai, Kushal Babel, Jovan Komatovic, Tobias Klenze, Sourav Das, Fatima Elsheimy, Mike Setrin, John Bergschneider, and Babak Gilkalaye

Category Labs

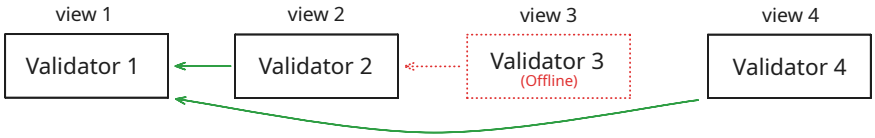
arXiv:2502.20692v4 [cs.DC] 21 Jul 2026

Abstract. This paper introduces MonadBFT, a novel Byzantine Fault Tolerant (BFT) consensus protocol that advances both performance and robustness. MonadBFT is implemented as the consensus protocol in the Monad blockchain. As a HotStuff-family protocol, MonadBFT has *linear message complexity* in the common case and is *optimistically responsive*, operating as quickly as the network allows. A central feature of MonadBFT is its *tail-forking resistance*. In pipelined BFT protocols, when a leader goes offline, the previous proposal is abandoned. Malicious leaders can exploit this tail-forking behavior as a form of Maximal Extractable Value (MEV) attack by deliberately discarding their predecessor's block, depriving that proposer of rewards and enabling transaction reordering, censorship, or theft. MonadBFT prevents such tail-forking attacks, preserving both fairness and integrity in transaction execution. Another related feature of MonadBFT is its notion of *speculative finality*, which enables parties to execute ordered transactions after a single round (i.e., a single view), with reverts occurring only in the rare case of provable leader equivocation. This mechanism reduces user-perceived latency. Additionally, we introduce the *leader fault isolation* property, which ensures that the protocol can quickly recover from a failure. To our knowledge, no prior pipelined, leader-based BFT consensus protocol combines all of these properties in a single design.

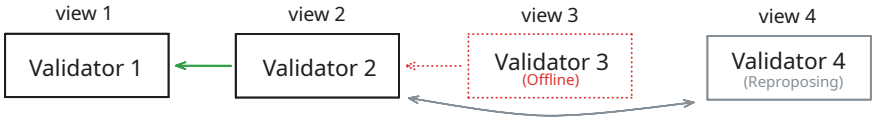
1 Introduction

Byzantine Fault Tolerant (BFT) consensus protocols allow participants in a network to reach consensus even when some participants may act arbitrarily. With the rise of blockchains, however, BFT consensus protocols must address additional challenges: ensuring fairness, since participants are directly rewarded or penalized for their behavior, and achieving scalability to support hundreds of *validators* coordinating within the protocol. In the context of a high-performance L1 blockchain, these requirements give rise to several key design goals for MonadBFT. While prior work addresses some of these goals in isolation, MonadBFT is, to the best of our knowledge, the first protocol to achieve all of them within a single framework.

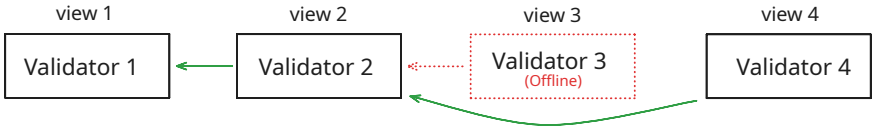
- *Rotating leaders, pipelining & linear complexity:* Unlike traditional leader-based BFT protocols [9] that have a steady leader driving consensus, blockchain BFT protocols employ rotating leaders [7, 27] to enhance fairness and robustness against slow or faulty leaders. Moreover, the protocol must pipeline commit decisions across successive leaders to achieve high throughput. Additionally, it should maintain linear communication complexity in the common case to lower validator overhead, promote wider participation, and reduce operational costs [27].
- *Optimistic responsiveness:* The *optimistic responsiveness* property lets a protocol make progress at the actual network speed δ rather than on the worst-case message delay $\Delta \gg \delta$ [14, 18, 27]. We require blockchain BFT protocols to have optimistic responsiveness, as it minimizes reliance on a carefully calibrated value of Δ, which is notoriously difficult to tune in practice and significantly affects both latency and throughput.
- *Tail-forking resistance:* Recent work [15] has identified a fairness and liveness vulnerability in (pipelined) BFT protocols, known as *tail-forking*. This vulnerability arises from a protocol design where leaders are responsible for disseminating certificates for the preceding leader's block. In Figure 1 (a), we show how an offline leader causes the previous block to be abandoned, despite the block gathering the support of most



(a) Example of tail forking in a pipelined protocol.



(b) MonadBFT prevents tail-forking via reproposals (standard recovery).



(c) Fast recovery in MonadBFT, also tail-forking resistant.

Fig. 1: Example of tail-forking (a) and how MonadBFT's standard (b) and fast (c) recovery prevent it.
(a) In standard pipelined consensus, blocks can easily be abandoned. Here, Validator 2 proposes a block that gets abandoned because Validator 3 fails to collect votes for it.
(b) MonadBFT guarantees that a correct validator's block is never abandoned. In this case, it is reproposed by Validator 4.
(c) MonadBFT often enables fast recovery via backup QC, which skips the reproposal while still preventing tail-forking.

validators. Furthermore, a malicious validator can purposefully abandon the prior block to steal, reorder, or insert transactions, capturing additional *Maximal Extractable Value (MEV)* [2,11]—a practice that has already led to hundreds of millions of dollars in user losses. Tail-forking thus amplifies MEV opportunities through cross-block transaction reordering and (short-term) censorship, giving leaders disproportionate influence over transaction inclusion. Such manipulation increases centralization pressure and undermines both the fairness and reliability of the consensus protocol.

- *Speculative commit:* To provide low-latency confirmations for typical executions, the protocol should support *speculative finality* after a single round (i.e., a single view), with revocation allowed only when the leader equivocates. Because equivocation is provable and punishable in a blockchain setting, this mechanism enables fast confirmations in the common case.
- *Leader fault isolation:* The protocol should recover from failures quickly. To this end, we introduce the *leader fault isolation* property, which states that a single Byzantine leader only incurs a single timeout delay; all views led by correct leaders operate at network speed.
- *Low latency:* The blockchain consensus protocol should have low commit latency, which leads to faster transaction confirmations and, consequently, a better user experience.

1.1 MonadBFT: Technical Overview

We now provide a brief overview of the design and highlight the key ideas in MonadBFT.

Base protocol. MonadBFT operates in the partially synchronous model, and falls within the HotStuff family of protocols, operating in a steady state under honest leaders and a stable network—referred to as the *happy path*—and transitioning to an *unhappy path* when recovering from Byzantine leaders or network instabilities.

MonadBFT progresses through a sequence of *views* (also known as *rounds*), each coordinated by a designated *leader*. Within a view, validators follow the standard pipelined consensus steps: they receive the leader's block proposal p, validate it, and send their vote to the leader of the next view. This next leader collects the issued votes and assembles a *quorum certificate* (QC) that points to p, confirming that more than $2/3$ of validators have voted for p, and then includes this QC in its own block proposal for the subsequent view. Once validators receive the next leader's proposal along with the QC for p, they enter the new view, forming a continuous, certified chain of block proposals. This describes MonadBFT's operation under ideal conditions — the happy path — where leaders act honestly and the network is stable. On the happy path, every view produces a QC that is immediately incorporated by the following leader, allowing the blockchain to steadily grow without interruption.

When failures occur, for instance due to an offline leader or long network delays, we call this the *unhappy path*. A validator will notice a failure when it remains in a view v for longer than a certain pre-specified duration. It then times out from the view and broadcasts (i.e., sends to all validators) a *timeout message* for the current view v. Timeout messages from more than $2/3$ validators indicate a failure of view v. The messages are combined into a *timeout certificate* (TC), which allows validators to advance to view $v + 1$ without waiting for a QC from view v. In general, any validator in a view $\leq v$ will transition to view $v + 1$ as soon as it receives either a QC or a TC for view v.

Tail-forking resistance via reproposals. Our design so far mirrors the pipelined HotStuff protocol, which lacks *tail-forking resistance* — the property that once a block proposed by a leader can gather a QC, it cannot be abandoned, unless the leader provably equivocated. To achieve this, MonadBFT introduces a *reproposal mechanism*, where a leader recovering from a failed view must repropose the block from that view instead of proposing a new one. Figure 1 shows MonadBFT's standard and fast recovery, which both prevent tail-forking.

Tips and high tip: A key challenge in the reproposal mechanism is to ensure that all validators agree on when and which block should be reproposed. MonadBFT enforces this by having each validator track the latest proposal it has voted for, called its *local tip*. Upon a view failure, validators include their local tip in their timeout messages. The next leader collects these messages, identifies the *high tip* — the tip with the highest view among them — and reproposes the corresponding block, unless it can construct a QC for that block. Finally, the leader includes the reported local tips into the constructed TC, allowing everyone to verify the reproposal decision. At a high level, reproposing the high tip ensures tail-forking resistance for the following reason. For a block B to obtain a QC, at least $f + 1$ honest validators must have voted for it. When recovering from a failed view, each of these honest validators reports either B or a descendant of B as its local tip. Since the high tip is determined from the $2f + 1$ local tips, at least one local tip from an honest validator that voted for B is guaranteed to be included in the high-tip computation. This ensures that B (or its extension) is always reproposed, thereby providing tail-forking resistance.

No endorsement certificates: Note that under the high-tip reproposal rule, the leader must possess the block corresponding to the high tip in order to repropose it. A naive but impractical solution would be to embed the entire block within the tip itself. Instead, in MonadBFT, a tip contains only the block's identity (its hash) and a small amount of auxiliary metadata. Additionally, MonadBFT allows a leader who does not possess the block corresponding to the high tip to retrieve the missing block by querying other validators. In rare cases where an honest leader fails to recover the block, validators jointly issue a cryptographic *no-endorsement certificate* (NEC). The existence of an NEC guarantees that the high-tip block could have never obtained a QC, allowing the honest leader to safely bypass reproposal and propose a new block instead.

Fast recovery optimizations. In addition to the above, we introduce *fast recovery* optimizations that help the protocol efficiently handle common failure scenarios. In particular, validators send *backup votes* to the current leader, enabling it to form and broadcast a *backup QC* for its own proposal, which can be used when the next leader fails to collect regular votes. Using the recovery from a single Byzantine leader as an example, Figure 3 illustrates how MonadBFT's fast recovery can restore the happy path faster than standard pipelined consensus protocols (Figure 2), while ensuring tail-forking resistance. We elaborate on these optimizations in Section 3 after formalizing the related efficiency requirements in Section 2.2.

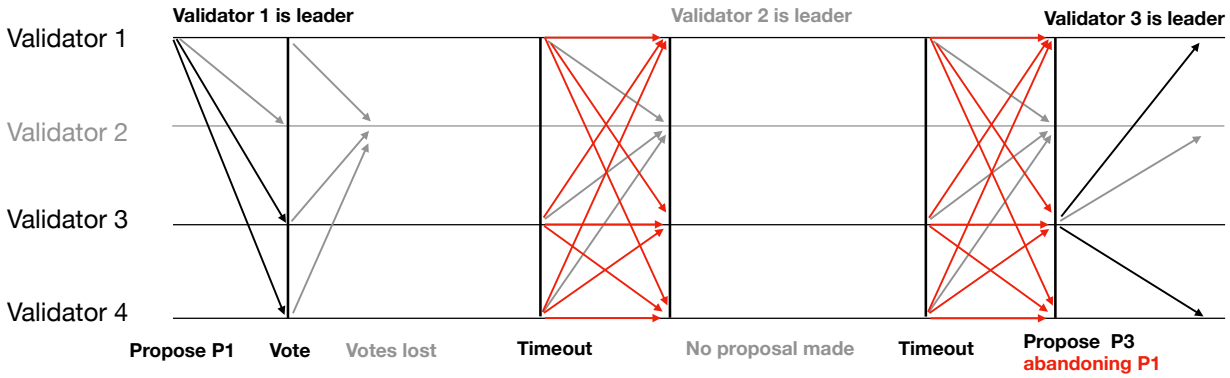


Fig. 2: In **standard pipelined BFT**, a single offline leader (Validator 2) causes two views to time out, as Validator 2 not only fails to make a proposal, but also fails to collect votes for Validator 1's block proposal P1. Furthermore, Validator 1's block is lost when Validator 3 makes its own proposal P3.

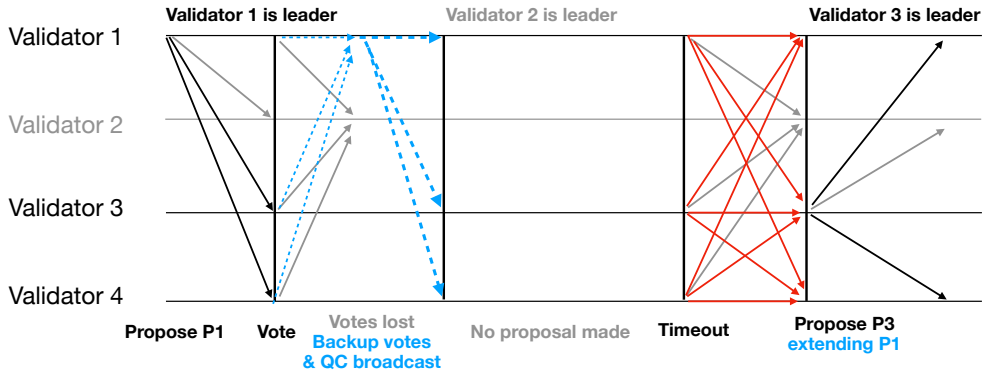


Fig. 3: In **MonadBFT**, a single offline leader (Validator 2) causes only one view to time out, as Validator 1 collects backup votes (thin dashed blue) and broadcasts a backup QC (thick dashed blue) for its own block proposal P1, allowing the view to complete successfully. Validator 3, recovering from the failure, can build directly on the backup QC of P1, ensuring that P1 (Validator 1's block) is not abandoned. To rule out tail-forks, Validator 3 must prove that Validator 2 did not produce a block that received sufficient support. This figure illustrates MonadBFT's *fast recovery* (introduced later in the paper), which avoids reproposals.

1.2 Achieved Properties of MonadBFT

Optimistic linear complexity. As in prior work [14, 18], MonadBFT achieves linear message complexity on the happy path and quadratic complexity on the unhappy path. We adopt this design for two reasons. First, we expect the protocol to operate predominantly on the happy path, since faulty leaders are rare in proof-of-stake blockchains due to financial penalties [16, 25]. Second, the quadratic communication pattern accelerates recovery after a view failure by leveraging the all-to-all message exchange. Specifically, MonadBFT relies on the all-to-all communication Bracha amplification approach [5] (hence quadratic cost) to rapidly disseminate view failure information and synchronize validators to the next view.

Optimistic responsiveness. On the happy path, MonadBFT progresses as soon as the next leader collects the QC from the previous view and includes it in its proposal, enabling the protocol to operate at network speed. Even on the unhappy path, progress remains largely independent of timeouts — these are used only to detect a view failure and trigger the transition to the unhappy path. Specifically, validators issue timeout

Table 1: Comparison of chain-based rotating leader BFT SMR protocols. δ is the actual network delay (c.f. Section 2.1).

Protocol	Min. Commit Latency	Tail-forking Resilience	Pipelined	Happy Path Complexity	Unhappy Path Complexity	Leader-based. Opt. Resp.
HotStuff [21]	7δ	×	✓	$O(n)$	$O(n)$	✓
Fast-HotStuff [18]	5δ	×	✓	$O(n)$	$O(n^2)$	✓
Jolteon [14]	5δ	×	✓	$O(n)$	$O(n^2)$	✓
HotStuff-2 [21]	5δ	×	✓	$O(n)$	$O(n)$	×
PaLa [10]	4δ	×	✓	$O(n)$	$O(n^2)$	×
Banyan [26]	3δ	×	×	$O(n^2)$	$O(n^2)$	×
BeeGees [15](1)	3δ	✓	✓	$O(n)$	$O(n^2)$	×
VBFT [17]	2δ(2)	✓	✓	$O(n^2)$	$O(n^2)$	✓
Moonshot [12]	3δ	×	✓	$O(n^2)$	$O(n^2)$	✓
HotStuff-1 (Pipelined) [19]	3δ(2)	×(3)	✓	$O(n)$	$O(n)$	×
This work	3δ(2)	✓	✓	$O(n)$	$O(n^2)$	✓

(1) Although BeeGees achieves asymptotic complexities comparable to MonadBFT, certain deployment-oriented considerations are not reflected in these bounds; we discuss these aspects in Section 5.
(2) Latency is minimized due to speculation.
(3) The HotStuff-1 pipelined variant mitigates tail-forking with continuous block proposals by the same leader ("slotting") but becomes vulnerable upon leader rotation, the original reason for the tail-forking vulnerability.

messages after waiting for Δ time units to detect such a failure. A TC, composed of these timeout messages, triggers the transition to the unhappy path; after that, the protocol operates at network speed.

Tail-forking resistance & speculative commit. Tail-forking resistance ensures that any QC-backed block, i.e., blocks that have been voted for by at least $f + 1$ honest validators, will appear in the chain at its proposed height, unless the leader equivocates.[1] We leverage this property and the ability to impose financial penalties for equivocation in proof-of-stake protocols to allow validators to *speculatively commit* blocks, i.e., validators speculatively commit a block B upon obtaining its QC. If the leader proposing B equivocates, validators may revoke the speculative commit, potentially incurring wasted effort. However, when coupled with economic disincentives for equivocation, such provable misbehavior is likely to be a rare occurrence. Therefore, we anticipate most users will adopt the speculatively finalized state, gaining the UX benefit of eliminating at least one round trip delay. In particular, latency-sensitive applications can act on the speculatively finalized state, including trading and market-making updates, oracle and index maintenance, and low-value interactive workloads (in-game asset updates, NFT listings/bids, social interactions). In contrast, settlement-critical operations spanning multiple parties or domains (bridging, custodial exchange deposits/withdrawals, treasury movements, multi-party escrows) will likely defer action until full finality. Ultimately, the latency–safety threshold is application-specific: each application can choose to utilize the speculative state or wait for full finalization, depending on its tolerance for risk and need for speed.

Latency. An immediate consequence of the speculative commitment mechanism is that validators can speculatively execute the transactions contained in a block within 1.5 round-trip time from its proposal: 1 round-trip (one view) to construct the QC and 0.5 round-trip (half a view) to disseminate it. This optimization translates into a 60% latency reduction over HotStuff [27], which requires 3.5 round trips, and a 40% reduction over faster variants [3, 18], which require 2.5 round trips. Additionally, MonadBFT achieves leader fault isolation, which limits the delays of a single Byzantine leader to a single timeout, through the use of backup QCs, which allow each honest validator to complete their own view without relying on the next leader to collect votes.

We summarize the properties of MonadBFT and compare them with existing schemes in Table 1, while a more detailed discussion of related work is provided in Section 5.

[1] In Section 2.2, we specify the precise conditions under which a QC-backed block can be reverted.

MonadBFT's implementation in the Monad blockchain. MonadBFT is implemented as the consensus protocol of the Monad blockchain, where all messages are authenticated, and validators are weighted based on their stakes. Additionally, the implementation utilizes RaptorCast [23] for efficient, erasure-coded block propagation, BlockSync [22] for retrieving missing blocks from peers, and StateSync [24] for synchronizing transaction execution states when validators fall behind. We refer to `https://docs.monad.xyz` for more details.

Paper overview. The rest of this paper is organized as follows. In Section 2, we introduce the system model, describe the properties of consensus, and present the data structures that underpin our design. In Section 3, we specify and describe the MonadBFT protocol. In Section 4, we give an intuition of how and why MonadBFT satisfies the safety, liveness, and efficiency properties introduced above; full proofs are available in Appendix A. Finally, in Section 5, we discuss related work.

2 System Model & Preliminaries

2.1 System Model

Participants. The system comprises $n = 3f + 1$ parties or validators, indexed from 1 to n, out of which up to f are Byzantine [20], i.e., they can deviate from the protocol and behave arbitrarily. (Our protocol can trivially be adapted to support $n > 3f + 1$.) Depending on the type of Byzantine behavior, we also call Byzantine validators *offline* or *malicious*. All other validators (called *correct* or *honest*) follow the protocol.

Cryptographic tools. Our protocol relies on standard cryptographic primitives, summarized below:

- *Digital signatures:* Each validator has a public–private key pair and knows the public keys of all other validators. Messages, along with their message type, are signed with the private key, while the corresponding public key is used to verify their authenticity.
- *Signature aggregation:* Our protocol additionally relies on aggregate signatures [4] to aggregate multiple validator's signatures (on potentially different messages) into a single signature. In particular, our implementation uses the pairing-based aggregate signatures from [4].

Communication network. Validators communicate over point-to-point authenticated channels. We assume that this network is *reliable*, meaning that any message sent by a correct validator to another correct validator is eventually delivered.

We assume the network to be *partially synchronous* [13], where the network behaves asynchronously up to some unknown point in time, after which it operates synchronously. Formally, this model assumes the existence of an unknown Global Stabilization Time (GST) and a known delay bound $\Delta > 0$. Any message sent at real time τ is guaranteed to be delivered by time $\max(\tau, \text{GST}) + \Delta$. We use $\delta \leq \Delta$ to denote the *actual* network delay.

2.2 Consensus Properties

Block-chaining paradigm. The primary objective of a blockchain consensus protocol is to ensure that client requests (transactions), organized into *blocks*, are *committed* (or *finalized*) onto an ever-growing linear sequence of blocks. Concretely, each validator maintains its own *local log*, but the protocol must guarantee that the local logs of all correct validators remain consistent. This guarantees that all correct validators maintain a consistent view of the log, equivalent to that of a single fault-free server. Before formalizing our protocol, we introduce the abstract notion of a block used to specify the properties of our protocol. The concrete block data structure and all related components will be detailed later in the paper; here, we rely only on the abstract formulation needed for the problem definition.

Our work follows the *block-chaining paradigm*, in which each leader proposes a value in the form of a block that explicitly extends a previously proposed block B by including the pointer of B, referred to as its *parent*. This creates a unique chain over all preceding blocks in the log. The first block in this chain is the *genesis block*, and the distance from the genesis block to a block B is its *height*. A block B_k at height k has the form

$$B_k \leftarrow \big(b_k,\, P(B_{k-1})\big),$$

where b_k is the proposed payload at height k, B_{k-1} is the block at height $k-1$ and $P(B_{k-1})$ is a pointer to block B_{k-1}. (In our protocol, pointers are realized through quorum certificates; further details appear in Section 2.3.) A block B_k is said to *extend* a block B_h if $B_k = B_h$ or B_k is a descendant of B_h. All blocks on the path from the genesis block to B_k are the *ancestors* of B_k. In this paradigm, whenever a block B_k is committed, all of its ancestors are committed as well.

Security properties. MonadBFT satisfies the following fundamental security properties:

- *Safety:* No two correct validators commit different blocks at the same log position (height) in their local logs.
- *Liveness:* After GST, every block proposed by a correct validator is eventually committed by every correct validator.

Intuitively, safety ensures that all correct validators have a consistent view of the log (see Theorem 1), while liveness ensures that, once the network stabilizes, blocks issued by correct validators are eventually committed (see Theorem 3).

Another fundamental property of leader-based pipelined blockchain consensus protocols is *tail-forking resistance*. In its classical form, as defined in [15], this property guarantees that every block proposed by a correct validator after GST is eventually committed (a guarantee we refer to as *liveness* in this work). In non-pipelined protocols, this guarantee is relatively easy to achieve: an honest leader performs all the work required to drive its own block to commitment and can therefore ensure that its block is not abandoned. In pipelined protocols, however, this reasoning breaks down because the commitment of a block proposed in view v requires the leader of view $v+1$ to collect votes for the proposal; thus, even if the leader of view v behaves correctly, it cannot unilaterally guarantee that its block will eventually be committed. This interdependence across views makes tail-forking resistance significantly more subtle to ensure in the pipelined setting.

In this work, we adopt a strictly stronger form of the property. Our variant requires that any block that receives votes from at least $f+1$ correct validators must eventually be committed, unless the leader equivocates. MonadBFT provides this guarantee for *fresh* proposals, i.e., proposals containing blocks that have not been issued before (see Section 2.3 for formal definition of fresh proposals). Since progress is already guaranteed by the above liveness property, we formalize tail-forking resistance as the following safety property (proven in Theorem 2):

- *Tail-forking resistance:* If a leader proposes a block B as part of a fresh proposal and at least $f+1$ correct validators vote for it, then—unless the leader equivocates by issuing multiple proposals—any block B' committed by a correct validator must satisfy that either B extends B' or B' extends B.

MonadBFT also incorporates *speculative commitments*, and we require explicit guarantees on when such commitments may be revoked. A block is speculatively committed once a quorum certificate (QC) is formed for a fresh proposal containing that block. Since a QC aggregates $2f+1$ votes, this implies that at least $f+1$ correct validators voted for the proposal. Our stronger tail-forking resistance property then *directly implies* that a speculatively committed block cannot be reversed unless the issuing leader equivocates, and equivocation is a provable and punishable fault that is expected to be rare in practice. Thus, the stronger definition of the tail-forking resistance property simultaneously ensures that every block proposed by a correct leader after GST is eventually committed (a security property for honest leaders) and that speculative commitments can only be revoked in the presence of equivocation (a security property for applications operating on the speculatively finalized state). The latter property, following directly from our strengthened tail-forking resistance property, can be formalized as (and proven in Corollary 1):

– *Reversion of speculatively committed blocks:* Suppose a block B contained in a fresh proposal is speculatively committed by a correct validator, and another block B' is committed by a correct validator such that neither B extends B' nor B' extends B. Then, the leader that proposed B equivocated.

Efficiency properties. Next, we discuss the efficiency properties satisfied by MonadBFT. We assume that local computation has negligible time overhead. Some of these properties depend on the high-level design of the protocol, which we will outline as needed. We begin by formalizing the properties that characterize the protocol's behavior in the common-case operation—that is, when the network is synchronous and all (or most) validators behave honestly. In this setting, we require MonadBFT to operate as efficiently as the network permits, and achieve both low latency and linear communication overhead. We define these properties formally below.

– *Leader-based optimistic responsiveness [12]:* After GST, a correct leader of view v needs to wait only until $n - f = 2f + 1$ messages of view $v - 1$ of the same type (e.g., votes) have been received to guarantee that it can create a proposal that will be voted upon by all correct validators.[2]
– *Optimistic responsiveness:* If all validators are correct, then during any post-GST time interval $[\tau_1, \tau_2]$, each correct validator commits $\Omega\big((\tau_2 - \tau_1)/\delta\big)$ blocks. Here, the interval $[\tau_1, \tau_2]$ is post-GST if $\tau_1 \geq$ GST, and δ denotes the actual network delay. (See Theorem 4.)
– *Linear communication:* When all validators are correct, after GST, each view requires only linear message complexity.
– *Low latency:* After GST, a block proposed by an honest leader is speculatively finalized in 1.5 round trips and fully finalized in 2.5 round trips,[3] measured from the time the leader issues the proposal.

Note that the properties discussed above, such as optimistic responsiveness and linear communication, are stated under the assumption that all (or most) validators behave correctly. However, MonadBFT provides strictly stronger guarantees. Concretely, in any post-GST period that contains views led by correct leaders, new blocks are committed within $O(\delta)$ time, and only a linear number of messages are exchanged —without requiring all validators to be correct.

Next, we capture the behavior of MonadBFT in the presence of a faulty leader. To characterize the behavior in faulty views, our protocol defines $\Theta_{\text{view}} \in O(\Delta)$ as the local timeout duration after which a view is deemed failed. It depends on the fixed upper bound Δ of the network delay, whereas δ, the actual network delay, may be substantially lower.

We now introduce the *leader fault isolation* property. It ensures that, after GST, a single Byzantine leader only causes delays of a single view timeout. This holds as an honest leader can propose and obtain a backup QC for its proposal within the same view without depending on the next leader, thereby making progress without incurring timeout delays. Recall from Section 1 that MonadBFT proceeds in sequential *views*, each with a designated leader.

– *Leader fault isolation*: After GST, the *duration* of a view $v+1$ by a Byzantine leader, following an honest leader of view v and followed by an honest leader in view $v+2$, is bound by $\Theta_{\text{view}} + O(\delta)$. The duration of a view v is the time between the first honest validator entering v, and the last honest validator entering a view $> v$. (See Theorem 6.)

Note that the duration of a view $v + 1$ by an honest leader following another honest leader in view v is bound by $O(\delta)$, i.e., the view progresses at the actual network speed and does not depend on a predetermined timeout period. This is due to the backup QC mechanism, which allows the leader of view $v + 1$ to build a QC independently from the leader of view $v + 2$.

[2] Note that by our liveness guarantee, the block of this proposal is guaranteed to be committed.
[3] Full finalization in MonadBFT requires two consecutive honest leaders.

2.3 Data Structures

In this subsection, we present the data structures we use in MonadBFT.

Views. A *view* is a positive integer representing a logical round of the consensus algorithm, each associated with a distinct *leader* (or *proposer*) responsible for issuing a block. In MonadBFT, leadership rotates in a round-robin fashion, ensuring no two consecutive views share the same leader.

Blocks and block headers. A block B is formally defined as

$$B = \langle \texttt{block_view}, \texttt{payload}, \texttt{payload_hash}, \texttt{qc}, \texttt{block_hash} \rangle,$$

with the components defined as follows:

- `block_view` is the view in which the block is first proposed.
- `payload` is the block's content.
- `payload_hash` is the cryptographic hash of `payload`.
- `qc` is a quorum certificate for the parent block (i.e., the block that this block extends), a cryptographic proof that $2f + 1$ validators have endorsed that block.
- `block_hash` is the cryptographic hash of $\langle \texttt{block_view}, \texttt{payload_hash}, \texttt{qc} \rangle$ and is used to identify the block.

A block header `block_header` is identical to a block, except with `payload` omitted.

Votes & quorum certificates. A vote *vote* is the basic unit of validator endorsement, and is defined as:

$$vote = \langle \texttt{view}, \texttt{block_hash}, \texttt{proposal_id}, \sigma \rangle,$$

where `block_hash` is defined as above, with the components specified as follows:

- `view` is the view in which the vote is cast.
- `block_hash` is the hash of the block being endorsed.
- `proposal_id` is the identifier of the block proposal being endorsed (we will formally define it in the "Proposals" paragraph).
- σ is the validator's signature over all the above fields.

A quorum certificate *qc* aggregates votes on a proposal from a quorum of validators, providing a collective attestation of support. Formally,

$$qc = \langle \texttt{view}, \texttt{block_hash}, \texttt{proposal_id}, \Sigma \rangle,$$

where `view`, `block_hash` and `proposal_id` are as defined above, and:

- Σ is an aggregated signature over $\langle \texttt{view}, \texttt{block_hash}, \texttt{proposal_id} \rangle$, combining $2f+1$ distinct validators' signatures to certify that the proposal has been endorsed by a quorum.

Only votes cast in the same view and for the same proposal can be aggregated. We say that a QC *qc points to* a proposal p if and only if $qc.\texttt{proposal_id} = p.\texttt{proposal_id}$. Each validator in MonadBFT also maintains the most recent QC that enabled it to advance to a later view. This QC, referred to as the validator's *local high QC* and stored in `last_qc`, is updated whenever the validator enters a new view due to receiving a QC (see Section 3.3).

Proposals. A block proposal, or simply a proposal p, encapsulates a block and is defined as

$$p = \langle \texttt{view}, \texttt{proposal_id}, \texttt{block}, \sigma, \texttt{tc}, \texttt{nec} \rangle,$$

with components defined as follows:

- `view` denotes the view in which the proposal is made.
- `proposal_id` is the hash of $\langle$`block.block_hash`, `view`$\rangle$.
- `block` is the block being proposed; we say that "Proposal contains Proposal.`block`".
- σ is the leader's signature over `proposal_id`.
- `tc` is a timeout certificate, providing a cryptographic proof that view `view`-1 has failed (formally defined later in the "Timeout certificates" paragraph). It can also be $\perp$.
- `nec` is a no-endorsement certificate, closely related to timeout certificates (formally defined below). It can also be $\perp$.

A *fresh proposal* contains a new block. A *reproposal*, in contrast, is a proposal that embeds a previous block. It corresponds to a previously issued proposal that failed to gather sufficient support during the initial issuance. Reproposals play a crucial role in ensuring tail-forking resistance. For a fresh proposal p, p.`view` $= p$.`block.block_view` holds, whereas for a reproposal, p.`view` $> p$.`block.block_view` holds.

Tips. The tip T of a proposal is the proposal without the payload of the contained block. Formally,

$$T = \langle \texttt{view}, \texttt{proposal_id}, \texttt{block_header}, \sigma, \texttt{tc}, \texttt{nec} \rangle.$$

Each validator maintains a *local tip* in its local_tip variable, which is crucial for enforcing MonadBFT's tail-forking resistance. The local tip is updated whenever the validator processes a new proposal; further details on updating the local tip appear in Section 3.

High tips. Given a set of tips $\{T_1, T_2, \ldots, T_j\}$, the *high tip* of the set is defined as the T_i with the largest view number, i.e., the one maximizing T_i.`view` for $1 \leq i \leq j$. If multiple distinct tips attain the same maximal view, we break ties by the largest T.`block_header.qc.view`. This corresponds to the highest-QC rule in pipelined consensus algorithms. If there are still tied tips, the leader is free to choose one of them.

Timeout messages. Validators expect regular progress by moving to higher views. A validator in view v issues a timeout message M for v if a pre-specified amount of time has passed since entering v. A timeout message carries either a local tip with its corresponding tip vote, or a QC in the `qc` field, but not both. If the validator holds a QC qc with qc.`view` $\geq$ `tip.view`, it includes only the QC, since it is more relevant. The message also includes a certificate (QC or TC) from the previous view, which allows validators in a lower view to advance to the same view. Formally, a timeout message is defined as

$$M = \langle \texttt{view}, \texttt{tip}, \texttt{tip_vote}, \texttt{qc}, \texttt{last_cer}, \sigma \rangle,$$

with components defined as follows:

- `view` denotes the view for which the timeout message is issued.
- `tip` is the tip of the latest *fresh* proposal (by view number) that the validator has voted for, i.e., the validator's local tip; can be $\perp$.
- `tip_vote` represents the validator's vote for the included tip; can be $\perp$.
- `qc` is the last QC (by view number) observed; can be $\perp$.
- `last_cer` is the last certificate (by view number) observed by the validator, i.e., a certificate from view `view` $- 1$.
- σ is a digital signature on the tuple $\langle$`view`, `tip_view`, `qc_view`$\rangle$, where if the local tip's view is lower or equal to the view of the latest QC, then `qc_view` is the view of the latest QC and `tip_view` $= \perp$. Otherwise, `qc_view` $=$ `tip.block_header.qc.view` and `tip_view` $=$ `tip.view`.

Timeout certificates. A *timeout certificate* tc is created from a quorum of $2f + 1$ timeout messages for the same view. It represents agreement on a timeout condition and determines whether the leader must repropose a previous block, and if so, which one. To determine this, only the highest tip or QC among the timeout messages is relevant, so the TC embeds just one: whichever has the higher view (ties favor the QC). To prove this is indeed the highest tip (resp. highest QC) among all $2f + 1$ messages, the TC contains

the view numbers of the local tip (resp. local high QC) from each contributing timeout message via the `tips_views` and `qcs_views` vectors, allowing validators to verify the correctness of the selection. Formally, a TC is defined as

$$tc = \langle \texttt{view}, \texttt{tips_views}, \texttt{high_tip}, \texttt{qcs_views}, \texttt{high_qc}, \Sigma \rangle,$$

with components defined as follows:

- `view` is the view for which the TC is issued.
- `tips_views` is the vector of validator-view tuples from each of the $2f + 1$ timeout messages. Specifically, `tips_views` is a vector of (i, v) tuples for each validator i whose timeout message with local tip from view v contributes to the TC.
- `high_tip` is the tip with the highest view among those received in the timeout messages; can be $\perp$.
- `qcs_views` is the vector of validator-view pairs from each of the $2f + 1$ timeout messages. It is a vector of (i, v) tuples for each validator i that submitted a timeout message M, where v is defined as
  - M.`qc`.`view` if M.`qc` $\neq \perp$, or
  - M.`tip`.`block_header`.`qc`.`view` otherwise.
- `high_qc` is the most recent QC among those received in the timeout messages; can be $\perp$.
- Σ is the aggregated signature over the timeout messages, certifying the correctness of the `view`, `tips_views` and `qcs_views` fields.

Block recovery & no-endorsement certificates. If a leader needs to repropose a high tip, but does not have the full block proposal corresponding to that tip, it initiates *block recovery*. MonadBFT guarantees that the leader is able to either obtain the requested proposal, or construct a *no-endorsement certificate* (NEC), which is a message signed by $2f + 1$ validators attesting that they did not vote for the proposal. Crucially, in the latter case, no QC could have been formed for it. Consequently, the block can be safely discarded, and the leader may issue a fresh proposal extending the QC of the requested high tip, since tail-forking resistance is only relevant for blocks supported by at least $f + 1$ honest validators. Formally, we define an NEC *nec* as:

$$nec = \langle \texttt{view}, \texttt{high_tip_qc_view}, \Sigma \rangle,$$

with components defined as follows:

- `view` is the view for which the NEC is issued.
- Σ is the aggregate signature of $2f + 1$ validators on $\langle$`view`, `high_tip_qc_view`$\rangle$
- `high_tip_qc_view` is the view of the QC of the requested high tip

We introduce the data structures and messages that are used in block recovery in Section 3.4.

Terminology. A leader *equivocates* if it issues more than one distinct proposal in a view; these are *equivocated* proposals. Moreover, we say that a block B_1 is the *parent* of a block B_2 if and only if there exists a proposal p such that

$$p.\texttt{block} = B_1 \quad \text{and} \quad p.\texttt{proposal_id} = B_2.\texttt{qc}.\texttt{proposal_id}.$$

Since the `proposal_id` field of a proposal is determined solely by the view and the block of the proposal, any two proposals with the same `proposal_id` must contain the same block. Consequently, each block has exactly one parent. We further say that a block B_2 *extends* a block B_1 if and only if $B_1 = B_2$ or B_2 is a descendant of B_1 under the parent relation. A block B_2 *strictly extends* B_1 if and only if B_2 extends B_1 and $B_2 \neq B_1$.

We now extend this terminology to proposals. Concretely, we say that a proposal p_1 is a *parent* of a proposal p_2 if and only if p_1.`block` is the parent of p_2.`block`. Since multiple proposals may contain the same block, a proposal p_2 may have more than one parent; recall that a proposal carries additional fields beyond `proposal_id`. Then, we say that a proposal p_2 *extends* a proposal p_1 if and only if block p_2.`block` extends block p_1.`block`. Finally, a proposal p_2 *strictly extends* a proposal p_1 if and only if block p_2.`block` strictly extends block p_1.`block`.

3 MonadBFT

We begin with an overview of MonadBFT, and then present its two logical modes of operation, the *happy* and *unhappy* path in detail in Sections 3.1 and 3.2. We formally describe the protocol in Algs. 1 and 2 with helper functions in Algs. 4 to 6. In Section 3.3 and Alg. 3, we describe the protocol's pacemaker, a tool critical for liveness. In Section 3.4 and Alg. 7 we describe block recovery. Our algorithms use the keyword **require** `<condition>` as a shortcut for *if not* `<condition>` *then return;* (or *return* false when a return value is required).

Protocol overview. MonadBFT protocol is a leader-based blockchain (aka state machine replication) consensus protocol. On the happy path, the protocol operates normally, without failures or network instabilities, with a new block being added to the chain in every view. This process encompasses a leader collecting votes and constructing a quorum certificate for the previous proposal, creating and disseminating a new proposal that extends the previous one, and validators voting to endorse the newly issued proposal.

Validators expect regular progress. When no valid proposal arrives within the timeout period, for instance due to network instabilities or a Byzantine leader, validators suspect a failure and broadcast timeout messages to initiate the unhappy path. Timeout messages are then aggregated by each validator into a timeout certificate, which serves two purposes: First, it proves that the view has timed out and it is safe to advance to the next view. Second, the information contained in timeout messages and aggregated in TCs prescribe a specific recovery path. We distinguish two recovery types:

- *Standard recovery* involves the leader reproposing the highest tip (also called *high tip*) reported in the collected timeout messages. If the leader already has the block B corresponding to the tip, reproposal is straightforward. Otherwise, the leader first queries for the block B using a sub-protocol that we call *block recovery*. This mechanism either (i) recovers B, allowing the leader to repropose it, or (ii) returns a proof of its unavailability in the form of an NEC, certifying that a majority of correct validators could not obtain B. In the latter case, the leader proposes a fresh block with the above NEC, and extends the block referenced by the QC included in the high tip. Looking ahead, upon successful recovery of B, reproposing B is critical to ensure tail-forking resistance.
- *Fast recovery* involves the leader obtaining or constructing a QC either from the previous view (allowing a happy-path proposal directly extending the QC), or such that the QC proves that the high tip proposal does not need to be reproposed. The leader can directly make a fresh proposal extending the block that obtained a QC.

Fast recovery streamlines recovery in the most common failure scenarios, but it is merely an optimization and not strictly necessary for MonadBFT's security properties.

3.1 Happy Path

We illustrate the happy path workflow of MonadBFT in Figure 4. All proposals in the happy path are *fresh*, and directly extend the block from the previous view. Therefore, for any such proposal p, we have

$$p.\texttt{view} = p.\texttt{block.block_view} = p.\texttt{block.qc.view} + 1; \quad p.\texttt{tc} = p.\texttt{nec} = \perp.$$

Description of the happy path. Consider a view v, led by leader L_v. L_v builds or receives a QC from view $v-1$, forms a proposal p_v using this QC, and broadcasts it to all validators.

Upon receiving proposal p_v, each validator i verifies its validity using the ValidProposal() function in Alg. 5. Specifically, validator i checks that the proposal p_v was issued by the designated leader L_v of view v. If the QC included in p_v originates from the immediately preceding view $v-1$, then it is a happy path proposal, and there should not be a TC. Validator i continues to process the proposal p_v by retrieving the *parent* (the proposal referenced by p_v.`block.qc`) and the *grandparent* (the proposal referenced by *parent*.`block.qc`) of proposal p_v, and then:

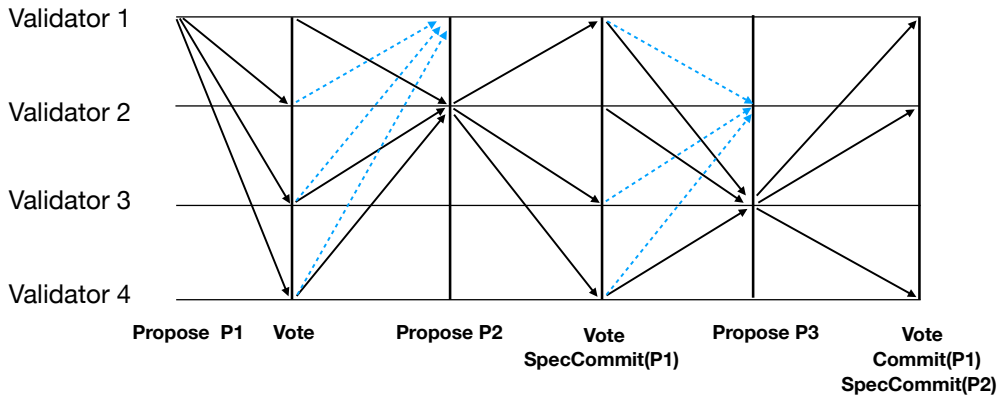


Fig. 4: Happy path workflow in MonadBFT. Note that validator sends votes to both the current (dashed, blue) and the next leader (solid, black). Each validator speculatively commits the parent block of a proposal after observing a QC from the previous round for the parent, and commits the grandparent block if the parent and grandparent have QCs from consecutive views. For simplicity, we omit the dissemination of the backup QC, as it is not required in this example.

1. It transitions to view v.
2. It (irrevocably) finalizes the *grandparent* block if the QCs pointing to *grandparent* and *parent* originate from consecutive views, i.e., $p_v.\texttt{block.qc.view} = \textit{parent}.\texttt{block.qc.view} + 1$.
3. It speculatively finalizes the *parent* if *parent* is a fresh proposal (since tail-forking resistance only holds for fresh proposals).
4. Next, validator i casts its vote for proposal p_v and sends the vote to *both* L_v and to the next leader (i.e., the leader of view $v + 1$). Additionally, validator i updates its local tip to the tip of p_v.

Upon constructing a QC from view v, the next leader L_{v+1} continues along the happy path.

Genesis. The blockchain is initialized with a *genesis tip* and a *genesis QC* for view 0, which are valid without a signature, and are used to initialize the local_tip and last_qc variables in each validator. When the protocol starts, all validators call Alg. 3.IncrementView(last_qc) to enter the first view, and the leader of that view additionally invokes Alg. 4.CreateProposalBasedOnQC(last_qc) to issue its proposal. For brevity, we omit the details on genesis and initialization of variables from the algorithms.

Achieving leader fault isolation. Since the backup QC mechanism that is crucial for leader fault isolation already plays a role on the happy path, we describe it here. Recall that v denotes the current view with leader L_v, and p_v is the proposal by L_v. Let L_{v+1} denote the leader of the next view $v+1$.

To achieve the *leader fault isolation* property (see Section 2.2), we introduce the notion of a *backup QC*. A backup QC is a quorum certificate that a leader obtains for the block it proposes. To enable L_v to form such a backup QC, we augment MonadBFT with the following modifications:

(1) Each validator, upon receiving p_v from L_v, also sends its vote for p_v back to L_v. Once L_v collects $2f+1$ votes, it constructs a *backup QC* and broadcasts it to all validators.

Intuitively, this change prevents a malicious L_{v+1} from forcing validators to timeout in view v by withholding its proposal. This also provides an alternative way for a QC to be formed and disseminated as illustrated in Figure 5.

However, sending votes to L_v alone does not always ensure that L_v can construct a backup QC. For instance, a malicious L_{v+1} might prematurely trigger view changes by sending its proposal p_{v+1} to a subset of honest validators, causing them to advance to view $v+1$ before voting for p_v. Consequently, L_v would fail to obtain a backup QC due to insufficient votes. To address this, we make the following additional change:

Algorithm 1: MonadBFT: Pseudocode for validator i [part 1 of 2]

```
// we highlight the steps needed for view synchronization (the Pacemaker) in blue
Global variables: local_tip, highest_voted_view, last_tc, last_qc, timeouts, votes, timeout_votes, cur_view
upon receiving a proposal p from validator L with p.view ≥ cur_view do
    require L is the leader of view p.view and Alg. 5.ValidProposal(p);  // if proposal is invalid, reject it
    if p.tc ≠ ⊥ then Alg. 3.IncrementView(p.tc); else Alg. 3.IncrementView(p.block.qc);
    if p.block.qc.view = p.view − 1 then
        // proposal contains the QC from the preceding view
        if i is the leader of view p.block.qc.view then
            Alg. 4.BroadcastBackupQC(p.block.qc);                    // disseminate the backup QC
        send p.block.qc to the leader of view p.block.qc.view, if not already sent;
    Alg. 4.CommitAndSpecCommit(p.block.qc); // attempt commit and speculatively commit based on QC
    if p.view > highest_voted_view then
        local_tip ← Alg. 6.GetTip(p);                                // update the local tip
        vote ← Alg. 6.CreateVote(cur_view, local_tip.block_header);  // create a vote for the proposal
        send vote to the leaders of views cur_view and cur_view + 1;
        highest_voted_view ← cur_view;          // update the highest view in which a vote has been issued
upon receiving a vote vote with vote.view ≥ cur_view do
    require Alg. 5.ValidVote(vote);
    require validator i is the leader of view vote.view or view vote.view + 1;  // only leaders handle votes
    qc ← Alg. 4.HandleVote(vote);                                // try to form a QC from 2f+1 votes
    require qc ≠ ⊥;                                  // do nothing if not collected enough votes yet
    Alg. 3.IncrementView(qc);
    Alg. 4.CommitAndSpecCommit(qc);
    if validator i is the leader of view qc.view then
        Alg. 4.BroadcastBackupQC(qc);
    else
        Alg. 4.CreateProposalBasedOnQC(qc);
upon receiving a QC qc from a validator j with qc.view ≥ cur_view do
    require Alg. 5.ValidQC(qc);
    if j is the leader of view qc.view then
        Alg. 3.IncrementView(qc);                                // enter the next view
        Alg. 4.CommitAndSpecCommit(qc);
        Alg. 4.ForwardBackupQCToNextLeader(qc);                  // process a backup QC
    if i is the leader of view qc.view then
        Alg. 4.BroadcastBackupQC(qc);                            // spread the QC
    if i is the leader of view qc.view + 1 then
        Alg. 3.IncrementView(qc);                                // enter the next view
        Alg. 4.CommitAndSpecCommit(qc);
        Alg. 4.CreateProposalBasedOnQC(qc);  // utilize the received QC to issue a new proposal
upon timing out in view cur_view do
    highest_voted_view ← cur_view;          // this prevents voting for proposals of the current view
    M ← Alg. 6.CreateTimeoutMessage(cur_view);
    broadcast M;
```

Algorithm 2: MonadBFT: Pseudocode for validator i [part 2 of 2]

upon receiving a timeout message M with $M.\texttt{view} \geq$ cur_view **do**
  require Alg. 5.ValidTimeoutMessage(M);
  if $M.\texttt{last_cer}$ is a TC **and** i has not yet broadcast a timeout message or TC for $M.\texttt{last_cer.view}$ **then**
    broadcast $M.\texttt{last_cer}$;
  Alg. 3.IncrementView($M.\texttt{last_cer}$); *// set* cur_view *to* $M.\texttt{last_cer.view} + 1$
  if i is the leader of view $M.\texttt{last_cer.view} + 1$ **and** no proposal has been issued for that view **then**
    if $M.\texttt{last_cer}$ is a QC **then**
      Alg. 4.CreateProposalBasedOnQC($M.\texttt{last_cer}$); *// utilize the QC to issue a new proposal*
    else
      Alg. 4.CreateProposalBasedOnTC($M.\texttt{last_cer}$); *// utilize the TC to issue a new proposal*
  if $M.\texttt{last_cer}$ is a QC **then**
    if i is the leader of view $M.\texttt{last_cer.view}$ **then**
      Alg. 4.BroadcastBackupQC($M.\texttt{last_cer}$);
    send $M.\texttt{last_cer}$ to the leader of view $M.\texttt{last_cer.view}$;
    send $M.\texttt{last_cer}$ to the leader of view $M.\texttt{last_cer.view} + 1$;
    Alg. 4.CommitAndSpecCommit($M.\texttt{last_cer}$);
  $tc, qc \leftarrow$ Alg. 3.HandleTimeout(M); *// returns a TC or a QC, or neither*
  if $qc \neq \perp$ **then**
    Alg. 3.IncrementView(qc); *// further timeout messages from same view will be ignored after this*
    Alg. 4.CommitAndSpecCommit(qc);
    if i is the leader of view $qc.\texttt{view} + 1$ **then**
      Alg. 4.CreateProposalBasedOnQC(qc); *// utilize the constructed QC to issue a new proposal*
  if $tc \neq \perp$ **then**
    Alg. 3.IncrementView(tc); *// further timeout messages from same view will be ignored after this*
    if i is the leader of view $tc.\texttt{view} + 1$ **then**
      Alg. 4.CreateProposalBasedOnTC(tc); *// utilize the constructed TC to issue a new proposal*

upon receiving a TC tc with $tc.\texttt{view} \geq$ cur_view **do**
  require Alg. 5.ValidTC(tc);
  Alg. 3.IncrementView(tc);
  if no TC and no timeout message broadcast for view $tc.\texttt{view}$ **then**
    broadcast tc;
  if i is the leader of view $tc.\texttt{view} + 1$ **then**
    Alg. 4.CreateProposalBasedOnTC(tc); *// utilize the received TC to issue a new proposal*

upon receiving a proposal request $\langle$ProposalRequest, $tc\rangle$ from validator L with $tc.\texttt{view} + 1 \geq$ cur_view **do**
  require Alg. 5.ValidTC(tc) **and** $tc.\texttt{high_tip} \neq \perp$ **and** L is the leader of view $tc.\texttt{view} + 1$;
  Alg. 3.IncrementView(tc);
  Alg. 7.ProcessProposalRequest(tc);

upon receiving an NE request $\langle$NERequest, $tc\rangle$ from validator L with $tc.\texttt{view} + 1 \geq$ cur_view **do**
  require Alg. 5.ValidTC(tc) **and** $tc.\texttt{high_tip} \neq \perp$ **and** L is the leader of view $tc.\texttt{view} + 1$;
  Alg. 3.IncrementView(tc);
  Alg. 7.ProcessNERequest(tc);

Algorithm 3: MonadBFT: Pacemaker

```
1  Function IncrementView(cer):
2      require cer.view ≥ cur_view;
3      stop local timer for cur_view;
4      cur_view ← cer.view + 1;
5      start local timer for cur_view;
6      if cer is a QC then last_qc ← cer; else last_tc ← cer;
7  Function HandleTimeout(M): // precondition guaranteed by caller: Alg. 5.ValidTimeoutMessage(M)
8      view ← M.view;
9      if M.tip ≠ ⊥ then
10         qc ← Alg. 4.HandleTimeoutVote(M.tip_vote);   // attempt fast recovery by building QC from tip
               votes carried by timeout messages
11         if qc ≠ ⊥ then
12             return ⊥, qc;                                          // QC is from view M.view
13     timeouts[view] ← timeouts[view] ∪ {M};
14     if |timeouts[view]| = f + 1 then
15         trigger the timeout event for M.view (= cur_view), if not already triggered;   // Bracha mechanism
16     if |timeouts[view]| < 2f + 1 then
17         return ⊥, ⊥;                                          // not enough timeouts yet to build a TC
18     // build a TC using timeout messages; only done once per view, as IncrementView is called on
19     // the resulting TC, and this function is only called for messages from cur_view
20     tips, qcs ← maps from validators to views, initialized to map all validators to 0;
21     for each msg ∈ timeouts[view] do
22         if msg.tip ≠ ⊥ then
23             tips[the sender of msg] ← msg.tip.view;
24             qcs[the sender of msg] ← msg.tip.block_header.qc.view;
25         else
26             qcs[the sender of msg] ← msg.qc.view;
27     // compare maximum reported tip view and maximum reported qc view
28     if max(tips) > max(qcs) then // TC created with high tip
29         i ← a validator such that tips[i] = max(tips) and no validator j satisfies tips[j] = max(tips) and
                qcs[j] > qcs[i];                    // select validator with highest tip and highest QC among ties
30         msg ← the timeout message from validator i in timeouts[view];
31         return Alg. 6.CreateTC(view, tips, msg.tip, qcs, ⊥, timeouts[view]), ⊥;
32     else // TC created with high QC
33         i ← a validator such that qcs[i] = max(qcs);
34         msg ← the timeout message from validator i in timeouts[view];
35         return Alg. 6.CreateTC(view, tips, ⊥, qcs, msg.qc, timeouts[view]), ⊥;
```

Algorithm 4: MonadBFT: Helper Functions [part 1 of 3]

Function IsFreshProposal(p)**:**
  return p.block.qc.view $=$ p.view $-\ 1$ **or** p.nec $\neq \perp$ **or** (p.tc $\neq \perp$ **and** p.tc.high_qc $\neq \perp$);

Function CommitAndSpecCommit(qc)**:** *// precondition guaranteed by caller:* Alg. 5.ValidQC(qc)
  parent $\leftarrow$ a proposal to which qc points; *// parent*.proposal_id $=$ qc.proposal_id
  if IsFreshProposal(*parent*) **then**
    speculatively commit *parent*.block;
  grandparent $\leftarrow$ a proposal to which *parent*.block.qc points;
  if qc.view $=$ *parent*.block.qc.view $+ 1$ **then**
    // if parent and grandparent are from consecutive views, commit grandparent
    commit *grandparent*.block;

Function CreateProposalBasedOnQC(qc)**:** *// precondition guaranteed by caller:* Alg. 5.ValidQC(qc)
  $p \leftarrow$ Alg. 6.BuildProposal(cur_view, $\perp$, qc, $\perp$, $\perp$); *// produce a fresh proposal*
  broadcast p;

Function CreateProposalBasedOnTC(tc)**:** *// precondition guaranteed by caller:* Alg. 5.ValidTC(tc)
  if tc.high_qc $\neq \perp$ **then**
    // produce a fresh proposal via the high QC
    $p \leftarrow$ Alg. 6.BuildProposal(cur_view, $\perp$, tc.high_qc, tc, $\perp$);
  else if the block B with B.block_hash $=$ tc.high_tip.block_header.block_hash is available **then**
    $p \leftarrow$ Alg. 6.BuildProposal(cur_view, B, $\perp$, tc, $\perp$); *// produce a reproposal of the high tip*
  else
    $(p_{old}, nec) \leftarrow$ Alg. 7.Recover(tc); *// send request to recover the missing high tip block*
    if $nec \neq \perp$ **then**
      // the block could not be retrieved — produce a fresh proposal
      $p \leftarrow$ Alg. 6.BuildProposal(cur_view, $\perp$, tc.high_tip.block_header.qc, tc, nec);
    else
      // the block could be retrieved — repropose it
      $p \leftarrow$ Alg. 6.BuildProposal(cur_view, p_{old}.block, $\perp$, tc, $\perp$); *// produce a reproposal of the high tip*
  broadcast p;

Function HandleVote($vote$)**:** *// precondition guaranteed by caller:* Alg. 5.ValidVote($vote$)
  view $\leftarrow$ $vote$.view; *proposal_id* $\leftarrow$ $vote$.proposal_id;
  votes[*proposal_id*] $\leftarrow$ votes[*proposal_id*] $\cup \{vote\}$; *// individual votes*
  if |votes[*proposal_id*]| $= 2f + 1$ **then**
    $qc \leftarrow$ Alg. 6.CreateQC(votes[*proposal_id*]);
    return qc;
  return $\perp$;

Function HandleTimeoutVote($vote$)**:** *// precondition guaranteed by caller:* Alg. 5.ValidVote($vote$)
  // identical to HandleVote*, but exclusively for votes embedded in timeout messages*
  view $\leftarrow$ $vote$.view; *proposal_id* $\leftarrow$ $vote$.proposal_id;
  timeout_votes[*proposal_id*] $\leftarrow$ timeout_votes[*proposal_id*] $\cup \{vote\}$; *// votes embedded in timeout messages*
  if |timeout_votes[*proposal_id*]| $= 2f + 1$ **then**
    $qc \leftarrow$ Alg. 6.CreateQC(timeout_votes[*proposal_id*]);
    return qc;
  return $\perp$;

Function BroadcastBackupQC(qc)**:**
  broadcast qc, if not already broadcast;

Function ForwardBackupQCToNextLeader(qc)**:**
  send qc to the leader of view qc.view $+ 1$, if not already sent;

Algorithm 5: MonadBFT: Helper Functions [part 2 of 3, continues on next page]

Function ValidBlock(B)**:**
  return B.payload_hash = Hash(B.payload) **and** ValidBlockHeader(the block header of B);

Function ValidBlockHeader(H)**:**
  return ValidQC(H.qc) **and** H.block_hash = Hash(H.block_view, H.payload_hash, H.qc);

Function ValidVote($vote$)**:**
  require $vote$.proposal_id = Hash($vote$.block_hash, $vote$.view);
  return $vote.\sigma$ is a valid signature on $\langle vote$.view, $vote$.block_hash, $vote$.proposal_id$\rangle$;

Function ValidQC(qc)**:**
  require qc.proposal_id = Hash(qc.block_hash, qc.view);
  return $qc.\Sigma$ aggregates $2f + 1$ valid signatures on $\langle qc$.view, qc.block_hash, qc.proposal_id$\rangle$;

Function ValidProposal(p)**:**
  require ValidBlock(p.block) **and** p.proposal_id = Hash(p.block.block_hash, p.view);
  require $p.\sigma$ is a valid signature on p.proposal_id of the leader of view p.view;
  require p.view $\geq p$.block.block_view **and** p.view $> p$.block.qc.view;
  if Alg. 4.IsFreshProposal(p) **then**
    return ValidFreshTip(the tip of p); *// verify the validity of the tip*
  // not fresh; verify that p.tc.high_tip has been reproposed
  require p.view $> p$.block.block_view;
  require p.tc $\neq \perp$ **and** ValidTC(p.tc) **and** p.view $= p$.tc.view $+ 1$ **and** p.tc.high_tip $\neq \perp$;
  return the block header of p.block is equal to p.tc.high_tip.block_header;

Function ValidFreshTip(T)**:**
  require ValidBlockHeader(T.block_header);
  require T.proposal_id = Hash(T.block_header.block_hash, T.view);
  require $T.\sigma$ is a valid signature on T.proposal_id of the leader of view T.view;
  require T.block_header.block_view $= T$.view;
  require T.view $> T$.block_header.qc.view;
  if T.view $= T$.block_header.qc.view $+ 1$ **then**
    return T.tc $= \perp$ **and** T.nec $= \perp$; *// tip constructed on the happy path*
  require T.tc $\neq \perp$ **and** ValidTC(T.tc) **and** T.view $= T$.tc.view $+ 1$;
  if T.nec $= \perp$ **then**
    // tip constructed using the high QC from the TC
    return T.tc.high_qc $\neq \perp$ **and** T.block_header.qc $= T$.tc.high_qc;
  require ValidNEC(T.nec); *// otherwise, the fresh tip is constructed with an NEC*
  return T.view $= T$.nec.view **and** T.block_header.qc.view $= T$.nec.high_tip_qc_view;

Function ValidTimeoutMessage(M)**:**
  if M.last_cer is a QC **then require** ValidQC(M.last_cer); **else require** ValidTC(M.last_cer);
  require M.last_cer.view $= M$.view $- 1$;
  require (M.tip $\neq \perp$ **and** M.qc $= \perp$) **or** (M.tip $= \perp$ **and** M.qc $\neq \perp$); *// must have exactly one*
  if M.tip $\neq \perp$ **then**
    require M.tip_vote $\neq \perp$ **and** ValidVote(M.tip_vote) **and** M.tip_vote.view $= M$.view;
    require ValidFreshTip(M.tip) **and** M.tip.view $\leq M$.view;
    return $M.\sigma$ is a valid signature on $\langle M$.view, M.tip.view, M.tip.block_header.qc.view$\rangle$;
  require ValidQC(M.qc) **and** M.qc.view $< M$.view;
  return $M.\sigma$ is a valid signature on $\langle M$.view, $\perp$, M.qc.view$\rangle$;

Algorithm 5: MonadBFT: Helper Functions [continuation of part 2 of 3]

Function ValidTC(tc)**:**
  require tc.high_tip $\neq \perp$ **or** tc.high_qc $\neq \perp$;
  require $tc.\Sigma$ aggregates $2f+1$ valid signatures on tc.view, tc.tips_views and tc.qcs_views;
  if tc.high_qc $\neq \perp$ **then**
    require tc.high_tip $= \perp$ **and** ValidQC(tc.high_qc) **and** tc.high_qc.view $< tc$.view;
    return $\max(tc$.tips_views$) \leq tc$.high_qc.view **and** tc.high_qc.view $= \max(tc$.qcs_views$)$;
  require ValidFreshTip(tc.high_tip) **and** tc.high_tip.view $\leq tc$.view;
  $tips \leftarrow$ tc.tips_views; $qcs \leftarrow tc$.qcs_views;
  for every validator i such that $tips[i] \neq \perp$, **require** $qcs[i] < tips[i]$;
  require not exists i such that $tips[i] > tc$.high_tip.view; // *no more recent tip than* tc.high_tip
  // *ties must be broken by view of QC*
  require not exists i such that $tips[i] = tc$.high_tip.view and $qcs[i] > tc$.high_tip.block_header.qc.view;
  return tc.high_tip.view $> \max(qcs)$;

Function ValidNEC(nec)**:**
  require nec.high_tip_qc_view $< nec$.view $- 1$;
  return $nec.\Sigma$ aggregates $2f+1$ valid signatures on $\langle nec$.view, nec.high_tip_qc_view$\rangle$;

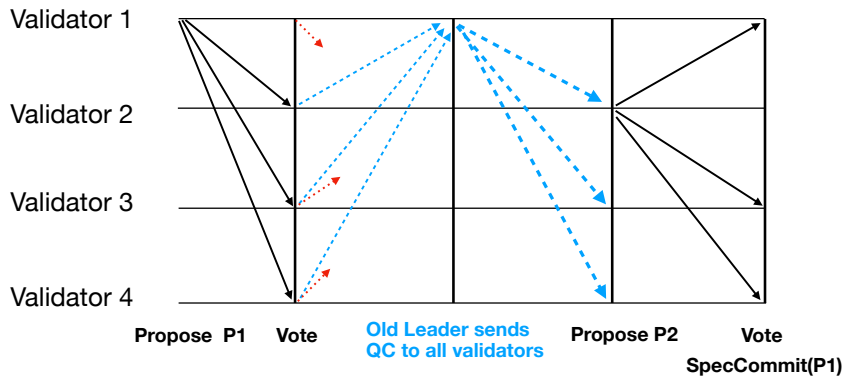


Fig. 5: Happy path workflow in MonadBFT using backup QC. In this scenario, votes sent to the current leader, Validator 2, are arbitrarily delayed (dotted, red), preventing the formation of a QC. In this case, the old leader, Validator 1, will construct a QC using the backup votes (dashed, blue) and broadcast it to all validators, including the next leader (bold dashed, blue). This enables the next leader to extend the proposal. Although some messages are lost, this example belongs to the *happy path*, as a QC is successfully constructed and a timeout is avoided. Validators 3 and 4 also forward the backup QC received from Validator 1 to Validator 2. For simplicity, we omit this, as it is not required in this example.

(2) Each validator, upon receiving a proposal p_{v+1} from L_{v+1} that extends p_v via a QC from view v, forwards the QC p_{v+1}.block.qc to L_v. Upon receiving this QC, L_v treats it as its backup QC and broadcasts it to all validators. x

Lastly, we add the following changes to ensure the leader fault isolation property when L_v is Byzantine but L_{v+1} is honest:

(3) Each validator, upon receiving a backup QC for p_v, increments its view, and forwards the QC to L_{v+1} unless it has already received a valid proposal from L_{v+1}.

Algorithm 6: MonadBFT: Helper Functions [part 3 of 3]

```
1  Function BuildProposal(view, B, qc, tc, nec):
2      if B = ⊥ then
3          // fresh proposal request; build a new block
4          block_view ← view;
5          payload ← fill with mempool transactions;
6          payload_hash ← Hash(payload);
7          block_hash ← Hash(block_view, payload_hash, qc);
8          B ← ⟨block_view, payload, payload_hash, qc, block_hash⟩;
9      proposal_id ← Hash(B.block_hash, view);
10     σ ← Sign(proposal_id);
11     return ⟨view, proposal_id, B, σ, tc, nec⟩;
12 Function GetTip(p):
13     if Alg. 4.IsFreshProposal(p) then
14         return the tip of p;
15     return p.tc.high_tip;
16 Function CreateVote(view, H):
17     proposal_id ← Hash(H.block_hash, view);
18     σ ← Sign(view, H.block_hash, proposal_id);
19     return ⟨view, H.block_hash, proposal_id, σ⟩;
20 Function CreateQC(votes_proposal_view):
21     Σ ← aggregate signatures from all votes in votes_proposal_view;
22     vote ← any vote in votes_proposal_view;
23     return ⟨vote.view, vote.block_hash, vote.proposal_id, Σ⟩;
24 Function CreateTC(view, tips, high_tip, qcs, high_qc, timeouts):
25     return ⟨view, tips, high_tip, qcs, high_qc, aggregate signatures from all messages in timeouts⟩;
26 Function CreateTimeoutMessage(view):
27     if last_qc.view = view − 1 then
28         view_cer ← last_qc;
29     else
30         view_cer ← last_tc;
31     if local_tip.view ≤ last_qc.view then
32         σ ← Sign(view, ⊥, last_qc.view);
33         return ⟨view, ⊥, ⊥, last_qc, view_cer, σ⟩;
34     else
35         vote ← CreateVote(view, local_tip.block_header);
36         σ ← Sign(view, local_tip.view, local_tip.block_header.qc.view);
37         return ⟨view, local_tip, vote, ⊥, view_cer, σ⟩;
38 Function CreateNEC(view, qc_view, sigs):
39     return ⟨view, qc_view, aggreagate signatures from sigs⟩;
```

Algorithm 7: MonadBFT: Block Recovery

Function RECOVER(tc)**:**
  $X \leftarrow$ any set of κ validators; *// optimization: prefer validators who reported tc.*`high_tip`
  send $\langle$ProposalRequest, $tc\rangle$ to all validators in X;
  NEset $\leftarrow \emptyset$; *requested* $\leftarrow X$;
  broadcast $\langle$NERequest, $tc\rangle$ to all validators;
  // function proceeds asynchronously until a proposal or NEC is returned, or RECOVER *is called with*
  // a TC from a higher view
  upon receiving $\langle$ProposalResponse, $p\rangle$ **do**
    require p.`proposal_id` = tc.`high_tip.proposal_id`; *// if false, drop message, but don't return*
    return $(p, \perp)$;
  upon first receipt of $ne_i = \langle \mathsf{NE}, \sigma_i \rangle$ from validator j **do**
    // if condition is false, drop message, but don't return
    require σ_i is a valid signature on $\langle tc.\texttt{view} + 1, tc.\texttt{high_tip.block_header.qc.view}\rangle$ of validator j;
    NEset $\leftarrow$ *NEset* $\cup \{ne_i\}$;
    if $|NEset| = 2f + 1$ **then**
      return ($\perp$, Alg. 6.CREATENEC(tc.`view` + 1, tc.`high_tip.block_header.qc.view`, *NEset*));
  upon at regular intervals of duration Θ_{interval} **do**
    $X \leftarrow$ any κ-sized subset of (the set of all validators $\setminus$ *requested*);
    send $\langle$ProposalRequest, $tc\rangle$ to validators in X;
    requested $\leftarrow$ *requested* $\cup X$;
Function PROCESSPROPOSALREQUEST(tc)**:**
  if i has cached a proposal p with p.`proposal_id` = tc.`high_tip.proposal_id` **then**
    send $\langle$ProposalResponse, $p\rangle$ to the leader of view tc.`view` + 1;
Function PROCESSNEREQUEST(tc)**:**
  if i has not voted for a proposal p with p.`proposal_id` = tc.`high_tip.proposal_id` **and** has not sent an NE message in view tc.`view` + 1 **then**
    $\sigma \leftarrow$ SIGN(tc.`view` + 1, tc.`high_tip.block_header.qc.view`);
    send $\langle \mathsf{NE}, \sigma\rangle$ to the leader of view tc.`view` + 1;

This rule prevents a malicious L_v from triggering a view change at all validators (except the leader) to view v+1 much earlier than L_{v+1}, which would effectively shorten L_{v+1}'s timeout window (view duration) and could lead to unnecessary timeouts in view v+1.

3.2 Unhappy Path

The unhappy path handles recovery when a view fails to progress—typically due to a faulty leader or prolonged network delays. Once $2f + 1$ validators declare the view failed, MonadBFT transitions to the unhappy path[4], enabling an honest leader to recover while ensuring tail-forking resistance.

Timeout messages. Let v be the current view. If a validator does not enter the next view $v + 1$ within a predetermined timeout period after entering view v, it broadcasts a timeout message, signaling that view v has failed from its perspective. Recall that a timeout message includes: (1) the current view, (2) the local tip, (3) a vote for the local tip, (4) the most recent observed QC (i.e., the local high QC), (5) the QC or TC that caused the validator to enter view v, and (6) a signature.

[4] We use the terms *happy* and *unhappy path* only for explanatory purposes; the actual protocol does not have such an unambiguous global state, as there may be divergent local perspectives by different validators—for instance, when one validator receives a proposal, but another does not and times out.

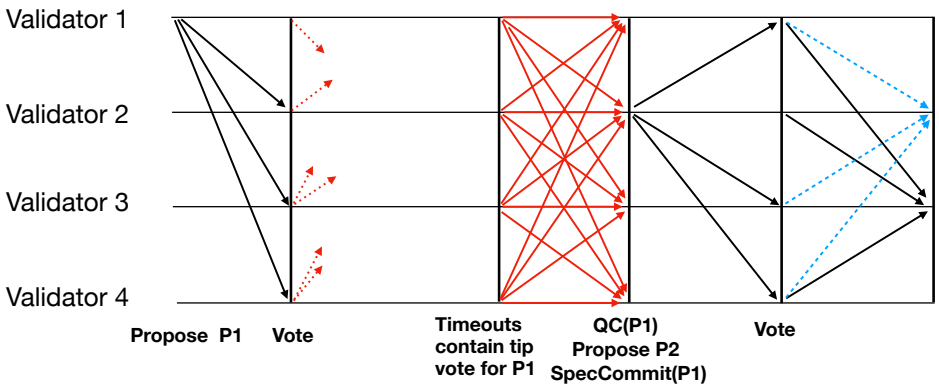


Fig. 6: Unhappy path, but fast recovery workflow in MonadBFT, illustrating formation of a QC from tip votes. In this scenario, votes for proposal P_1, sent to both the current and next leader (dotted, red), are delayed. Consequently, no QC can be build for the proposal and the view times out. Upon timing out, validators broadcast timeout messages (solid, red), which also include their votes for P_1. The next leader (validator 2) then aggregates these votes to form a QC for P_1 and extends it with a new proposal P_2. This example also illustrates that a validator can cast both a vote and a timeout message in the same view and that both a QC and a TC can be formed in the same view.

To create the timeout message, each validator i maintains its local tip T_i and the most recent QC qc_i it has seen, and checks whether $qc_i.\texttt{view} \geq T_i.\texttt{view}$. If so, the validator sets $(\texttt{qc}, \texttt{tip}, \texttt{tip_vote}) \leftarrow (qc_i, \bot, \bot)$. Otherwise, the validator sets the $(\texttt{qc}, \texttt{tip}) \leftarrow (\bot, T_i)$. In the latter case, the validator also populates the `tip_vote` field with a vote for T_i.

Fast recovery: Building a QC. Recall that in the happy path, the leader L_{v+1} of the view $v+1$ directly collects votes and builds a QC for the proposal of view v. Even when this fails, in two cases a QC for the same proposal can be constructed, allowing the next honest leader to quickly recover from the failure and make a regular fresh proposal:

- **Case 1:** The previous leader L_v constructs a QC for its own proposal, broadcasts it, and it is received by the next honest leader.
- **Case 2:** Enough timeout messages from view v contain *tip votes*, and they can be used to construct a QC for it.

3.2.1 Processing Timeout Messages

If constructing or obtaining a QC from view v is not possible, the leader L_{v+1} waits until $2f+1$ timeout messages are received and then builds a TC. Let `tips_views` and `qcs_views` be the set of views of local tips and local high QCs reported in the timeout messages, respectively. L_{v+1} creates a TC tc as:

$$tc.\texttt{view} \leftarrow v; \quad tc.\texttt{tips_views} \leftarrow \texttt{tips_views}; \quad tc.\texttt{qcs_views} \leftarrow \texttt{qcs_views}.$$

While creating tc, L_{v+1} uses the signatures σ included in the timeout messages to compute $tc.\Sigma$, and justify $tc.\texttt{tips_views}$ and $tc.\texttt{qcs_views}$.

Let T and qc be the highest tip and local high QC (ordered by view number) reported in these timeout messages. If multiple such tips have the same maximum view, the tip that contains the highest QC must be chosen (as validated by ValidTC in Alg. 5). If $qc.\texttt{view} \geq T.\texttt{view}$, L_{v+1} sets $(\texttt{tc.high_tip}, tc.\texttt{high_qc}) \leftarrow (\bot, qc)$, otherwise, L_{v+1} sets $(\texttt{tc.high_tip}, tc.\texttt{high_qc}) \leftarrow (T, \bot)$.

Now, depending upon the TC, the following cases can occur.

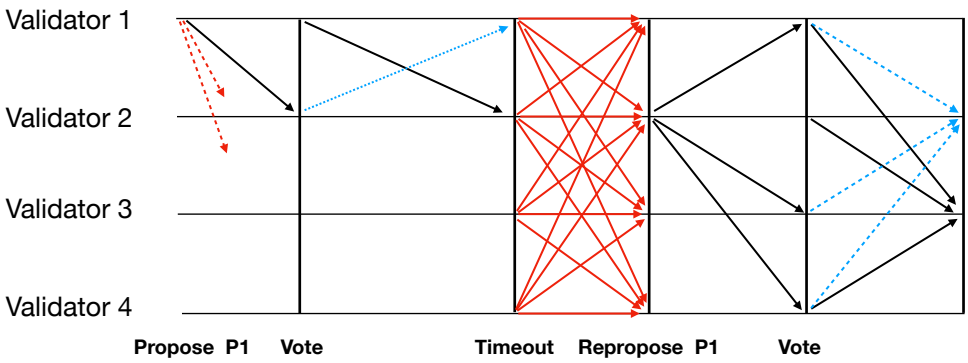


Fig. 7: Unhappy path workflow in MonadBFT using standard recovery. In this example, the proposal P1 is not received by sufficiently many validators to form a QC, neither via regular (solid, black), nor backup (dashed, blue), nor tip votes (solid, red). P1 appears as the high tip of the timeout certificate constructed. It is then reproposed by the next leader.

Case 3: If $qc.\texttt{view} \geq T.\texttt{view}$, L_{v+1} proposes a fresh proposal p with a new block B with $B.\texttt{qc} \leftarrow qc$ and:

$$p.\texttt{view} \leftarrow v+1; \quad p.\texttt{block} \leftarrow B; \quad p.\texttt{tc} \leftarrow tc; \quad p.\texttt{nec} \leftarrow \perp.$$

This is a fast recovery path since the TC itself directly implies that no reproposal is needed. We now turn to the two standard recovery cases.

Case 4: If $T.\texttt{view} > qc.\texttt{view}$, L_{v+1} then checks if it possesses the block B corresponding to the tip T. If it does, L_{v+1} reproposes the block B in a proposal p where:

$$p.\texttt{view} \leftarrow v+1; \quad p.\texttt{block} \leftarrow B; \quad p.\texttt{tc} \leftarrow tc; \quad p.\texttt{nec} \leftarrow \perp.$$

Case 5: Otherwise, i.e., if L_{v+1} does not have the block B corresponding to the tip T, the leader invokes the block recovery part of the consensus protocol. As we discuss in Section 3.4, this recovery either returns the block B or a no-endorsement certificate (NEC), certifying that the block B is not recoverable.

If the block recovery returns B, L_{v+1} reproposes B exactly as in case 4. Otherwise, let *nec* be the NEC returned by the block recovery. L_{v+1} then issues a fresh proposal p with a new block B' with $B'.\texttt{qc} \leftarrow T.\texttt{qc}$, and:

$$p.\texttt{view} \leftarrow v+1; \quad p.\texttt{block} \leftarrow B'; \quad p.\texttt{tc} \leftarrow tc; \quad p.\texttt{nec} \leftarrow nec.$$

3.2.2 Processing Proposals in the Unhappy Path

This section will describe how the validators process the proposals they receive in the unhappy path.

A validator processes QCs and TCs to advance views regardless of how they are obtained – whether constructed locally or received directly or indirectly as part of a proposal, timeout message, or tip. Moreover, upon receiving any valid message, each validator executes the pacemaker logic first (Section 3.3), before using the message for other protocol actions.

Case 1 & 2: During view $v+1$, if a validator receives a proposal p from L_{v+1} with $p.\texttt{view} = v+1$ and $p.\texttt{block.qc.view} = v$, the validator processes p exactly as in the happy path (see Section 3.1).

Case 3: Alternatively, upon receiving a proposal p with:

$$p.\texttt{tc} \neq \perp; \quad p.\texttt{tc.high_tip} = \perp; \quad p.\texttt{tc.high_qc} \neq \perp; \quad p.\texttt{nec} = \perp,$$

each validator checks that the proposal p is consistent with $p.\texttt{tc}$ using the ValidProposal() function in Algorithm 5. Intuitively, these checks ensure that L_{v+1} followed the procedure in case 3 of Section 3.2.1 to create p. Additionally, the validator checks that: (i) $p.\texttt{tc.high_qc} = p.\texttt{block.qc}$, i.e., the proposed block extends the block corresponding to $p.\texttt{tc.high_qc}$; and (ii) $p.\texttt{tc.high_qc.view} < p.\texttt{view} - 1$. Check (ii) here prevents L_{v+1} from triggering case 3, when L_{v+1} could have created p as per case 1 and 2 in Section 3.2.1.

If all these checks are successful, as in the happy path, the validator: (i) sends its vote for p to both L_{v+1} and the leader of view $v+2$; and (ii) updates its local tip to the tip of proposal p.

Case 4: Alternatively, upon receiving a proposal p with:

$$p.\texttt{tc} \neq \bot; \quad p.\texttt{tc.high_tip} \neq \bot; \quad p.\texttt{tc.high_qc} = \bot; \quad p.\texttt{nec} = \bot,$$

as in the previous case, each validator checks that the proposal p is consistent with $p.\texttt{tc}$ to ensure that L_{v+1} followed the procedure in case 4 of Section 3.2.1 to create p. Moreover, if all these checks are successful, the validator casts votes. This is also the only scenario in which the local tip is *not* updated to the tip of proposal p. Since p is a reproposal, the local tip is instead set to the high tip carried in the included TC. Note that p and the high tip refer to the same block, as p is a valid reproposal of that high tip.

Case 5: There are two possibilities here, depending upon whether $p.\texttt{nec}$ is set to $\bot$ or not. If $p.\texttt{nec} = \bot$, the validator processes this proposal exactly as in case 4 above. Otherwise, the validator checks that: (i) $p.\texttt{view} = p.\texttt{nec.view} = v+1$, i.e., the NEC is from the current view, (ii) $p.\texttt{block.qc.view} = p.\texttt{nec.high_tip_qc_view}$, i.e., the block extends the parent of the high tip that was requested by the leader, and, (iii) $p.tc \neq \bot$ and valid. If all these checks are valid, the validator casts votes and updates its local tip to the tip of proposal p.

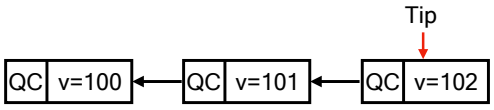


Fig. 8: The tip points to the latest proposal.

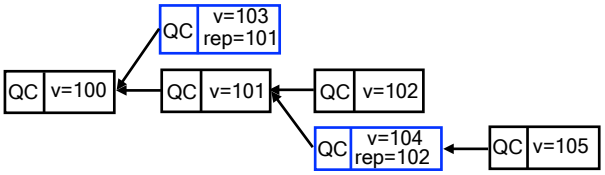


Fig. 9: Fresh proposals and reproposals.

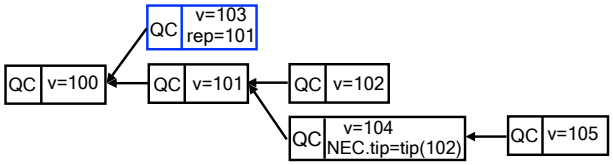


Fig. 10: NEC allowing a leader to make a fresh proposal

The figures illustrate the selection of the `high_tip` and the subsequent proposal processes during view failures. Blue rectangles indicate reproposals, while black rectangles indicate fresh proposals.

Remark on the recursive structure of tips. At first glance, the definition of tips appears recursive: a tip may contain a TC, which in turn may contain a tip, and so on. In practice, this recursion can be trivially avoided. Consider any tip T with $T.\texttt{tc} \neq \bot$. We distinguish two cases:

- If $T.\texttt{nec} \neq \bot$, then the TC can be safely discarded (by setting $T.\texttt{tc} = \bot$). Since a valid NEC already provides all the information needed to validate the tip, the embedded TC becomes redundant and the recursive reference can be safely removed.
- If $T.\texttt{nec} = \bot$, then $T.\texttt{tc}$ necessarily contains a nonempty `high_qc` (i.e., $T.\texttt{tc.high_qc} \neq \bot$), which in turn implies $T.\texttt{tc.high_tip} = \bot$. Thus, the TC does not contain another tip, so no recursion occurs.

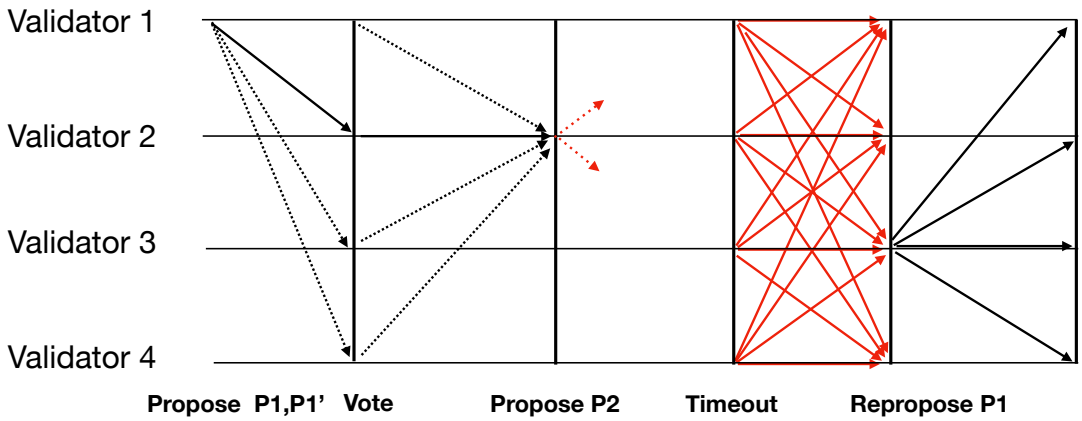


Fig. 11: Example to illustrate that tail-forking cannot be ruled out under leader equivocation. Validator 1, a malicious leader, makes two equivocating fresh proposals, sending P1 (solid, black) to validator 2 and a conflicting proposal P1' (dashed, black) to validators 3 and 4 (f+1 honest validators). Each validator votes for the proposal they received and sends their vote to the next leader, validator 2 (the figure omits backup votes). Validator 2 collects enough votes to form a QC for P1', but it fails to send proposal P2, causing validators to send timeout messages containing their highest observed tips. Validators 3 and 4 send timeout message including the tip of P1' as the highest tip. Validator 2 and the Byzantine validator 1 send timeout messages including the tip of P1 as the highest tip. Validator 3 builds a TC containing both P1 and P1' as high tips. Since both are proposals building on the same QC, validator 3 can freely choose to repropose one of them. In this example, validator 3 reproposes P1, leading P1' to be abandoned despite having obtained a QC.

Hence, although the data type permits a recursive structure, valid tips that arise during the protocol can always be represented in a non-recursive form. For simplicity, we leave the recursive definition in the presentation, but in the actual MonadBFT implementation these recursions are eliminated.

Examples of tip handling & reproposal dynamics. Figure 8 illustrates the local tip held by a validator. Figure 9 depicts multiple view failures: first, views 100 and 101 succeed, i.e., QC is formed for the blocks proposed in these views. Next, view 102 fails, and the tip from view 101 is selected as the high tip and reproposed in view 103. The proposal from view 102 did not propagate to a sufficient number of validators and, consequently, was not selected as the high tip in view 103. Following the failure of view 103, the tip from view 102 resurfaces as part of the subsequent TC and is selected as the high tip for view 104. Note that the tip from view 103 is not considered a candidate for the high tip, as only fresh proposals are eligible. The tip from view 102 is then reproposed in view 104, and view 104 succeeds.

Figure 10 depicts an alternative scenario for view 104. After the failure of view 103, the leader does not hold the block corresponding to the high tip (i.e., the tip associated with the fresh proposal of view 102) and therefore requests it using the block recovery protocol. The leader of view 104 subsequently receives an NEC for the high tip from view 102, enabling it to issue a fresh proposal in view 104 instead of reproposing the block from view 102. The NEC includes the view of the highest observed QC, view 101, which the fresh proposal must extend; in other words, the parent of the high tip from view 102 becomes the parent of the newly issued fresh proposal.

3.2.3 Tail-forking Resistance and Equivocation

Recall that tail-forking resistance is guaranteed only for blocks that receive at least $f + 1$ votes from honest validators and whose issuing leader has not equivocated (see Section 2.2). In this section, we analyze how

equivocation interacts with tail-forking resistance, in particular how equivocation can cause a block that already carries a QC to be reverted. We illustrate this in Figure 11.

First, observe that equivocation by itself does not compromise the voting process: in particular, it cannot lead to two conflicting QCs for the same view. This is because each correct validator casts at most *one* vote per view—that is, it votes for at most one proposal in any view. Once a validator has voted in a given view, all subsequent proposals for that view are rejected (enforced by the highest_voted_view variable; see the check in Algorithm 1, line 12).

The critical point arises when considering how equivocation can interact with the protocol's recovery mechanisms whose goal is to prevent tail-forking attacks. Suppose the leader of some view v equivocates by issuing multiple proposals $p_{set} = \{p_1, p_2, \ldots, p_k\}$ $(k > 1)$ for the same height in the blockchain. As argued above, at most one proposal $p^\star \in p_{set}$ can gather a QC. We now distinguish two cases:

- *Case 1: Sufficient QC dissemination.* If, by the "backup voting" mechanism, at least $f + 1$ honest validators receive the QC for $p^\star$ while still in view v, then all of them advance to view $v+1$ without ever issuing a timeout message for view v. Consequently, no TC for view v can be formed. This forces every proposal in view $v+1$ to carry a QC from view v, which in turn ensures that it must extend $p^\star$, thereby preserving tail-forking resistance.
- *Case 2: Insufficient QC dissemination.* If fewer than $f + 1$ honest validators receive the QC for $p^\star$, then a TC for view v may be formed. In that case, although $p^\star$'s tip must still be present among the $2f + 1$ timeout messages, the subsequent leader may also observe some other conflicting proposal from p_{set} and choose to repropose it. This effectively abandons $p^\star$, even though a QC for it exists, thereby breaking tail-forking resistance. For this reason, MonadBFT guarantees tail-forking resistance only when the issuing leader has not equivocated.

3.3 Pacemaker

The pacemaker is a coordination protocol that validators use to advance views once the network becomes synchronous. Specifically, it guarantees that after GST, if a correct validator enters view v, then all correct validators will enter view v or higher within a bounded time. In MonadBFT, we implement the pacemaker using quorum certificates (QCs), timeout certificates (TCs), and Bracha-style amplification based on timeout messages [5, 6]. We summarize the pacemaker protocol in Alg. 3 and describe it below.

Consider validator i whose current view is v. Then, validator i performs the following actions:

1. Upon receiving a QC qc with $qc.\texttt{view} \geq v$, (i) forwards qc to the leader of view $qc.\texttt{view}+1$, (ii) updates its local view to $qc.\texttt{view}+1$, and (iii) updates its local high QC to qc.
2. Upon receiving a TC tc with $tc.\texttt{view} \geq v$, (i) broadcasts a timeout message for view $tc.\texttt{view}$, unless it has already done so, and (ii) updates its local view to $tc.\texttt{view}+1$.
3. Upon receiving $f+1$ timeout messages for some view $v' \geq v$, broadcasts its own timeout message for view v', unless it has already done so. Validator i uses a QC and TC included in any of these $f+1$ timeout message to create its timeout message. This step implements the Bracha amplification mechanism [5, 6].
4. Upon receiving $2f+1$ timeout messages for some view $v' \geq v$, constructs a TC tc and processes it as in step 2 above.

3.4 Block Recovery

Recall from Section 3.2.1, during the unhappy path, a leader L in view v collects a TC tc and in certain scenarios, reproposes the block corresponding to the high-tip in tc. Let p be the proposal corresponding to $tc.\texttt{high_tip}$, and say L seeks to repropose the block $p.\texttt{block}$. Now, if L does not possess the $p.\texttt{block}$, it invokes RECOVER(tc) to recover p and consequently the block $p.\texttt{block}$ or a NEC certifying that $p.\texttt{block}$ need not be reproposed. We specify the recovery protocol in Alg. 7, and describe it next.

- The leader L sends the $\langle \mathsf{ProposalRequest}, tc \rangle$ message to batches of κ validators, prioritizing those who included tip $tc.\texttt{high_tip}$ in their timeout messages. L continues to periodically send $\langle \mathsf{ProposalRequest}, tc \rangle$ to additional validators, until it recovers a proposal or an NEC. Upon receiving $\langle \mathsf{ProposalRequest}, tc \rangle$, each validator validates tc and checks that L is the designated leader for $tc.\texttt{view}$. Upon successful validation, the validator increments its view, and if it possesses the proposal p, sends p back to L.
- Simultaneously, L broadcasts the $\langle \mathsf{NERequest}, tc \rangle$. Upon receiving $\langle \mathsf{NERequest}, tc \rangle$, each validator validates the $\mathsf{NERequest}$ similarly to $\mathsf{ProposalRequest}$. Upon successful validation, the validator increments its view, and responds with a no endorsement (NE) message $\langle \mathsf{NE}, \sigma \rangle$ unless it has previously cast a vote for p. Here, σ is a digital signature on the tuple $\langle tc.\texttt{view} + 1, tc.\texttt{high_tip.block_header.qc.view} \rangle$.
- The leader, upon receiving $2f + 1$ valid NE messages, aggregates the signatures to create an NEC.
- The block recovery protocol terminates when either L receives the proposal p or constructs an NEC.

Remark 1. Note that it is possible to simultaneously recover proposal p and construct an NEC if fewer than $f + 1$ honest validators hold the proposal. The presence of an NEC does not imply that the block is unrecoverable; rather, it indicates that fewer than $f + 1$ honest validators obtained it during its propagation. Consequently, the block can be safely abandoned without compromising the tail-forking resistance property.

Remark 2. Note that, in the worst case, the recovery protocol we describe incurs $O(n/\kappa)$ latency. This can happen when only $O(1)$ correct validators have the block, and f malicious validators who claim to have the proposal in their timeout messages do not assist the leader to recover p. Although it is possible to improve the protocol using erasure codes, we opt for this simpler protocol as we envision that such situations will occur rarely in practice.

4 Analysis

This subsection provides an overview and intuition of how and why MonadBFT satisfies its properties. Our goal is to develop intuition for the core mechanisms of MonadBFT and how they guarantee these properties, and we therefore include several detailed explanations. The full proofs can be found in Section A.

Safety. Safety requires that no two correct validators commit conflicting blocks, that is, different blocks at the same height in their respective local logs. Informally, the commitment rule in MonadBFT states that a block is committed once a block has obtained a "QC of a QC" and both QCs are from consecutive views. More precisely, suppose a block B appears in a proposal p from view v. A correct validator commits B upon observing (i) a QC for p from view v, and (ii) a QC for a proposal p' from view $v + 1$ such that p' directly extends p (by carrying a QC for p). We now explain why, after this point, no conflicting block B' can ever be committed.

First, consider QCs formed in view v. Since $2f + 1$ validators (including at least $f + 1$ correct validators) vote for proposal p in view v, and correct validators never vote for two different proposals within the same view, it is impossible to collect $2f + 1$ votes in view v for any other proposal. Hence, the only QC that can be formed in view v is the QC for p. By the same quorum-intersection reasoning, the only proposal that can obtain $2f + 1$ votes in view $v + 1$ is the proposal p'. Thus, the only QC that can be formed in view $v + 1$ is the QC for p', which strictly extends proposal p. Therefore, in the two consecutive views responsible for committing B, the protocol permits only the QC for p in view v and only the QC for p' in view $v + 1$.

Next, consider any valid proposal p^* formed in view $v+2$. We show that such a proposal must necessarily strictly extend p, which already suffices to protect the commitment of B. There are two possibilities for p^*:

- *Case 1:* Proposal p^* is a fresh proposal carrying a QC from view $v + 1$. In view $v + 1$, the only QC that can form is the QC for p', and p' strictly extends p. Thus, proposal p^* carrying a QC from view $v + 1$ must strictly extend p.
- *Case 2:* Proposal p^* is either (i) a fresh proposal carrying an NEC from view $v + 2$ or carrying a TC from view $v + 1$ whose $\texttt{high_qc}$ field is non-$\perp$, or (ii) a reproposal carrying a TC from view $v + 1$. All of these possibilities imply the existence of a TC formed in view $v + 1$.

We first observe that any TC tc formed in view $v+1$ must satisfy two properties: (1) its `high_tip` field must be non-$\perp$ (and consequently its `high_qc` field must be $\perp$), and (2) this high tip must come from view $v+1$ and must carry a QC from view v. Indeed, at least $f+1$ correct validators (namely, those that voted for proposal p') update their local tip to the tip of p' in view $v+1$; recall that p' is a fresh proposal because it carries a QC from view v. Since TCs aggregate timeout messages from $2f+1$ validators, at least one of the updated tips from these correct validators must appear in any valid TC, and therefore in tc. Moreover, because the highest-view tip is selected as the high tip (breaking ties by choosing the tip that carries the most recent QC), the high tip of tc must be the tip from view $v+1$ that carries a QC from view v. Finally, because no conflicting QC can be formed in view v, the QC carried by this tip must be the QC for p, which implies that the high tip of tc strictly extends proposal p.
With this structure in place, any valid proposal p^* derived from a TC formed in view $v+1$ must also strictly extend p:

- If p^* is a fresh proposal carrying an NEC from view $v+2$, observe that forming an NEC requires at least $f+1$ correct validators to issue NE messages upon receiving a TC from view $v+1$. Each NE message identifies the view whose QC the leader must incorporate into the new proposal. Since every TC formed in view $v+1$ has a high tip carrying a QC from view v, the NEC forces p^* to include that QC, which in turn points to p. Thus, p^* strictly extends p.
- Note that p^* cannot carry a TC from view $v+1$ whose `high_qc` field is non-$\perp$, since we have established that every TC formed in view $v+1$ must use its high tip (and therefore has its `high_qc` field set to $\perp$).
- If p^* is a reproposal carrying a TC from view $v+1$, it reproposes the high tip of that TC. As shown above, that high tip strictly extends p, so p^* strictly extends p as well.

Thus, every valid proposal in view $v+2$ strictly extends p, thereby preserving the commitment of p. The same reasoning applies inductively to all views beyond $v+2$. Therefore, once B is committed, no future proposal can revert or conflict with p, and no conflicting block B' can ever be committed.

Tail-forking resistance. We now give an intuitive explanation of why MonadBFT satisfies tail-forking resistance. Assume a leader proposes a block B as part of a fresh proposal p in some view v, that at least $f+1$ correct validators vote for p, and that the leader does not equivocate in view v. Tail-forking resistance states that proposal p (equivalently, block B carried by p) can never be abandoned. As before, we illustrate the argument for view $v+1$; the reasoning then applies inductively to all later views.

Consider any valid proposal p^* issued in view $v+1$. We now show that p^* must extend p, meaning that p^* is either a reproposal of p or builds directly on top of p. There are several possibilities for the form of p^*:

- *Case 1:* Proposal p^* is a fresh proposal carrying a QC from view v. Since $f+1$ correct validators voted for p in view v and never vote for two different proposals in the same view, any QC formed in view v must be the QC for p. Therefore, the parent of proposal p^* must be p, which shows that p is not abandoned by p^*.
- *Case 2:* Proposal p^* is either (i) a fresh proposal carrying an NEC from view $v+1$ or carrying a TC from view v with a non-$\perp$ `high_qc` field, or (ii) a reproposal carrying a TC from view v. All of these possibilities rely on the existence of a TC formed in view v.
  We first show that any TC tc formed in view v must satisfy two conditions: (1) its `high_tip` field must be non-$\perp$ (implying that its `high_qc` field is $\perp$), and (2) this high tip must be the tip of proposal p. Indeed, at least $f+1$ correct validators (precisely those who voted for p) update their local tip to the tip of p in view v. Since a TC aggregates timeout messages from $2f+1$ validators, at least one of these updated tips must appear in any valid TC and therefore in tc. Furthermore, because the highest-view tip is chosen as the high tip and the leader does not equivocate in view v, the high tip of tc must be the tip of p.
  This implies that any valid proposal p^* derived from a TC formed in view v must extend p:
  - If p^* is a fresh proposal carrying an NEC from view $v+1$, observe that forming an NEC requires at least $f+1$ correct validators to issue NE messages upon receiving a TC from view v. Since this TC has the tip of p as its high tip and at least $f+1$ correct validators supported p in view v, none

of these validators issue an NE message. Consequently, such an NEC cannot be constructed, making this case impossible.
- Proposal p^* cannot carry a TC from view v whose `high_qc` field is non-$\perp$, since we have already shown that every TC from view v must use its high tip and therefore has its `high_qc` field set to $\perp$.
- If p^* is a reproposal carrying a TC from view v, it must repropose the high tip of that TC. As argued above, that high tip is exactly the tip of proposal p, so p^* reproposes p (its block B).

Liveness. We now explain how MonadBFT guarantees that every block proposed by a correct leader after GST is eventually committed. Consider a proposal p (fresh or a reproposal) issued by a correct leader in some post-GST view v. Since the network is stable after GST, the leader disseminates p to all validators sufficiently quickly so that all correct validators issue their votes for p. Because votes are sent both to the current leader and to the next leader, the leader of view v receives at least $2f + 1$ votes for p and constructs a QC for it. This QC is then broadcast to all validators, and every correct validator receives a QC before timing out from view v. Consequently, no TC can be formed in view v. This fact is essential: it implies that any valid proposal in any later view must strictly extend p, meaning that p remains on the chain of all future valid proposals.

Eventually, there must be three consecutive correct leaders in views strictly greater than v. Let these views be v^*, $v^* + 1$, and $v^* + 2$, all greater than v. Since any valid proposal after view v must extend p, the proposal issued in view v^* necessarily extends p. Because we are post-GST, the leader of view v^* successfully disseminates its proposal to all validators, who all vote for it. Thus, the leader of view $v^* + 1$ collects these votes and forms a QC for the proposal from view v^*. The leader of view $v^* + 1$ then issues a proposal extending the view-v^* proposal via this QC; clearly, this proposal from view $v^* + 1$ also strictly extends p. All correct validators vote for this view-$(v^* + 1)$ proposal as well, ensuring that the leader of view $v^* + 2$ forms a QC for the proposal from view $v^* + 1$ and issues a new proposal carrying that QC. When a correct validator receives the proposal from view $v^* + 2$, it observes two consecutive QCs for the view-v^* proposal: one from view v^* and one from view $v^* + 1$. The commitment rule causes the validator to commit the view-v^* proposal. Since the view-v^* proposal strictly extends p, and since all ancestors of a committed proposal are also committed, proposal p (and thus its block) is committed at this moment.

Optimistic responsiveness. The (non-leader-based) optimistic responsiveness property states that, when all validators are correct, MonadBFT commits $\Omega(d/\delta)$ blocks during any post-GST time interval of duration d, where δ denotes the actual network delay (as opposed to the pessimistic bound Δ). This follows from the fact that, after GST, every view successfully extends the previous one by forming a QC. Since each view is led by a correct leader, the leader reliably disseminates its proposal to all validators, all correct validators vote for it, and the next correct leader collects these votes to form a QC and issues a proposal that carries it. As a consequence, a block proposed in view v is committed by the end of view $v + 2$: view v forms a QC for the proposal, view $v + 1$ forms the "child" QC extending it, and view $v + 2$ disseminates these two consecutive QCs, completing the commit. Since each view takes $O(\delta)$ time when all validators are correct, a block proposed in view v is committed within $O(\delta)$ time. Over a duration of d, the protocol executes $\Omega(d/\delta)$ views, and all but the last two of these views produce committed blocks. (The final two views also lead to commitments, but their corresponding commits occur just after the end of the time interval.) Lastly, since each correct leader can issue its proposal as soon as it collects either $2f + 1$ votes (sufficient to form a QC) or $2f + 1$ timeout messages (sufficient to form a TC), MonadBFT satisfies the leader-based optimistic responsiveness property.

Leader fault isolation. This property states that, after GST, if a view $v + 1$ led by a Byzantine leader is "sandwiched" between two views v and $v + 2$ led by correct leaders, then the duration of view $v + 1$ is at most $\Theta_{\text{view}} + O(\delta)$, where Θ_{view} is a single timeout duration. In other words, a single Byzantine leader cannot delay progress for more than one Θ_{view} period.

The intuition is as follows. Since view v is led by a correct leader and we are after GST, the leader of view v successfully constructs a QC for its proposal and disseminates it to all validators. Upon receiving this QC, all correct validators move to view $v + 1$, and they do so within $O(\delta)$ time of one another. Let

$\tau(v+1)$ denote the time when the first correct validator enters view $v+1$. From this moment, it takes at most $\Theta_{\text{view}} + O(\delta)$ time for all correct validators to issue their timeout messages for view $v+1$. Indeed, every correct validator issues a timeout message after spending Θ_{view} time in view $v+1$ (unless it has already progressed to later views), and all correct validators enter view $v+1$ within $O(\delta)$ time of $\tau(v+1)$. Thus, by time $\tau(v+1) + \Theta_{\text{view}} + O(\delta)$, every correct validator has produced its timeout message for view $v+1$. Consequently, after another δ time, all correct validators can construct a TC for view $v+1$ and transition to view $v+2$, ensuring that the Byzantine leader's influence is confined to a single view-duration period.

Linearity and low latency on the happy path. On the happy path (after GST, with leaders behaving correctly), each view incurs only linear communication. This is because each view follows the "leader-to-all, all-to-next-leader" pattern, which incurs linear number of messages. Importantly, no timeout messages are issued on the happy path, and therefore no TCs are formed. Since quadratic communication arises only from TC construction, all quadratic costs are confined to the unhappy path (i.e., during failures or before GST).

Regarding latency, forming and disseminating a QC for a proposal p requires exactly three communication steps on the happy path: step 1 disseminates p, step 2 disseminates votes for p, and step 3 disseminates the next proposal carrying the QC for p. Because MonadBFT permits speculative commitment upon observing a single QC, proposal p can be speculatively committed within these three communication rounds. (Recall that such speculative commitments can only be reverted in the presence of equivocation.)

For definitive, irrevocable commitment, MonadBFT requires one additional round-trip to form the second QC: one more step of vote dissemination for the proposal that carries a QC for p, followed by the dissemination of the next proposal that carries this second QC. Thus, irrevocable commitment is achieved with only two additional communication steps beyond speculative commitment, for a total of five communication steps.

5 Related Work

MonadBFT belongs to the HotStuff family of consensus protocols; therefore, we primarily discuss this class of protocols. We refer the reader to Table 1 for a detailed comparison with prior works.

BeeGees. Among existing protocols, BeeGees [15] is the most closely related to MonadBFT. It introduces the any-honest-leader (AHL) property, which implies tail-forking resistance. Since both BeeGees and MonadBFT are HotStufff [27] derivatives, they share linear complexity on the happy path, as well as pipelining as properties. However, the two protocols differ in several respects.

- First, BeeGees's Slow View Change requires the recovery leader to receive full block data from prior failed views: at the k-th consecutive failure, each of the n validators sends $\mathcal{O}(k \cdot b)$ data, resulting in $\mathcal{O}(k \cdot b \cdot n)$ inbound data at the recovery leader, where b is the block size. MonadBFT, having been designed with practical deployment in mind, must address this and it does so at the protocol level: the recovery leader either recovers the pending block or obtains a no-endorsement certificate proving the block is unavailable, without requiring full block data to be embedded in view change messages.[5]
- Second, MonadBFT introduces fast recovery via backup QCs, which ensures that a single Byzantine leader causes only one timeout delay (leader fault isolation).
- Third, when sufficient votes for implicit QC materialization are unavailable, BeeGees' recovery leader must wait $O(\Delta)$ before proposing, whereas MonadBFT maintains leader-based optimistic responsiveness on the unhappy path once timeout messages have been collected.
- Finally, MonadBFT introduces the notion of speculative commitment with precise reversion guarantees: a speculatively committed block can only be reverted if the proposing leader equivocated. While BeeGees' AHL property and reproposal mechanism may implicitly support a form of speculative finality, BeeGees does not define or discuss speculative commitment. MonadBFT makes this a first-class protocol property, with formal guarantees and concrete latency benefits for applications.

[5] We note that BeeGees could in principle be adapted to circumvent this issue; however, given its primarily theoretical focus, this does not appear to have been a design priority.

Concurrent work HotStuff-1 [19]. The concurrent work HotStuff-1 is a leader-based protocol with linear communication complexity in both the happy and unhappy paths. HotStuff-1 achieves a weaker speculative finalization property than MonadBFT, where validators can speculative finalize a block within 1.5 round trips only if the parent block is already committed. Moreover, HotStuff-1 does not prevent tail-forking attacks. To mitigate them, it assigns each validator a long view, subdivided into slots, where the leader proposes one block per slot. This design introduces several drawbacks. First, the view duration must be sufficiently long to allow an honest leader to finalize its block, which increases the risk of prolonged censorship and causes higher delays if the leader fails. In fact, a rational validator trying to maximize its MEV would censor MEV-related transactions until the last block, to maximize MEV [1]. Furthermore, HotStuff-1 lacks optimistic responsiveness (in the sense of view responsiveness property we define in Section 2.2) when a leader fails during view switch. Specifically, after detecting the failure of the previous view, leader in HotStuff-1 is still required to wait $O(\Delta)$ before proposing. Finally, tail-forking remains possible for the block proposed before the leader is rotated out, which is the root cause of the tail-forking problem.

Other protocols in the HotStuff family. Other members of the HotStuff family such as HotStuff [21], Fast-HotStuff [18], Jolteon [14], HotStuff-2 [21], and PaLa [10], have higher commit latencies and provide only limited protection against tail-forking.

Protocols in the PBFT family. Within the family of Practical Byzantine Fault Tolerance (PBFT) protocols [8, 12, 17], PBFT with tentative execution (PBFT-TE) [8] introduces a fast path that reduces latency to three message delays, while reverting to the traditional PBFT protocol if the fast path fails. PBFT-TE incurs $O(n^2)$ message complexity (even in the optimistic case), is not pipelined, and prevents tail-forking only if $f+1$ correct validators retain the QC for the block. The Moonshot protocol [12] also achieves three-message-delay latency but requires $O(n^2)$ communication on the happy path, and only provides weak tail-forking resistance. Finally, VBFT [17] provides tail-forking resistance, has speculative commit latency of two message delays, but incurs $O(n^2)$ message complexity in the optimistic case.

Acknowledgments

We thank Justin Jacob and Robert Kiel for helpful feedback on this work. We are especially grateful to Karolos Antoniadis, Manuel Bravo, Yacine Dolivet, and Denis Kolegov for their extensive comments and suggestions.

References

1. Alpturer, K., Babel, K., Saraf, A.: Timing games in responsive consensus protocols (2025), https://arxiv.org/abs/2510.25144
2. Babel, K., Daian, P., Kelkar, M., Juels, A.: Clockwork finance: Automated analysis of economic security in smart contracts. In: 2023 IEEE Symposium on Security and Privacy (SP). pp. 2499–2516. IEEE (2023)
3. Baudet, M., Ching, A., Chursin, A., Danezis, G., Garillot, F., Li, Z., Malkhi, D., Naor, O., Perelman, D., Sonnino, A.: State machine replication in the libra blockchain (2019), https://diem-developers-components.netlify.app/papers/diem-consensus-state-machine-replication-in-the-diem-blockchain/2019-06-28.pdf
4. Boneh, D., Gentry, C., Lynn, B., Shacham, H.: Aggregate and verifiably encrypted signatures from bilinear maps. In: Proceedings of the 22nd International Conference on Theory and Applications of Cryptographic Techniques. pp. 416–432. Springer-Verlag, Berlin, Heidelberg (2003)
5. Bracha, G.: Asynchronous byzantine agreement protocols. Inf. Comput. **75**(2), 130–143 (Nov 1987), https://doi.org/10.1016/0890-5401(87)90054-X
6. Bravo, M., Chockler, G., Gotsman, A.: Making byzantine consensus live. Distrib. Comput. **35**(6), 503–532 (Dec 2022), https://doi.org/10.1007/s00446-022-00432-y
7. Buchman, E.: Tendermint: Byzantine fault tolerance in the age of blockchains (Jun 2016)
8. Castro, M.: Practical Byzantine Fault Tolerance. Ph.D., MIT (Jan 2001), also as Technical Report MIT-LCS-TR-817

9. Castro, M., Liskov, B.: Practical Byzantine fault tolerance. In: Proceedings of the Third Symposium on Operating Systems Design and Implementation. pp. 173–186. OSDI '99, USENIX Association, Berkeley, CA, USA (1999)
10. Chan, T.H.H., Pass, R., Shi, E.: Pala: A simple partially synchronous blockchain. Cryptology ePrint Archive, Report 2018/981 (2018), https://eprint.iacr.org/2018/981
11. Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L., Juels, A.: Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In: 2020 IEEE Symposium on Security and Privacy (SP). pp. 910–927 (2020)
12. Doidge, I., Ramesh, R., Shrestha, N., Tobkin, J.: Moonshot: Optimizing chain-based rotating leader BFT via optimistic proposals (2024), https://arxiv.org/abs/2401.01791
13. Dwork, C., Lynch, N., Stockmeyer, L.: Consensus in the presence of partial synchrony. Journal of the ACM (JACM) **35**(2), 288–323 (1988)
14. Gelashvili, R., Kokoris-Kogias, L., Sonnino, A., Spiegelman, A., Xiang, Z.: Jolteon and Ditto: Network-adaptive efficient consensus with asynchronous fallback (2021)
15. Giridharan, N., Suri-Payer, F., Ding, M., Howard, H., Abraham, I., Crooks, N.: BeeGees: Stayin' alive in chained BFT. In: Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing. p. 233–243. PODC '23, Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3583668.3594572
16. He, Z., Li, J., Wu, Z.: Don't trust, verify: The case of slashing from a popular ethereum explorer. In: Companion Proceedings of the ACM Web Conference 2023. pp. 1078–1084 (2023)
17. Jalalzai, M.M., Feng, C., Lemieux, V.: VBFT: Veloce byzantine fault tolerant consensus for blockchains (2024), https://arxiv.org/abs/2310.09663
18. Jalalzai, M.M., Niu, J., Feng, C., Gai, F.: Fast-hotstuff: A fast and robust bft protocol for blockchains. IEEE Trans. Dependable Secur. Comput. **21**(4), 2478–2493 (Jul 2024). https://doi.org/10.1109/TDSC.2023.3308848, https://doi.org/10.1109/TDSC.2023.3308848
19. Kang, D., Gupta, S., Malkhi, D., Sadoghi, M.: Hotstuff-1: Linear consensus with one-phase speculation (2024), https://arxiv.org/abs/2408.04728
20. Lamport, L., Shostak, R., Pease, M.: The Byzantine generals problem. ACM Transactions on Programming Languages and Systems **4**(3), 382–401 (1982)
21. Malkhi, D., Nayak, K.: Extended abstract: HotStuff-2: Optimal two-phase responsive BFT. Cryptology ePrint Archive, Paper 2023/397 (2023), https://eprint.iacr.org/2023/397, https://eprint.iacr.org/2023/397
22. Monad Foundation: Blocksync. https://docs.monad.xyz/monad-arch/consensus/blocksync (2025)
23. Monad Foundation: Raptorcast. https://docs.monad.xyz/monad-arch/consensus/raptorcast (2025)
24. Monad Foundation: Statesync. https://docs.monad.xyz/monad-arch/consensus/statesync (2025)
25. Slashed validators. https://www.beaconcha.in/validators/slashings, accessed: 2024-10-04
26. Vonlanthen, Y., Sliwinski, J., Albarello, M., Wattenhofer, R.: Banyan: Fast rotating leader BFT. In: Proceedings of the 25th International Middleware Conference. p. 494–507. Middleware '24, Association for Computing Machinery, New York, NY, USA (2024), https://doi.org/10.1145/3652892.3700788
27. Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I.: Hotstuff: Bft consensus with linearity and responsiveness. In: Proceedings of the 2019 ACM PODC. p. 347–356. PODC '19, Association for Computing Machinery, New York, NY, USA (2019)

A Proofs

A.1 Safety

Before proceeding with the proof of safety, we refine our "extension" terminology to also apply to tips.

Tips and proposals. We say that a proposal p is a *parent* of a tip T of a fresh proposal p' if and only if p is a parent of p'. Since p' might have multiple parents, tip T can also have multiple parents. We further say that a tip T of a fresh proposal p' *extends* a proposal p if and only if p' extends p. Finally, a tip T of a fresh proposal p' *strictly extends* p if and only if p' strictly extends p.

Proof. Recall from Section 2.2 that the safety property requires that no two correct validators commit distinct blocks at the same log position in their local logs. To establish this, we first show that no correct validator votes for two distinct proposals within the same view.

Lemma 1. *Let $vote_1$ and $vote_2$ be any two votes cast by a correct validator i such that $vote_1$.`view` = $vote_2$.`view`. Then,*

$$vote_1.\texttt{proposal_id} = vote_2.\texttt{proposal_id}.$$

Proof. For the sake of contradiction, assume that the statement of the lemma does not hold. Without loss of generality, suppose validator i issues vote $vote_1$ before issuing vote $vote_2$. It is impossible for validator i to send two different votes at line 15 of Alg. 1, as it updates the highest_voted_view variable immediately after casting the first vote (line 16 of Alg. 1). Likewise, validator i cannot send two different votes at line 43 of Alg. 1, as it can time out from a given view $v = vote_1.\texttt{view} = vote_2.\texttt{view}$ at most once. Furthermore, $vote_1$ cannot be sent at line 43 of Alg. 1 as, upon timing out from view v, validator i updates its highest_voted_view variable to v (line 41 of Alg. 1), thereby preventing it from sending a subsequent vote at line 15 of Alg. 1. Hence, we conclude that $vote_1$ must be sent at line 15 of Alg. 1, and $vote_2$ must be sent at line 43 of Alg. 1.

When validator i creates $vote_1$, it invokes the CreateVote function with parameters cur_view and local_tip.`block_header` (line 14 of Alg. 1). When issuing $vote_2$, validator i's local_tip and cur_view variables remain unchanged, since neither is modified after $vote_1$ is cast. (If either variable were modified, validator i would have entered a view greater than v, which would prevent it from issuing $vote_2$ for view v.) Therefore, when $vote_2$ is created, validator i invokes the CreateVote function with parameters cur_view and local_tip.`block_header` (line 35 of Alg. 6). Since these parameters are identical to those used for $vote_1$, the statement of the lemma directly follows. □

Next, we show that two distinct proposals cannot both obtain QCs in the same view.

Lemma 2. *Let qc_1 and qc_2 be any two QCs such that qc_1.`view` = qc_2.`view`. Then,*

$$qc_1.\texttt{proposal_id} = qc_2.\texttt{proposal_id}.$$

Proof. This follows from quorum intersection: since each QC has $2f + 1$ votes and $n = 3f + 1$, at least one honest validator must have voted for both. As honest validators never vote for two distinct proposals in the same view (by Lemma 1), we have $qc_1.\texttt{proposal_id} = qc_2.\texttt{proposal_id}$. □

The lemma below shows that whenever a correct validator votes for a fresh proposal, its local tip is aligned with that proposal's block.

Lemma 3. *Let vote be a vote cast by a correct validator i in view v for a fresh proposal p:*

$$vote.\texttt{proposal_id} = p.\texttt{proposal_id}, \quad vote.\texttt{view} = v, \quad \textit{and} \quad p.\texttt{block.block_view} = v.$$

Then, at the moment vote is cast, the following is satisfied at validator i: (1) the block header stored in local_tip *matches the block header of p.*`block`*, and (2)* local_tip.`view` $= v$.

Proof. Validator i constructs *vote* by invoking CreateVote(cur_view, local_tip.block_header) (see line 14 of Alg. 1 and line 35 of Alg. 6). Recall that

$$p.\texttt{proposal_id} = \textsc{Hash}(p.\texttt{block.block_hash}, p.\texttt{view} = v)$$

and

$$\mathit{vote}.\texttt{proposal_id} = \textsc{Hash}(\texttt{local_tip.block_header.block_hash}, \texttt{cur_view}).$$

Since $\mathit{vote}.\texttt{proposal_id} = p.\texttt{proposal_id}$, it follows that

$$\texttt{local_tip.block_header.block_hash} = p.\texttt{block.block_hash} \qquad \text{and} \qquad \texttt{cur_view} = p.\texttt{view} = v.$$

(Otherwise, $p.\texttt{proposal_id}$ would differ from $\mathit{vote}.\texttt{proposal_id}$.) Note additionally that

$$p.\texttt{block.block_hash} = \texttt{local_tip.block_header.block_hash} = \textsc{Hash}(p.\texttt{block.block_view} = v, \cdot, \cdot).$$

Let p' be the proposal whose reception most recently updated local_tip to its current value, which is now used in *vote*. Since validator i only processes valid proposals, p' must be valid. Moreover, because local_tip is used in a vote for view v, we must have $p'.\texttt{view} \leq v$. We now consider the two cases for p':

- *Case 1:* $p'.\texttt{view} < v$.
  Since p' is valid, it follows from line 14 of Alg. 5 that $p'.\texttt{block.block_view} \leq p'.\texttt{view} < v$. We further distinguish:
  - If IsFreshProposal(p') = true, then local_tip is updated to the tip of proposal p' (see the GetTip(p') function), which itself contains the block header of $p'.\texttt{block}$. However, as $p.\texttt{block.block_hash} = \textsc{Hash}(v, \cdot, \cdot)$ and $p'.\texttt{block.block_hash} = \textsc{Hash}(< v, \cdot, \cdot)$, we must have

    $$\texttt{local_tip.block_header.block_hash} \neq p.\texttt{block.block_hash},$$

    contradicting the equality above. Therefore, this case is impossible.
  - If IsFreshProposal(p') = false, then, according to the check at line 20 of Alg. 5, we have that

    $$p'.\texttt{tc.high_tip.block_header.block_hash} = p'.\texttt{block.block_hash} = \textsc{Hash}(< v, \cdot, \cdot).$$

    In this case, local_tip is updated to $p'.\texttt{tc.high_tip}$ (see the GetTip(p') function). However, since $p.\texttt{block.block_hash} = \textsc{Hash}(v, \cdot, \cdot)$ while $p'.\texttt{tc.high_tip.block_header.block_hash} = \textsc{Hash}(< v, \cdot, \cdot)$, it follows that

    $$\texttt{local_tip.block_header.block_hash} \neq p.\texttt{block.block_hash}.$$

    Thus, this case is also impossible.
- *Case 2:* $p'.\texttt{view} = v$.
  Since p' is valid, line 14 of Alg. 5 ensures that $p'.\texttt{block.block_view} \leq v$. We distinguish:
  - If IsFreshProposal(p') = true, then $p'.\texttt{block.block_view} = v$ (because of the check at line 25 of Alg. 5, called at line 16) and local_tip is updated to the tip of p' (see the GetTip(p') function). Furthermore, since $\texttt{local_tip.block_header.block_hash} = p.\texttt{block.block_hash} = p'.\texttt{block.block_hash}$, it follows that $p.\texttt{block} = p'.\texttt{block}$ and thus local_tip.block_header indeed matches the block header of $p.\texttt{block}$. Furthermore, $\texttt{local_tip.view} = v$ (as local_tip is updated to the tip of p' with $p'.\texttt{view} = v$), proving the lemma in this case.
  - If IsFreshProposal(p') = false, then line 18 of Alg. 5 implies $p'.\texttt{block.block_view} < v$. Moreover, due to the check at line 20 of Alg. 5, we have that

    $$p'.\texttt{tc.high_tip.block_header.block_hash} = p'.\texttt{block.block_hash} = \textsc{Hash}(< v, \cdot, \cdot).$$

    In this case, local_tip is updated to $p'.\texttt{tc.high_tip}$ (see the GetTip(p') function). However, since $p.\texttt{block.block_hash} = \textsc{Hash}(v, \cdot, \cdot)$ while $p'.\texttt{tc.high_tip.block_header.block_hash} = \textsc{Hash}(< v, \cdot, \cdot)$, it follows that

    $$\texttt{local_tip.block_header.block_hash} \neq p.\texttt{block.block_hash},$$

    which renders this case impossible.

The only feasible scenario is the subcase where $p'.\texttt{view} = v$ and IsFreshProposal$(p') =$ true, under which the desired statement holds. This concludes the proof. □

The following lemma is critical for the safety property of the MonadBFT protocol. In essence, this shows that once $f + 1$ correct validators vote for the "second-level" QC of a proposal, every valid proposal in subsequent views must strictly extend that proposal.

Lemma 4. *Let p be any proposal with $p.\texttt{view} = v$. Suppose that at least $f + 1$ correct validators vote for a proposal p' satisfying*

$$p'.\texttt{view} = v + 1, \quad p'.\texttt{block.qc.view} = v, \quad \textit{and} \quad p'.\texttt{block.qc.proposal_id} = p.\texttt{proposal_id}.$$

Then, any valid proposal p^* issued in a view $v^* > v + 1$ (i.e., $p^*.\texttt{view} = v^* > v + 1$) must strictly extend p.

Proof. Let S denote the set of correct validators, with $|S| \geq f + 1$, that voted for fresh proposal p' in view $v + 1$. For every correct validator $i \in S$, the following holds. At the moment i votes for proposal p' in view $v + 1$, Lemma 3 implies that (1) local_tip's block header matches the block header of $p'.\texttt{block}$, and (2) local_tip.view $= v + 1$. Hence, since validator i cannot update its local_tip variable to a tip with view $v + 1$ in any view smaller than $v + 1$, it follows that i must have updated its local tip in view $v + 1$ to a tip whose block header matches the block header of $p'.\texttt{block}$.

We prove the lemma using induction on the view number v'. Let us define the following five induction hypotheses, $X(v')$, $Y(v')$, $Q(v')$, $W(v')$, and $Z(v')$:

- $X(v')$: Any (valid) proposal p^* in view $v' > v+1$ strictly extends proposal p and satisfies $p^*.\texttt{block.qc.view} \geq v$. Consequently, any (valid) tip T of a fresh proposal from view $v' > v + 1$ also strictly extends proposal p and satisfies $T.\texttt{block_header.qc.view} \geq v$.
- $Y(v')$: If any validator $i \in S$ updates its local tip to some tip T in view $v' \geq v + 1$, then $T.\texttt{view} \geq v + 1$ and $T.\texttt{block_header.qc.view} \geq v$.
- $Q(v')$: For any QC qc with $qc.\texttt{view} = v' \geq v+1$, any proposal p_{qc} with $p_{qc}.\texttt{proposal_id} = qc.\texttt{proposal_id}$ strictly extends proposal p.
- $W(v')$: Let tc be any TC with $tc.\texttt{view} = v' \geq v + 1$ such that $tc.\texttt{high_qc} \neq \perp$. Then, any proposal p_{tc} with $p_{tc}.\texttt{proposal_id} = tc.\texttt{high_qc.proposal_id}$ strictly extends p and $tc.\texttt{high_qc.view} \geq v + 1$.
- $Z(v')$: Let tc be any TC with $tc.\texttt{view} = v' \geq v + 1$ and $tc.\texttt{high_tip} \neq \perp$. Then, the proposal whose tip is $tc.\texttt{high_tip}$ strictly extends p, $tc.\texttt{high_tip.view} \geq v + 1$ and $tc.\texttt{high_tip.block_header.qc.view} \geq v$.

Note that proving $X(v')$ is equivalent to proving the statement of the lemma. Therefore, we start by proving the rest.

$Y(v' = v + 1)$: Consider any correct validator $i \in S$. By the first paragraph of the proof, validator i updates its local tip in view $v + 1$ to a tip whose block header matches that of $p'.\texttt{block}$ and whose view is $v + 1$. Since i can update its local tip at most once per view and $p.\texttt{view} = v$, it follows that $Y(v' = v + 1)$ holds.

$Q(v' = v + 1)$: Let qc be any QC with $qc.\texttt{view} = v+1$. Since $f+1$ correct validators voted for proposal p' in view $v+1$ and at least one of these validators must contribute to qc, Lemma 1 implies that $qc.\texttt{proposal_id} = p'.\texttt{proposal_id}$. Hence, the block contained in any proposal p_{qc} with $p_{qc}.\texttt{proposal_id} = qc.\texttt{proposal_id}$ must be identical to the block contained in p'. Therefore, p_{qc} strictly extends p.

$W(v' = v + 1)$: Let tc be any TC with $tc.\texttt{view} = v + 1$ and $tc.\texttt{high_qc} \neq \perp$. By quorum intersection (since $|S| \geq f + 1$ and tc contains information from $2f + 1$ validators), tc includes information from at least one validator $i \in S$. Since $Y(v+1)$ holds and i updated its local tip to a tip whose view is $v+1$ and whose block header matches the block header of $p'.\texttt{block}$ (from the first paragraph of the proof) before contributing to tc, $tc.\texttt{high_qc.view} \geq v + 1$. Moreover, since $tc.\texttt{view} = v + 1$, we have $tc.\texttt{high_qc.view} \leq v$ (due to the ValidTC function). Therefore, this case is impossible; that is, no TC from view $v+1$ can have a non-empty high_qc field.

$Z(v' = v+1)$: Let tc be any TC with $tc.\texttt{view} = v+1$ and $tc.\texttt{high_tip} \neq \bot$. By quorum intersection (since $|S| \geq f+1$ and tc contains information about $2f+1$ validators), tc includes information from at least one validator $i \in S$. Because $Y(v+1)$ holds and i updated its local tip to a tip whose view is $v+1$ and whose block header matches the block header of $p'.\texttt{block}$ (from the first paragraph of the proof) before contributing to tc, $tc.\texttt{high_tip.view} \geq v+1$. Given that $tc.\texttt{view} = v+1$, we have that $tc.\texttt{high_tip.view} \leq v+1$ (by the ValidTC function). Therefore, $tc.\texttt{high_tip.view} = v+1$. Moreover, since the local tip of validator $i \in S$ contains a block header that matches the block header of $p'.\texttt{block}$, it follows that $tc.\texttt{tips_views}[i] = v+1$ and $tc.\texttt{qcs_views}[i] = v$. (Note that validator i reports its local tip and not its last_qc variable, since its local tip is from view $v+1$, whereas its last_qc variable cannot hold any QC from any view greater than v.) This further implies that $tc.\texttt{high_tip.block_header.qc.view} = v$. As a QC was formed for proposal p in view v, Lemma 2 ensures that a parent of the proposal whose tip is $tc.\texttt{high_tip}$ must be p. Therefore, the proposal whose tip is $tc.\texttt{high_tip}$ strictly extends p, which implies that $Z(v' = v+1)$ holds.

$X(v' = v+2)$: Let us consider all possible cases for a valid proposal p^* with $p^*.\texttt{view} = v+2$:

- Suppose p^* is a fresh proposal. Consider the following three possibilities:
  - Let $p^*.\texttt{block.qc.view} = v+1$. Then, by quorum intersection, at least one validator $i \in S$ must contribute to $p^*.\texttt{block.qc}$. Hence, Lemma 1 implies that a parent of proposal p^* must be proposal p'. Since p' strictly extends proposal p, p^* strictly extends proposal p, thereby proving $X(v' = v+2)$ in this case.
  - Let $p^*.\texttt{nec} \neq \bot$. We first prove that $p^*.\texttt{nec.high_tip_qc_view} = v$. In order for $p^*.\texttt{nec}$ to be formed, a correct validator i must have received an NE request accompanied by a TC tc with $tc.\texttt{view} = v'-1 = v+1$ and $tc.\texttt{high_tip} \neq \bot$. Given that $Z(v+1)$ holds, $tc.\texttt{high_tip}$ strictly extends proposal p and $tc.\texttt{high_tip.view} \geq v+1$. Moreover, given that $tc.\texttt{view} = v+1$, it follows that $tc.\texttt{high_tip.view} = v+1$ (due to the ValidTC function). Given that $Z(v+1)$, we additionally have that $tc.\texttt{high_tip.block_header.qc.view} \geq v$. Because $tc.\texttt{high_tip.view} = v+1$ and $tc.\texttt{high_tip.block_header.qc.view} < v+1$, we have that $p^*.\texttt{nec.high_tip_qc_view}$ indeed must be equal to v.
    Moreover, observe that $p^*.\texttt{block.qc.view} = p^*.\texttt{nec.high_tip_qc_view} = v$ (by the ValidProposal function). Because a QC is formed for proposal p in view v, we obtain $p^*.\texttt{block.qc.proposal_id} = p.\texttt{proposal_id}$ (by Lemma 2). Thus, p^* strictly extends proposal p, establishing $X(v' = v+2)$.
  - Let $p^*.\texttt{nec} = \bot$ and $p^*.\texttt{tc.high_qc} \neq \bot$. Note that $p^*.\texttt{tc.view} = v+1$. By quorum intersection (since $|S| \geq f+1$ and $p^*.\texttt{tc}$ contains information from $2f+1$ validators), tc includes information from at least one validator in S. Given that $Y(v+1)$ holds and all validators from the set S update their local tip to a tip whose view is $v+1$ and whose block header matches the block header of $p'.\texttt{block}$ (from the first paragraph of the proof) before (potentially) contributing to $p^*.\texttt{tc}$, it follows that $p^*.\texttt{tc.tips_views}$ includes $v+1$. (Note that each validator $i \in S$ reports its local tip and not its last_qc variable, since its local tip is from view $v+1$, whereas its last_qc variable cannot hold any QC from any view greater than v.) Therefore, it must hold that $p^*.\texttt{tc.high_qc.view} \geq v+1$. However, the fact that $p^*.\texttt{tc.view} = v+1$, implies that $p^*.\texttt{tc.high_qc.view} \leq v$ (due to the ValidTC function). Therefore, this case cannot occur.
- Suppose p^* is a reproposal. In this case, p^* is a reproposal of $p^*.\texttt{tc.high_tip}$. Since $p^*.\texttt{tc.view} = v+1$, $Z(v+1)$ implies that p^* strictly extends p and $p^*.\texttt{block.qc.view} \geq v$, which proves the $X(v' = v+2)$ statement in this case.

We now prove that the inductive step holds.

$Y(v' > v+1)$: Consider a validator $i \in S$ that modifies its local tip in view $v' > v+1$. Let us consider all possible cases for that to happen:

- Let i modify its local tip upon receiving a fresh proposal p^* in view $v' > v+1$. We further consider three possibilities:

- Let p^*.block.qc.view $= p^*$.view $- 1$. In this case, validator i updates its local tip to $T(p^*)$, the tip of proposal p^*, which satisfies: $T(p^*)$.view $> v+1$ and $T(p^*)$.block_header.qc.view $\geq v+1$ (as p^*.block.qc.view $= v' - 1 \geq v+1$). Thus, $Y(v' > v+1)$ holds in this case.
- Let p^*.nec $\neq \bot$. In this case, validator i updates its local tip to the tip $T(p^*)$ of proposal p^* with $T(p^*)$.view $> v+1$ (as p^*.view $= v' > v+1$). We now prove that $T(p^*)$.block_header.qc.view $= p^*$.nec.high_tip_qc_view $\geq v$. In order for p^*.nec to be formed, a correct validator i must have received an NE request accompanied by a TC tc with tc.view $= v' - 1 \geq v+1$. Given that $Z(v' - 1 \geq v+1)$ holds, tc.high_tip.block_header.qc.view $\geq v$, which implies that p^*.nec.high_tip_qc_view $\geq v$.
- Let p^*.nec $= \bot$ and p^*.tc.high_qc $\neq \bot$. In this case, validator i updates its local tip to the tip $T(p^*)$ of proposal p^* with $T(p^*)$.view $> v+1$ (as p^*.view $= v' > v+1$). Moreover, a parent of p^* is a proposal to which p^*.tc.high_qc points (guaranteed by the ValidProposal function). Given that p^*.tc.view $= v' - 1 \geq v+1$ and $W(v'-1)$ holds, p^*.block.qc.view $\geq v+1$. Hence, $Y(v' > v+1)$ is satisfied even in this case.

– Let validator i modify its local tip upon receiving a reproposal p^*. In this case, validator i updates its local tip to p^*.tc.high_tip (note that p^*.tc.view $= v' - 1 \geq v+1$ and p^*.tc.high_tip $\neq \bot$ in this case). As $Z(v' - 1 \geq v+1)$ holds, p^*.tc.high_tip.view $\geq v+1$ and p^*.tc.high_tip.block_header.qc.view $\geq v$, which proves $Y(v' > v+1)$.

As $Y(v' > v+1)$ is preserved in all possible cases, the statement holds.

$Q(v' > v+1)$: Let qc be any QC with qc.view $= v' > v+1$. We distinguish between two cases:

– Suppose that at least one correct validator contributing to qc did so upon receiving a proposal in view v'. In this case, the received proposal strictly extends proposal p, since $X(v')$ holds. Hence, $Q(v' > v+1)$ is satisfied.
– Suppose that no correct validator contributing to qc did so upon receiving a proposal in view v'. In this case, all correct votes issued in view v' are based on timeout messages, each corresponding to the local tip of the validator at that time. Since $Y(x)$ holds for all views $x \in [v+1, v']$, and each validator in S updates its local tip in view $v+1$ to a tip that strictly extends p (by the first paragraph of the proof) before potentially issuing any timeout-based vote in view v', it follows that at least one correct validator $i \in S$ must have contributed to qc after having voted for proposal p'. If validator i updates its local tip for this vote in view $v+1$, then—since a correct validator updates its local tip at most once per view, and it has already done so in view $v+1$ to a tip that strictly extends p (by the first paragraph of the proof)—the vote must target a block that strictly extends p. Otherwise, the vote must also target a block that strictly extends p by $X(v^*)$, for any $v^* \in [v+2, v']$. In both cases, $Q(v' > v+1)$ is satisfied.

$W(v' > v+1)$: Let tc be any TC with tc.view $= v' > v+1$ and tc.high_qc $\neq \bot$. As (1) $|S| \geq f+1$, and (2) tc includes information about $2f+1$ validators, information about at least one validator $i \in S$ is included in tc. Furthermore, since (1) validator i updates its local tip to a tip whose view is $v+1$ and that strictly extends proposal p (from the first paragraph of the proof) before contributing to tc, and (2) $Y(x)$ holds for every view $x \in [v+1, v']$, it follows that tc.high_qc.view $\geq v+1$ (as validator i reports either its local tip from a view $\geq v+1$ or its last_qc variable from a view $\geq v+1$). Moreover, the validity of tc dictates that tc.high_qc.view $< v'$ (see the ValidTC function). Lastly, since $Q(x)$ holds for every view $x \in [v+1, v'-1]$, it must be that a proposal to which tc.high_qc points strictly extends proposal p, thus proving $W(v' > v+1)$.

$Z(v' > v+1)$: Let tc be any TC with tc.view $= v' > v+1$ and tc.high_tip $\neq \bot$. Since (1) $|S| \geq f+1$, and (2) tc includes information about $2f+1$ validators, it must be that information about at least one validator $i \in S$ is included in tc. Given that (1) validator i updates its local tip to a tip whose view is $v+1$ and that strictly extends proposal p (from the first paragraph of the proof) before contributing to tc, and (2) $Y(x)$ holds for every view $x \in [v+1, v']$, tc.high_tip.view $\geq v+1$ (as validator i reports either its local tip from a view $\geq v+1$ or its last_qc variable from a view $\geq v+1$). We now consider two possible cases:

– Let tc.high_tip.view $= v+1$. Given that (1) tc.high_tip $\neq \bot$, (2) tc.high_tip.view $= v+1$, (3) validator i updated its local tip to a tip whose view is $v+1$ and whose block header matches the block

header of p'.block (from the first paragraph of the proof) before contributing to tc, and (4) $Y(x)$ holds for every view $x \in [v+1, v']$, it must follow that tc.tips_views$[i] = v+1$ and tc.qcs_views$[i] = v$. (Note that if i were to include a QC in its timeout message, this would imply that its view is $\geq v+1$, which in turn would force tc.high_tip $= \perp$.) Therefore, tc.high_tip.block_header.qc.view $= v$. Lastly, given that a QC is formed in view v for proposal p, Lemma 2 ensures that a parent of tc.high_tip must be p. This implies that tc.high_tip strictly extends proposal p, which proves $Z(v' > v+1)$ in this case.
- Let tc.high_tip.view $> v+1$. Note that tc.high_tip.view $\leq v'$. As $X(tc$.high_tip.view$)$ is satisfied, we know that tc.high_tip strictly extends proposal p and it holds that tc.high_tip.block_header.qc.view $\geq v$. Thus, $Z(v' > v+1)$ holds here as well.

$X(v' > v+2)$: Let us consider all possible cases for a valid proposal p^* with p^*.view $= v' > v+2$:

- Suppose p^* is a fresh proposal. We further consider three possibilities:
  - Let p^*.block.qc.view $= v' - 1 \geq v+2$. In this case, a parent of p^* strictly extends proposal p (because $Q(v'-1 \geq v+2)$ holds), which implies that p^* strictly extends proposal p. Moreover, p^*.block.qc.view $\geq v+2$, which concludes the proof of $X(v' > v+2)$ in this case.
  - Let p^*.nec $\neq \perp$. We first prove that p^*.nec.high_tip_qc_view $= p^*$.block.qc.view $\geq v$. In order for p^*.nec to be formed, a correct validator i must have received an NE request accompanied by a TC tc with tc.view $= v' - 1 \geq v+2$ and tc.high_tip $\neq \perp$. Given that $Z(v'-1)$ holds, tc.high_tip.block_header.qc.view $\geq v$. Therefore, p^*.nec.high_tip_qc_view $= p^*$.block.qc.view $\geq v$. Hence, it is left to prove that p^* strictly extends proposal p. We now differentiate two cases:
    - Let p^*.block.qc.view $= v$. In this case, a parent of p^* must be p (by Lemma 2), which proves the statement of the invariant.
    - Let p^*.block.qc.view $> v$. In this case, the fact that $Q(p^*$.block.qc.view $\geq v+1)$ ensures that the statement of the invariant holds.
  - Let p^*.nec $= \perp$ and p^*.tc.high_qc $\neq \perp$. Note that p^*.tc.view $= v' - 1$. By quorum intersection (since $|S| \geq f+1$ and p^*.tc contains information from $2f+1$ validators), p^*.tc includes information from at least one validator in S. Given that $Y(x)$ holds for all views $v \in [v+1, v'-1]$ and all validators from the set S update their local tip to a tip whose view is $v+1$ and that strictly extends proposal p (from the first paragraph of the proof) before (potentially) contributing to p^*.tc (in view $v+1$), it follows that p^*.tc.high_qc.view $\geq v+1$; note that p^*.tc.high_qc.view $< v' - 1$. Next, $Q(p^*$.tc.high_qc.view$)$ shows that p^*.tc.high_qc points to a proposal that strictly extends proposal p. Since p^*.block.qc $= p^*$.tc.high_qc (ensured by the ValidProposal function), p^* strictly extends proposal p. Moreover, p^*.block.qc.view $\geq v+1$, which concludes the proof of $X(v' > v+2)$ in this case.
- Suppose p^* is a reproposal. In this case, p^* is a reproposal of p^*.tc.high_tip; note that p^*.tc.view $= v' - 1 \geq v+2$. Due to $Z(v'-1)$, we have that p^* strictly extends proposal p (as p^*.tc.high_tip does so) and p^*.block.qc.view $\geq v$ (as p^*.tc.high_tip.block_header.qc.view $\geq v$). Thus, $X(v' > v+2)$ holds in this case as well.

As $X(v' > v+2)$ holds in all possible scenarios, the proof of the lemma is completed. □

The following lemma shows that once $f+1$ correct validators vote for the "second-level" QC of a proposal, none of them can later vote for any proposal that does not strictly extend that proposal.

Lemma 5. *Let p be any proposal with p.view $= v$. Suppose that at least $f+1$ correct validators vote for a proposal p' satisfying*

$$p'.\texttt{view} = v+1, \quad p'.\texttt{block.qc.view} = v, \quad \textit{and} \quad p'.\texttt{block.qc.proposal_id} = p.\texttt{proposal_id}.$$

Then, if any correct validator that voted for p' in view $v+1$ later votes for a proposal p^* in some view $v^* > v+1$, the proposal p^* must strictly extend p.

Proof. Let S denote the set of correct validators, with $|S| \geq f + 1$, that voted for fresh proposal p' in view $v + 1$. Assume, for the sake of contradiction, that there exists a correct validator $\ell \in S$ that votes for a proposal p^* in some view $v^* > v + 1$ such that p^* does not strictly extend p. We know that, in view $v + 1$, validator ℓ's local tip stores a block header that matches the block header of proposal p' (by Lemma 3). The fact that ℓ issues a vote for proposal p^* (that does not strictly extend proposal p) in view v^* necessarily implies that ℓ updates its local tip to contain a block header that does not strictly extend proposal p after voting for proposal p'. Indeed, if validator ℓ casts its vote for p^* at line 15 of Alg. 1, it updates its local tip at line 13 of the same algorithm; otherwise, if the vote is issued via a timeout message, it is created at line 35 of Alg. 6. If validator ℓ had not updated its local tip to contain a block header that does not strictly extend proposal p after voting for p', then p^* would necessarily strictly extend p. Hence, validator ℓ must have updated its local tip to contain a block header that does not strictly extend proposal p after voting for p'.

Note that this update cannot occur in view $v+1$: once ℓ votes for p', it sets its highest_voted_view variable to $v + 1$ (line 16 or line 41 of Alg. 1), which prevents it from updating its local tip in view $v + 1$ (line 13 of Alg. 1). Therefore, validator ℓ must update its local tip to contain a block header that does not strictly extend proposal p in some view strictly greater than $v+1$. However, by Lemma 4, every valid proposal from any view greater than $v+1$—and hence every proposal that ℓ could use to update its local tip—must strictly extend p. This contradicts the assumption that validator ℓ updates its local tip in some view greater than $v + 1$ to contain a block header that does not strictly extend p, and it completes the proof. □

Finally, we are ready to prove that MonadBFT satisfies safety.

Theorem 1 (Safety). *No two correct validators commit different blocks at the same log position in their local logs.*

Proof. Suppose, toward contradiction, that two correct validators i and j commit distinct blocks B_i and B_j at the same log position, and that these blocks are committed as part of proposals p_i and p_j, respectively. Note that proposals p_i and p_j are conflicting; that is, neither extends the other. (Otherwise, the blocks B_i and B_j could not have been committed at the same log position.) Without loss of generality, let $p_i.\texttt{view} \leq p_j.\texttt{view}$. This implies the existence of a QC for proposal p_i in view $p_i.\texttt{view}$, another QC for the child of p_i in view $p_i.\texttt{view} + 1$, and a QC for proposal p_j in view $p_j.\texttt{view}$ (by the commitment rules of MonadBFT). We now consider three possible cases:

- $p_i.\texttt{view} = p_j.\texttt{view}$: Lemma 2 shows that $p_i.\texttt{proposal_id} = p_j.\texttt{proposal_id}$. However, this is impossible as the blocks B_i and B_j could not have been committed at the same log position.
- $p_i.\texttt{view} + 1 = p_j.\texttt{view}$: Again, Lemma 2 proves that $p_j.\texttt{proposal_id} = c_i.\texttt{proposal_id}$, where c_i denotes the child of proposal p_i. This cannot happen as the blocks B_i and B_j could not have been committed at the same log position.
- $p_i.\texttt{view} + 1 < p_j.\texttt{view}$: In this case, there exists at least one correct validator that contributed both to the QC for the child of p_i in view $p_i.\texttt{view} + 1$ and to the QC for proposal p_j in view $p_j.\texttt{view}$. By Lemma 5, this implies that p_j strictly extends p_i. This is impossible, however, because block B_j conflicts with block B_i.

As neither of these cases can occur, the proof is complete. □

A.2 Tail-Forking Resistance

Recall from Section 2.2 that the tail-forking resistance property asserts the following: if a leader proposes a block B as part of a fresh proposal p, at least $f + 1$ correct validators vote for p, and the leader does not equivocate, then whenever any correct validator commits a block B', it must hold that either B extends B' or B' extends B. We begin by showing that, under the conditions stated above, no NEC can be formed.

Lemma 6. *Let p be any fresh proposal with $p.\texttt{view} = v$, issued by the leader of view v. Then, no correct validator that voted for p in view v issues an NE message upon receiving a message $\langle \mathsf{NERequest}, tc \rangle$, where tc is a TC whose $tc.\texttt{high_tip}$ satisfies $tc.\texttt{high_tip}.\texttt{proposal_id} = p.\texttt{proposal_id}$.*

Proof. By contradiction, assume that some correct validator i that voted for proposal p in view v issues an NE message upon receiving the above $\langle \mathsf{NERequest}, tc \rangle$ message. Since p is a fresh proposal from view v, the ValidTC check implies that $tc.\texttt{view} \geq v$ (as $tc.\texttt{high_tip.view} = v$). Observe that validator i must have voted for p before receiving the NERequest. Indeed, if i processed the NERequest first, it would increment its cur_view to a value $\geq v+1$ (line 40 of Alg. 2), and therefore could not subsequently cast a vote for p in view v.

Thus, validator i votes for p and only later receives the NERequest. However, in this case i cannot issue an NE message, because the condition at line 25 of Alg. 7 fails. This yields a contradiction, and the lemma follows. □

Next, we show that any fresh proposal by a non-equivocating leader receiving at least $f+1$ votes from correct validators can never be abandoned.

Lemma 7. *Let p be any fresh proposal with $p.\texttt{view} = v$, issued by the leader of view v. Suppose that the leader of view v does not equivocate and that at least $f+1$ correct validators vote for p in view v. Then, any valid proposal p^* issued in a view $v^* \geq v+1$ (i.e., $p^*.\texttt{view} = v^* \geq v+1$) must extend p.*

Proof. Let S denote the set of correct validators, with $|S| \geq f+1$, that voted for fresh proposal p in view v. For every correct validator $i \in S$, the following holds. At the moment i votes for proposal p in view v, Lemma 3 implies that (1) local_tip.block_header matches the block header of $p.\texttt{block}$, and (2) local_tip.view $= v$. Hence, since validator i cannot update its local_tip variable to a tip with view v in any view smaller than v, it follows that i must have updated its local tip in view v to a tip that extends proposal p.

We prove the claim by induction on the view number v'. Consider the following four induction hypotheses, $X(v')$, $Y(v')$, $Q(v')$, and $Z(v')$:

- $X(v')$: Any (valid) proposal p^* in view $v' \geq v$ extends p. Consequently, any (valid) tip T of a fresh proposal from view $v' \geq v$ also extends p. Moreover, if $v' > v$, then $T.\texttt{block_header.qc.view} \geq v$.
- $Y(v')$: If any validator $i \in S$ updates its local tip to some tip T in view $v' \geq v$, then $T.\texttt{view} \geq v$.
- $Q(v')$: For any QC qc with $qc.\texttt{view} = v' \geq v$, any proposal p_{qc} with $p_{qc}.\texttt{proposal_id} = qc.\texttt{proposal_id}$ extends proposal p.
- $Z(v')$: Let tc be any TC with $tc.\texttt{view} = v' \geq v$ and $tc.\texttt{high_tip} \neq \bot$. Then, the proposal whose tip is $tc.\texttt{high_tip}$ extends p and $tc.\texttt{high_tip.view} \geq v$. Moreover, if $tc.\texttt{high_tip.view} > v$, then $tc.\texttt{high_tip.block_header.qc.view} \geq v$.

We begin by proving the base case. Note that $X(v')$ is equivalent to the lemma's statement.

$\underline{X(v'=v)}$: This follows directly from the fact that the leader of view v issues only proposal p in that view.

$\underline{Y(v'=v)}$: Consider any correct validator $i \in S$. By the first paragraph of the proof, validator i updates its local tip in view v to a tip whose view is v. Since i can update its local tip at most once per view, it follows that $Y(v'=v)$ holds.

$\underline{Q(v'=v)}$: Let qc be any QC with $qc.\texttt{view} = v$. Given that at least $f+1$ correct validators voted for proposal p in view v, Lemma 1 ensures that qc points to proposal p. Thus, $Q(v'=v)$ holds.

$\underline{Z(v'=v)}$: Let tc be any TC with $tc.\texttt{view} = v$ and $tc.\texttt{high_tip} \neq \bot$. The standard quorum intersection (as $|S| \geq f+1$ and tc contains information about $2f+1$ validators) ensures that tc includes information about at least one validator $i \in S$. Because $Y(v)$ holds and i updated its local tip to a tip whose view is v and that extends proposal p (from the first paragraph of the proof) before contributing to tc (in view v), $tc.\texttt{high_tip.view} \geq v$. (Note that validator i reports its local tip and not its last_qc variable, since its local tip is from view v, whereas its last_qc variable cannot hold any QC from any view greater than $v-1$.) Since $tc.\texttt{high_tip.view} \leq tc.\texttt{view} = v$ (by the ValidTC function), we get that $tc.\texttt{high_tip.view} = v$. As the only proposal issued in view v is p, it follows that $tc.\texttt{high_tip}$ extends proposal p, which implies that $Z(v'=v)$ holds.

We now prove that the inductive step holds.

$\underline{X(v'>v)}$: Let us consider all possible cases for a valid proposal p^* with $p^*.\texttt{view} = v' > v$:

– Suppose p^* is a fresh proposal. We further consider three possibilities:
  - Let $p^*.\texttt{block.qc.view} = v' - 1 \geq v$. In this case, a parent of p^* extends proposal p (because $Q(v' - 1 \geq v)$ holds), which implies that p^* extends proposal p. Thus, $X(v' > v)$ holds in this case.
  - Let $p^*.\texttt{nec} \neq \bot$. By quorum intersection, there exists a correct validator $i \in S$ that issues an NE message upon receiving a message $\langle \mathsf{NERequest}, tc \rangle$ in view $v' > v$, where $tc.\texttt{view} = v' - 1 \geq v$ and $tc.\texttt{high_tip} \neq \bot$. Since $Z(v' - 1 \geq v)$ holds, $tc.\texttt{high_tip}$ extends p and $tc.\texttt{high_tip.view} \geq v$. We now show that $tc.\texttt{high_tip.view} > v$. Assume, for contradiction, that $tc.\texttt{high_tip.view} = v$. Since p is the unique proposal issued in view v, we have $tc.\texttt{high_tip.proposal_id} = p.\texttt{proposal_id}$, contradicting the fact that validator i issued an NE message (by Lemma 6).
    Next, by $Z(v' - 1 \geq v)$, $tc.\texttt{high_tip.block_header.qc.view} \geq v$. Thus, $p^*.\texttt{nec.high_tip_qc_view} \geq v$. At the same time, the validity of $p^*.\texttt{nec}$ implies that

    $$p^*.\texttt{nec.high_tip_qc_view} < v' - 1.$$

    Since a parent of p^* is a proposal to which a QC from view $p^*.\texttt{nec.high_tip_qc_view}$ points (as enforced by the ValidProposal function), and since $Q(p^*.\texttt{nec.high_tip_qc_view} \in [v, v' - 2])$ holds, it follows that p^* extends p.
  - Let $p^*.\texttt{nec} = \bot$ and $p^*.\texttt{tc.high_qc} \neq \bot$. Note that $p^*.\texttt{tc.view} = v' - 1 \geq v$. By quorum intersection (since $|S| \geq f + 1$ and $p^*.\texttt{tc}$ contains information from $2f + 1$ validators), $p^*.\texttt{tc}$ includes information from at least one validator in S. Given that $Y(x)$ holds for all views $v \in [v, v' - 1]$ and all validators from the set S update their local tip to a tip whose view is v and that extends proposal p (from the first paragraph of the proof) before contributing to $p^*.\texttt{tc}$ (in view v), it follows that $p^*.\texttt{tc.high_qc.view} \geq v$; note that $p^*.\texttt{tc.high_qc.view} < v' - 1$ (by the ValidTC function). Next, $Q(p^*.\texttt{tc.high_qc.view} \in [v, v' - 2])$ shows that $p^*.\texttt{tc.high_qc}$ points to a proposal that extends proposal p. Since $p^*.\texttt{block.qc} = p^*.\texttt{tc.high_qc}$ (ensured by the ValidProposal function), p^* extends proposal p, which concludes the proof of $X(v' > v)$ in this case.
– Suppose p^* is a reproposal. In this case, p^* is a reproposal of $p^*.\texttt{tc.high_tip}$; note that $p^*.\texttt{tc.view} = v' - 1 \geq v$. Due to $Z(v' - 1)$, we have that p^* extends proposal p (as $p^*.\texttt{tc.high_tip}$ does so). Thus, $X(v' > v)$ holds in this case as well.

Therefore, $X(v' > v)$ holds in all possible scenarios.

$Y(v' > v)$: Consider a validator $i \in S$ that modifies its local tip in view $v' > v$. Let us consider all possible cases for that to happen:

– Let i modify its local tip upon receiving a fresh proposal p^* in view $v' > v$. In this case, i updates its local tip to some tip T with $T.\texttt{view} = v' > v$, which proves $Y(v' > v)$ in this scenario.
– Let validator i modify its local tip upon receiving a reproposal p^*. Here, $p^*.\texttt{tc} \neq \bot$. In this case, validator i updates its local tip to $p^*.\texttt{tc.high_tip}$ (note that $p^*.\texttt{tc.view} = v' - 1 \geq v$). By quorum intersection (since $|S| \geq f + 1$ and $p^*.\texttt{tc}$ contains information from $2f + 1$ validators), $p^*.\texttt{tc}$ includes information from at least one validator in S. Given that $Y(x)$ holds for all views $v \in [v, v' - 1]$ and all validators from the set S update their local tip to a tip whose view is v and that extends proposal p (from the first paragraph of the proof) before contributing to $p^*.\texttt{tc}$ (in view v), it follows that $p^*.\texttt{tc.tips_views}$ includes $\geq v$. Hence, $p^*.\texttt{tc.high_tip.view} \geq v$, which proves $Y(v' > v)$ even in this case.

As $Y(v' > v)$ is preserved in all possible cases, the statement holds.

$Q(v' > v)$: Let qc be any QC with $qc.\texttt{view} = v' > v$. We distinguish between two cases:

– Suppose that at least one correct validator contributing to qc did so upon receiving a proposal in view v'. In this case, the received proposal extends proposal p, since $X(v')$ holds. Hence, $Q(v' > v)$ is satisfied.
– Suppose that no correct validator contributing to qc did so upon receiving a proposal in view v'. In this case, all correct votes issued in view v' are based on timeout messages, each corresponding to the local tip of the validator at that time. Since $Y(x)$ holds for all views $x \in [v, v']$, and each validator in the set S updates its local tip to a tip whose view is v and that extends proposal p (from the first paragraph

of the proof) before contributing to qc, it follows that at least one correct validator $i \in S$ must have contributed to qc after having voted for proposal p. If validator i updates its local tip for this vote in view v, then—since a correct validator updates its local tip at most once per view, and it has already done so in view v to a tip that extends p (by the first paragraph of the proof)—the vote must target a proposal that extends p. Otherwise, the vote must also target a proposal that extends p by $X(v^*)$, for any $v^* \in [v+1, v']$. In both cases, $Q(v' > v)$ is satisfied.

Thus, $Q(v' > v)$ holds in both cases.

$Z(v' > v)$: Let tc be any TC with $tc.\texttt{view} = v' > v$ and $tc.\texttt{high_tip} \neq \bot$. Since (1) $|S| \geq f+1$, and (2) tc includes information about $2f+1$ validators, it must be that information about at least one validator $i \in S$ is included in tc. Given that (1) validator i updates its local tip to a tip whose view is v and that extends proposal p (from the first paragraph of the proof) before contributing to tc, and (2) $Y(x)$ holds for every view $x \in [v, v']$, a view $\geq v$ is included in $tc.\texttt{tips_views}$. This further means that $tc.\texttt{high_tip.view} \geq v$. We now consider two possible cases:

- Let $tc.\texttt{high_tip.view} = v$. Since the only proposal issued in view v is p, it follows that $tc.\texttt{high_tip}$ extends proposal p. Therefore, $Z(v' > v)$ is satisfied.
- Let $tc.\texttt{high_tip.view} > v$. Note that $tc.\texttt{high_tip.view} \leq v'$. As $X(tc.\texttt{high_tip.view})$ is satisfied, we know that $tc.\texttt{high_tip}$ extends proposal p and $tc.\texttt{high_tip.block_header.qc.view} \geq v$. Thus, $Z(v' > v)$ holds here as well.

Given that all invariants have been established, the proof is complete. □

The following lemma shows that if a non-equivocating leader issues a fresh proposal p and at least $f+1$ correct validators vote for it, then none of these validators can later vote for any proposal that does not extend proposal p.

Lemma 8. *Let p be any fresh proposal with $p.\texttt{view} = v$, issued by the leader of view v. Suppose that the leader of view v does not equivocate and that at least $f+1$ correct validators vote for p in view v. Then, if any correct validator that voted for p in view v later votes for a proposal p^* in some view $v^* \geq v+1$, the proposal p^* must extend p.*

Proof. Let S denote the set of correct validators, with $|S| \geq f+1$, that voted for fresh proposal p in view v. Assume, for the sake of contradiction, that there exists a correct validator $\ell \in S$ that votes for a proposal p^* in some view $v^* \geq v+1$ such that p^* does not extend p. We know that, in view v, validator ℓ's local tip stores a block header that matches the block header of proposal p's block (by Lemma 3). The fact that ℓ issues a vote for proposal p^* (that does not extend proposal p) in view v^* necessarily implies that ℓ updates its local tip to contain a block header that does not extend proposal p after voting for proposal p. Indeed, if validator ℓ casts its vote for p^* at line 15 of Alg. 1, it updates its local tip at line 13 of the same algorithm; otherwise, if the vote is issued via a timeout message, it is created at line 35 of Alg. 6. If validator ℓ had not updated its local tip to contain a block header that does not extend proposal p after voting for p, then p^* would necessarily extend p. Hence, validator ℓ must have updated its local tip to contain a block header that does not extend proposal p after voting for p.

Note that this update cannot occur in view v: once ℓ votes for p, it sets its highest_voted_view variable to v (line 16 or line 41 of Alg. 1), which prevents it from updating its local tip in view v (line 13 of Alg. 1). Therefore, validator ℓ must update its local tip to contain a block header that does not extend proposal p in some view strictly greater than v. However, Lemma 7 proves that every valid proposal from any view greater than v—and hence every proposal that ℓ could use to update its local tip—must extend p. This contradicts the assumption that validator ℓ updates its local tip in some view greater than v to contain a block header that does not extend p, and it completes the proof. □

We are now ready to prove the tail-forking resistance property.

Theorem 2 (Tail-forking resistance). *If a leader proposes a block B as part of a fresh proposal p and at least $f+1$ correct validators vote for it, then—unless the leader equivocates—any block B' committed by a correct validator must satisfy that either B extends B' or B' extends B.*

Proof. Let v denote p's view. First, observe that every correct validator that votes for proposal p in view v must have received some proposal p^* in view v whose block is identical to $p.\texttt{block}$ (by Lemma 3).

Assume, for contradiction, that the leader does not equivocate and a correct validator i commits a block B' such that B' neither extends B nor is extended by B. Let validator i commit a block B' as part of proposal p', where p' contains B' or strictly extends a proposal that contains B'. This implies the existence of a QC for proposal p' in view $p'.\texttt{view}$ and another QC for the child of p' in view $p'.\texttt{view}+1$. Note that proposals p and p' are conflicting (as B and B' do not extend each other).

From Lemma 8 and the fact that the leader does not equivocate, we have that $p'.\texttt{view} \leq p.\texttt{view}$. Now consider three possible cases:

- $p'.\texttt{view} = p.\texttt{view}$. There exists at least one correct validator that votes for both proposal p and proposal p' in view $p'.\texttt{view} = p.\texttt{view}$. Hence, $p'.\texttt{proposal_id} = p.\texttt{proposal_id}$ by Lemma 1. However, this is impossible as B and B' do not extend each other.
- $p'.\texttt{view}+1 = p.\texttt{view}$: There exists at least one correct validator that votes for both proposal p and for the child c' of proposal p' in view $p'.\texttt{view}+1 = p.\texttt{view}$. Therefore, $c'.\texttt{proposal_id} = p.\texttt{proposal_id}$ by Lemma 1, which further implies that B extends B', which violates the starting assumption.
- $p'.\texttt{view}+1 < p.\texttt{view}$: In this case, since every correct validator that voted for proposal p in view v receives some proposal p^* with $p^*.\texttt{block} = p.\texttt{block}$ (by the first paragraph of the proof), Lemma 4 implies that p^* strictly extends proposal p'. Consequently, the block B contained in p^* strictly extends the block B'. This is, however, impossible as B conflicts with B'.

As neither of the above cases can occur, this completes the proof. □

An immediate corollary of Theorem 2 is that a speculatively finalized block contained in a fresh proposal that has collected a QC can only be reverted if the leader that issued the proposal equivocated.

Corollary 1 (Reversion of speculatively committed blocks). *Suppose a block B contained in a fresh proposal p is speculatively finalized by a correct validator, and another block B' is committed by a correct validator such that neither B extends B' nor B' extends B. Then, the leader of view $p.\texttt{view}$ equivocated.*

Proof. Since block B contained in a fresh proposal p is speculatively finalized, at least $f+1$ correct validators must have voted for p. Assume, for contradiction, that the leader of view $p.\texttt{view}$ does not equivocate and that some block B' is committed by a correct validator such that neither B extends B' nor B' extends B. By Theorem 2, this situation cannot occur (as either B extends B' or B' extends B), which contradicts our assumption that the leader did not equivocate. Hence, the theorem follows. □

A.3 Liveness

This subsection establishes that MonadBFT satisfies the liveness property: every block issued by a correct leader after GST is eventually committed. We emphasize that this guarantee applies to *all* blocks, including those proposed by a correct leader as part of a reproposal. We begin by defining what it means for a correct validator to enter a view.

Definition 1 (Entering a view). *A correct validator is said to* enter *view v when it assigns v to its* cur_view *variable (line 4 of Alg. 3). If this assignment occurs at real time τ, we say that the validator enters view v* at time *τ.*

Next, we define what it means for a correct validator to accept a QC or a TC.

Definition 2 (Accepting a QC or a TC). *A correct validator* accepts *a QC qc or a TC tc if and only if it invokes* IncrementView(qc) *or* IncrementView(tc)*, and this invocation updates its* cur_view *variable to $qc.\texttt{view}+1$ or $tc.\texttt{view}+1$, respectively.*

Recall that a correct validator i accepts a QC qc in one of the following situations:

- upon receiving a proposal that carries it (line 5 of Alg. 1);
- upon forming it locally (line 22 of Alg. 1 and line 19 of Alg. 2);
- upon receiving it directly from another validator j (line 31 of Alg. 1 if validator j is the leader of view $qc.\texttt{view}$ and line 37 of Alg. 1 if validator i is the leader of view $qc.\texttt{view} + 1$);
- upon receiving it within a timeout message (line 5 of Alg. 2).

Similarly, a correct validator accepts a TC in one of the following situations:

- upon receiving a proposal that carries it (line 5 of Alg. 1);
- upon receiving it within a timeout message (line 5 of Alg. 2);
- upon forming it locally (line 24 of Alg. 2);
- upon receiving it directly (line 29 of Alg. 2);
- upon receiving it within a proposal request or an NE request (line 36 of Alg. 2 and line 40 of Alg. 2).

Let $\tau_i(v)$ denote the time at which a correct validator i enters view v, with $\tau_i(v) = \perp$ if validator i never enters that view. In the remainder of this section, we say that a correct validator *fully times out* from a view if it issues a timeout triggered by its own local clock—namely, after Θ_{view} time has elapsed according to that clock[6]—rather than due to the premature timeout invocation at line 15 of Alg. 3. Similarly, we say that a correct validator *prematurely times out* if it triggers the timeout event at line 15 of Alg. 2.

First, we show that the first correct validator to send a timeout message for any view v must have fully timed out from that view (line 40 of Alg. 1) before doing so.

Lemma 9. *Let i be the first correct validator that sends a timeout message for any view $v \geq 1$. Then, validator i has fully timed out from view v before sending the message.*

Proof. Assume, for the sake of contradiction, that validator i does not send a timeout message upon fully timing out from view v. The only remaining possibility is that i sends its timeout message upon prematurely timing out from view v (line 15 of Alg. 3). In this case, validator i must have already received $f + 1$ timeout messages associated with view v. Hence, i cannot be the first correct validator to send a timeout message for view v, which concludes the proof. □

Next, we prove that if any correct validator issues a timeout message associated with any view v, then the validator has previously entered view v.

Lemma 10. *If a correct validator i issues a timeout message for some view $v \geq 1$, then i must have entered view v beforehand.*

Proof. For validator i to issue a timeout message for view v (line 43 of Alg. 1), it must have either fully or prematurely timed out from view v beforehand. If i fully times out from view v, then it necessarily entered view v earlier. Otherwise, i prematurely times out upon receiving $f + 1$ valid timeout messages (line 15 of Alg. 3). Upon receiving the first such message, validator i enters view v (line 5 of Alg. 2). Therefore, the claim holds in both cases. □

The following lemma establishes that a correct validator never issues an invalid timeout message. In particular, if a correct validator issues a timeout message for some view $v \geq 1$, then that message must include either a QC or a TC from view $v - 1$.

Lemma 11. *Suppose that a correct validator i issues a timeout message m associated with some view $v \geq 1$. Then, m is a valid timeout message.*

[6] Note that before GST, although a validator's local clock may measure Θ_{view} time, the actual elapsed real time may differ, since local clocks are permitted to drift prior to GST.

Proof. If $v = 1$, then m is trivially valid, as it includes a genesis QC from view 0.

Consider $v > 1$. For message m to be broadcast (line 43 of Alg. 1), validator i must have already entered view v (by Lemma 10). For validator i to enter view v, it must have first accepted either a QC or a TC corresponding to view $v - 1$ (see the IncrementView procedure at line 1 of Alg. 3). This QC or TC is subsequently included in the timeout message (see the CreateTimeoutMessage function at line 26 of Alg. 6), thereby completing the proof. □

We will implicitly rely on the above lemma throughout the proof. In particular, it guarantees that all timeout messages issued by correct validators are valid and therefore will be processed by every other correct validator.

The following lemma shows that for any view $v > 1$, if a correct validator enters view v, then there must exist some correct validator that previously entered view $v - 1$.

Lemma 12. *Consider any correct validator i and any view $v > 1$ such that $\tau_i(v) \neq \perp$. Then, there exists a correct validator j for which $\tau_j(v-1) \neq \perp$ and $\tau_j(v-1) \leq \tau_i(v)$.*

Proof. If a correct validator i enters view $v > 1$, then a QC or a TC must have been previously formed for view $v - 1$. Therefore, let us consider two possible cases:

- Suppose that a QC is formed in view $v - 1$ before validator i enters view v. Then at least $f + 1$ correct validators must have voted in view $v-1$. Since a correct validator can cast a vote in view $v-1$ only after entering that view—whether the vote is issued upon receiving a proposal (line 15 of Alg. 1) or through a timeout message (by Lemma 10)—the lemma follows in this case.
- Suppose that a TC is formed in view $v - 1$ before validator i enters view v. This implies that at least $f + 1$ correct validators have issued timeout messages in view $v - 1$. Each of these validators must have entered view $v - 1$ before sending its timeout message (by Lemma 10), and thus the lemma follows in this case as well.

Since the lemma holds in both possible cases, the proof is complete. □

The following lemma proves that every view is eventually entered by a correct validator, i.e., no views are "skipped".

Lemma 13. *For every view $v \geq 1$, at least one correct validator enters view v.*

Proof. We prove this by contradiction. Assume that there exists a view that no correct validator enters, and let $v_{\min}$ denote the smallest such view. Clearly, $v_{\min} > 1$, since every correct validator starts at view 1. By Lemma 12, no correct validator can enter any view greater than $v_{\min}$. Moreover, by the minimality of $v_{\min}$, every view smaller than $v_{\min}$ is entered by at least one correct validator. Let i be a correct validator that enters view $v_{\min} - 1$. Since validator i never enters any view greater than $v_{\min} - 1$, it eventually issues a timeout message for view $v_{\min} - 1$ (line 43 of Alg. 1) upon timing out from view $v_{\min} - 1$.

Now, consider two cases: (i) $v_{\min} = 2$ and (ii) $v_{\min} > 2$. When $v_{\min} = 2$, then note that all correct validators start at $v_{\min} - 1 = 1$. Alternatively, when $v_{\min} > 2$, the aforementioned timeout message associated with view $v_{\min} - 1$ carries either a QC or a TC from view $v_{\min} - 2$ (as validator i entered view $v_{\min} - 1$ upon accepting a QC or a TC from view $v_{\min} - 2$, by Lemma 11). Upon receiving this message, every correct validator that has not yet entered view $v_{\min} - 1$ enters view $v_{\min} - 1$ (line 5 of Alg. 2). Therefore, in both possible cases, all correct validators eventually enter view $v_{\min} - 1$.

Since no correct validator ever enters any view greater than $v_{\min} - 1$, all correct validators eventually broadcast their timeout messages for view $v_{\min} - 1$ (line 43 of Alg. 1, either because they fully or prematurely time out from view $v_{\min} - 1$). Given that there are at least $2f + 1$ correct validators and $2f + 1$ timeout messages are required to form a TC, at least one correct validator eventually constructs a QC or a TC for view $v_{\min} - 1$ and enters view $v_{\min}$ (line 19 or line 24 of Alg. 2). This contradicts the starting assumption and concludes the proof. □

Lemma 13 is important because it guarantees that every view is eventually entered by at least one correct validator. We assume this lemma implicitly in the remainder of the proof.

Let $\tau(v)$ denote the time at which the first correct validator enters view v, i.e.,

$$\tau(v) \stackrel{\text{def}}{=} \min\big(\{\tau_i(v) \mid i \text{ is a correct validator and } \tau_i(v) \neq \bot\}\big).$$

By Lemma 13, $\tau(v)$ is well-defined for every view v. Next, we define a post-GST view.

Definition 3 (Post-GST view). *A view v is a* post-GST view *if and only if $\tau(v) \geq GST$.*

Without loss of generality, we assume that all correct validators start MonadBFT before GST. Therefore, any post-GST view v must satisfy $v > 1$ (because correct validators start at view 1 implying $\tau(1) < \text{GST}$).

Let Θ_{view} denote the local time each correct validator waits before issuing a timeout message for a given view. Stating differently, Θ_{view} represents the maximum time a correct leader needs to successfully collect and disseminate a QC in any post-GST view. In MonadBFT, we require Θ_{view} to be

$$\Theta_{\text{view}} = 3\Delta + \Theta_{\text{recovery}} + 3\Delta,$$

where Δ is the upper bound on message delays after GST and Θ_{recovery} denotes the time (after GST) required for a correct leader to construct or recover a proposal once it has obtained a QC or a TC from the preceding view (after GST). Intuitively, the first 3Δ term accounts for the delay that may occur before a correct leader enters the view — this can take up to 3Δ after the first correct validator has entered (as proven by Lemma 17). Additionally, the leader may require up to Θ_{recovery} time to reconstruct the block it must repropose or to generate an NEC, in the case of the unhappy path. Concretely, $\Theta_{\text{recovery}} = \left(\left\lceil \frac{n}{\kappa} \right\rceil - 1\right) \cdot \Theta_{\text{interval}} + 2\Delta$, where κ and Θ_{interval} are parameters of the recovery algorithm (see Alg. 7): proposal requests are disseminated to κ validators at a time, with Θ_{interval} time between successive batches. Therefore, $\left(\left\lceil \frac{n}{\kappa} \right\rceil - 1\right) \cdot \Theta_{\text{interval}}$ is sufficient to send proposal and NE requests to all validators. These requests are received by all validators within an additional Δ time, and their responses are received by the leader within another Δ, for a total of 2Δ communication delay. Thus, the total time required for this recovery process is $\Theta_{\text{recovery}} = \left(\left\lceil \frac{n}{\kappa} \right\rceil - 1\right) \cdot \Theta_{\text{interval}} + 2\Delta$.

Recall that when the leader obtains a QC, it is assumed to simultaneously obtain the proposal referenced by that QC, allowing it to issue a new proposal immediately upon obtaining the QC. Finally, the leader needs an additional Δ time to disseminate its proposal, Δ time to collect votes and form a QC, and another Δ time to disseminate the newly formed QC.

The following lemma establishes that no correct validator issues a timeout message associated with any post-GST view v before time $\tau(v) + \Theta_{\text{view}}$.

Lemma 14. *Let $v > 1$ be any post-GST view. Then, no correct validator issues a timeout message for view v before time $\tau(v) + \Theta_{\text{view}}$.*

Proof. By Lemma 9, the first correct validator to issue a timeout message for view v must do so only after fully timing out from that view. Since no correct validator enters view v before time $\tau(v)$, this event cannot occur earlier than $\tau(v) + \Theta_{\text{view}}$. □

We now prove that no TC can be formed for any post-GST view v before time $\tau(v) + \Theta_{\text{view}}$.

Lemma 15. *Let $v > 1$ be any post-GST view. Then, no TC for view v can be formed before time $\tau(v) + \Theta_{\text{view}}$.*

Proof. For a TC to be formed, at least $f + 1$ timeout messages must be issued by correct validators. Thus, the lemma follows directly from Lemma 14. □

The following lemma establishes that, for any post-GST view v, no correct validator can enter any view $v' > v$ before time $\tau(v) + \Theta_{\text{view}}$, unless at least one correct validator accepts a QC for view v prior to $\tau(v) + \Theta_{\text{view}}$.

Lemma 16. *Let $v > 1$ be any post-GST view. If there exists a view $v' > v$ such that $\tau(v') < \tau(v) + \Theta_{\text{view}}$, then there must exist a correct validator that accepts a QC for view v before time $\tau(v) + \Theta_{\text{view}}$.*

Proof. We prove this by contradiction. Suppose there exists a view $v' > v$ that is entered by a correct validator before time $\tau(v) + \Theta_{\text{view}}$, and that no correct validator accepts a QC for view v before $\tau(v) + \Theta_{\text{view}}$. By Lemma 12, it follows that $\tau(v+1) < \tau(v) + \Theta_{\text{view}}$. Let i be a correct validator that enters view $v+1$ at time $\tau(v+1)$. We consider two possibilities:

- Suppose that validator i enters view $v+1$ upon accepting a QC for view v. This contradicts the assumption that no correct validator accepts a QC for view v before $\tau(v) + \Theta_{\text{view}}$, rendering this case impossible.
- Suppose that validator i enters view $v+1$ upon accepting a TC for view v. This case is impossible due to Lemma 15.

As neither case is possible, the contradiction completes the proof. □

The following lemma establishes that every correct validator enters a post-GST view v with a correct leader within at most 3δ time from the moment the first correct validator entered view v, or that at least one correct validator has accepted a QC for view v by that time. Recall that $\delta \leq \Delta$ denotes the actual network delay.

Lemma 17. *Let $v > 1$ be any post-GST view with a correct leader. If no correct validator accepts a QC for view v by time $\tau(v) + 3\delta$, then every correct validator enters view v by time $\tau(v) + 3\delta$.*

Proof. We first prove that if no correct validator accepts a QC for view v by time $\tau(v) + 3\delta$, then no correct validator can enter any view $\geq v+1$ by that time. The reasoning is as follows. By Lemma 15, no TC can be formed for view v before time $\tau(v) + \Theta_{\text{view}} > \tau(v) + 3\Delta \geq \tau(v) + 3\delta$. Hence, no correct validator can enter view $v+1$ by time $\tau(v) + 3\delta$, since a validator can enter a new view only upon accepting either a QC or a TC associated with view v. Consequently, by Lemma 12, no correct validator enters a view $\geq v+1$ by time $\tau(v) + 3\delta$.

Since a correct validator enters view v at time $\tau(v)$, it must have accepted either a QC or a TC associated with view $v-1$ at that time. To establish the lemma, we therefore consider two possible cases:

- *Case 1:* Suppose that some correct validator i accepts a QC for view $v-1$ by time $\tau(v) + \delta$ at line 5 of Alg. 1 or at line 22 of Alg. 1 or at line 31 of Alg. 1 or at line 37 of Alg. 1 or at line 5 of Alg. 2. This accounts for all situations in which a correct validator may accept a QC, except for the case where it is formed from $2f+1$ timeout messages (line 19 of Alg. 2), which will be addressed separately in the second case.
  We now differentiate two scenarios:
  - Let validator i be the leader of view v. Note that validator i cannot accept a QC for view $v-1$ at line 5 of Alg. 1, since this would require the leader to have received a proposal from itself. However, issuing a proposal is only possible after the validator has already entered view v (see Alg. 1 and Alg. 2).
    In all remaining cases, the leader issues its proposal immediately upon accepting a QC (line 27 or line 39 of Alg. 1, or line 8 of Alg. 2). Consequently, all correct validators enter view v at line 5 of Alg. 1 by time $\tau(v) + 2\delta$, unless they have already done so, and thus the lemma follows in this case.
  - Let validator i be a non-leader in view v. In this case, we first show that the leader of view v issues a proposal (or a proposal request or an NE request) by time $\tau(v) + 2\delta$. If validator i accepts a QC at line 5 of Alg. 1, then the corresponding proposal must have been received from the leader of view v, implying that the leader issued its proposal by time $\tau(v) + \delta < \tau(v) + 2\delta$. If validator i accepts a QC at any other point in the protocol (covered by this case), it forwards that QC to the leader of view v (line 25 or line 33 of Alg. 1 or line 15 of Alg. 2). Hence, the leader of view v receives a QC associated with view $v-1$ by time $\tau(v) + 2\delta$. If the leader has not yet entered view v and issued a proposal (or a proposal request or an NE request) by this time, it accepts the QC (line 37 of Alg. 1) and then issues its proposal (line 39 of Alg. 1).

Therefore, every correct validator receives a proposal (or a proposal request or an NE request) from the leader of view v by time $\tau(v) + 3\delta$. Consequently, if not already done so, each correct validator enters view v by time $\tau(v) + 3\delta$—either at line 5 of Alg. 1, or at line 36 or line 40 of Alg. 2.

– *Case 2:* Suppose that no correct validator i accepts a QC for view $v - 1$ by time $\tau(v) + \delta$ at line 5 of Alg. 1 or at line 22 of Alg. 1 or at line 31 of Alg. 1 or at line 37 of Alg. 1 or at line 5 of Alg. 2. In this case, any correct validator that entered view v at time $\tau(v)$ must have done so either by accepting a TC for view $v - 1$ or by accepting a QC for view $v - 1$ at line 19 of Alg. 2 upon receiving $2f + 1$ timeout messages associated with view $v - 1$. We again consider two possible scenarios:
  - Suppose that the leader of view v issues its proposal (or proposal request or NE request) by time $\tau(v) + \delta$. Then, every correct validator receives this message by time $\tau(v) + 2\delta$, ensuring that all correct validators enter view v by that time (at line 5 of Alg. 1 or at line 36 of Alg. 2 or at line 40 of Alg. 2).
  - Suppose that the leader of view v does not issue its proposal (or proposal request or NE request) by time $\tau(v) + \delta$. In this case, we show that every correct validator either issues a timeout message associated with view $v - 1$ by time $\tau(v) + \delta$ or broadcasts a TC by the same time. Since at least $f + 1$ timeout messages are sent by correct validators by time $\tau(v)$, every correct validator receives $f + 1$ timeout messages by time $\tau(v) + \delta$. Upon receiving the first $f + 1$ timeout messages for view $v - 1$, one of the following must occur for each correct validator i:
    * Validator i sends a timeout message for view $v - 1$ after fully or prematurely timing out from view $v - 1$ by this moment.
    * Otherwise, validator i has already entered view v (as the check at line 1 of Alg. 2 fails). Recall that no validator can enter any view $\geq v + 1$ by this time, as argued in the first paragraph of the proof. Since no correct validator accepts a QC for view $v - 1$ at line 5 of Alg. 1 or at line 22 of Alg. 1 or at line 31 of Alg. 1 or at line 37 of Alg. 1 or at line 5 of Alg. 2 nor does the leader issue any proposal (or proposal request or NE request) by time $\tau(v) + \delta$, validator i must have entered view v upon accepting a TC at line 5 or line 29 of Alg. 2. (Note that if validator i moved to view v upon accepting a QC or a TC at line 19 or line 24 of Alg. 2, then it must have already issued a timeout message for view $v - 1$—a situation already covered above—since it must first time out from view $v - 1$ before transitioning to view v.) In this case, if validator i has not already issued a timeout message for view $v - 1$, it broadcasts the accepted TC (line 4 or line 31 of Alg. 2).

  Hence, every correct validator either issues a timeout message associated with view $v - 1$ by time $\tau(v) + \delta$ or broadcasts a TC associated with view $v - 1$ by that time. Finally, we distinguish two cases:
    * Suppose that some correct validator broadcasts a TC by time $\tau(v) + \delta$. Then every correct validator receives a TC associated with view $v - 1$ by time $\tau(v) + 2\delta$. In this case, unless already in view v, each correct validator accepts the TC and enters view v (line 29 of Alg. 2).
    * Otherwise, all correct validators receive $2f + 1$ timeout messages by time $\tau(v) + 2\delta$. If a correct validator has not yet entered view v by this time, it does so at line 19 or line 24 of Alg. 2, upon constructing a TC or a QC associated with view $v - 1$ (line 17 of Alg. 2).

Since both cases satisfy the lemma's claim, the proof is complete. □

Next, we show that for any post-GST view v with a correct leader, the leader enters view v by time $\tau(v) + 3\delta$.

Lemma 18. *Let $v > 1$ be any post-GST view with a correct leader. Then, the leader of view v enters view v by time $\tau(v) + 3\delta$.*

Proof. Let us consider two possible cases:

– *Case 1:* No correct validator accepts a QC for view v by time $\tau(v) + 3\delta$.
In this case, by Lemma 17, every correct validator (including the leader) enters view v by time $\tau(v) + 3\delta$.

– *Case 2:* Some correct validator accepts a QC for view v by time $\tau(v) + 3\delta$.
Note that no correct validator issues a timeout message for view v before time $\tau(v) + \Theta_{\text{view}} > \tau(v) + 3\delta$ (by Lemma 14). Therefore, every correct validator that contributed to the aforementioned QC must have done so upon receiving the leader's proposal for view v. Consequently, the leader of view v must have entered view v prior to that time. □

We now show that, in every post-GST view v with a correct leader, the leader issues a proposal by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + \delta$.

Lemma 19. *Let $v > 1$ be any post-GST view with a correct leader. If the leader of view v enters view v upon accepting a QC associated with view $v - 1$, then the leader issues its proposal by time*

$$\tau(v) + 3\delta.$$

Alternatively, if the leader of view v enters view v upon accepting a TC associated with view $v - 1$, then the leader issues its proposal by time

$$\tau(v) + 3\delta + \Theta_{\text{recovery}}.$$

Proof. By Lemma 18, the leader L of view v enters view v by time $\tau(v) + 3\delta$. If the leader L enters view v upon accepting a QC from view $v - 1$, then L issues its proposal immediately. This directly establishes the claim of the lemma in this case. (Recall that we assume the leader, upon obtaining a QC, simultaneously receives the proposal referenced by that QC. This enables the leader to issue a new proposal without delay.)

If the leader L enters view v upon accepting a timeout certificate tc from view $v - 1$, and either (i) $tc.\texttt{high_qc} \neq \perp$, or (ii) the leader already possesses a block corresponding to $tc.\texttt{high_tip}$, then L issues its proposal immediately (see the CreateProposalBasedOnTC function).

Finally, it remains to consider the case where the leader does not possess a block corresponding to $tc.\texttt{high_tip}$. Note that in this case, every correct validator receives both an NE request and a proposal request from the leader L by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} - \Delta$ (see Alg. 7). In this situation, we distinguish between two subcases:

– *Case 1:* Some correct validator enters some view $> v$ by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} - \Delta$.
By Lemma 12, we have

$$\tau(v+1) < \tau(v) + 3\delta + \Theta_{\text{recovery}} - \Delta < \tau(v) + \Theta_{\text{view}}.$$

Therefore, some correct validator must have accepted either a QC or a TC for view v by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} - \Delta$. However, by Lemma 15, no TC for view v can be formed before time $\tau(v) + \Theta_{\text{view}} > \tau(v) + 3\delta + \Theta_{\text{recovery}} - \Delta$. It follows that some correct validator must have accepted a QC for view v by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} - \Delta$. Since no correct validator issues a timeout message before time $\tau(v) + \Theta_{\text{view}} > \tau(v) + 3\delta + \Theta_{\text{recovery}} - \Delta$ (by Lemma 14), it follows that every correct validator that contributed to the aforementioned QC did so upon receiving a proposal from the leader, which concludes the proof.

– *Case 2:* No correct validator enters any view $> v$ by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} - \Delta$.
In this case, every correct validator sends either the block corresponding to $tc.\texttt{high_tip}$ or an NE message back to the leader by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} - \Delta$. (This is because, upon receiving an NE request or a proposal request, the validator cannot have entered any view $> v$, as these messages are received by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} - \Delta$.) Hence, by time $\tau(v) + 3\delta + \Theta_{\text{recovery}}$, the leader L either obtains the block corresponding to $tc.\texttt{high_tip}$ (if at least one correct validator possesses the block) or constructs an NEC (if no correct validator possesses the block). Consequently, by the same time, the leader L issues its proposal. □

Next, we show that for any post-GST view v with a correct leader, if no correct validator accepts a QC for view v by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + \delta$, then all correct validators cast their votes for a proposal associated with view v by that time.

Lemma 20. *Let $v > 1$ be any post-GST view with a correct leader, and let p be the proposal issued by that leader in view v. If $p.\texttt{tc} = \perp$ (resp., $p.\texttt{tc} \neq \perp$) and no correct validator accepts a QC for view v by time $\tau(v)+3\delta+\delta$ (resp., by time $\tau(v)+3\delta+\Theta_{\text{recovery}}+\delta$), then every correct validator issues its vote for proposal p by time $\tau(v)+3\delta+\delta$ (resp., by time $\tau(v)+3\delta+\Theta_{\text{recovery}}+\delta$).*

Proof. Note that, by Lemma 15, no TC for view v can be formed by time $\tau(v)+3\delta+\Theta_{\text{recovery}}+\delta$. Since a validator can enter a new view only upon accepting either a QC or a TC, this fact, together with the lemma's assumption, implies that no correct validator enters view $v+1$ or any higher view (as proven by Lemma 12) by time $\tau(v)+3\delta+\delta$ if $p.\texttt{tc} = \perp$ and by time $\tau(v)+3\delta+\Theta_{\text{recovery}}+\delta$ if $p.\texttt{tc} \neq \perp$.

Let us now differentiate two cases:

- Let $p.\texttt{tc} = \perp$. By Lemma 19, the leader issues its proposal p by time $\tau(v)+3\delta$. This proposal is received by every correct validator by time $\tau(v)+3\delta+\delta$, upon which they cast their votes for proposal p, entering view v first if they have not already done so. From the first paragraph, we know that no correct validator enters any view beyond v before this time. Hence, whenever a correct validator receives proposal p, it enters view v (unless it has already done so) and subsequently votes for it. Thus, the lemma follows in this case.
- Let $p.\texttt{tc} \neq \perp$. By Lemma 19, the leader issues its proposal p by time $\tau(v)+3\delta+\Theta_{\text{recovery}}$. This proposal is received by every correct validator by time $\tau(v)+3\delta+\Theta_{\text{recovery}}+\delta$, upon which they cast their votes for proposal p, entering view v first if they have not already done so. From the first paragraph, we know that no correct validator enters any view beyond v before this time. Hence, whenever a correct validator receives proposal p, it enters view v (unless it has already done so) and subsequently votes for it. Thus, the lemma follows. □

The following lemma proves that when leaders of two consecutive post-GST views are honest, then the leader of the latter view receives a QC within a bounded amount of time.

Lemma 21. *Let $v > 1$ and $v+1$ be two consecutive post-GST views, each led by a correct leader. Suppose that the leader of view v enters view v upon accepting a QC associated with view $v-1$. Then, the leader of view $v+1$ enters view $v+1$ at some time τ_L upon accepting a QC for view v. Moreover, the following bounds hold:*

$$\tau_L \leq \tau(v)+3\delta+2\delta,$$

and moreover,

$$\tau_L \leq \tau(v+1)+\delta.$$

Proof. Let L_v denote the leader of view v and L_{v+1} denote the leader of view $v+1$. Recall that $\Theta_{\text{view}} > 3\delta+2\delta$. By Lemma 15, a TC for view v cannot be formed before $\tau(v)+\Theta_{\text{view}} > \tau(v)+3\delta+2\delta$.

We distinguish two cases:

- *Case 1:* Suppose some correct validator accepts a QC for view v before time $\tau(v)+3\delta+\delta$.
  Let i denote the first correct validator to accept a QC for view v, occurring at some time $\tau_i^{qc} < \tau(v)+3\delta+\delta$. Observe that $\tau(v+1) = \tau_i^{qc}$, as no TC for view v can be formed by this time and i is the first correct validator to accept a QC for view v. Moreover, that validator i can accept this QC in one of the following ways: (i) upon receiving a proposal that carries this QC (line 5 of Alg. 1), (ii) upon forming the QC locally (line 22 of Alg. 1), (iii) upon receiving a QC directly (line 31 or line 37 of Alg. 1), or (iv) upon receiving a timeout message containing this QC (line 5 of Alg. 2). Validator i cannot, however, accept this QC at line 19 of Alg. 2, as doing so would require that at least one correct validator had already sent a timeout message. This is impossible, since no correct validator issues a timeout message before time $\tau(v)+\Theta_{\text{view}} > \tau(v)+3\delta+\delta$ (by Lemma 14).
  If validator i is the leader of view $v+1$, the lemma follows immediately. Otherwise, suppose that i is not the leader of view $v+1$.
  We now show that the leader L_{v+1} of view $v+1$ receives or accepts a QC associated with view v by time $\tau_i^{qc}+\delta$. Consider all possible points at which validator i could have accepted a QC for view v:

- If i accepts the QC at line 5 of Alg. 1, then it must have received a proposal carrying this QC from the leader L_{v+1}, which implies that the leader had already accepted the QC beforehand.
- If i accepts the QC at line 22 of Alg. 1, it forwards the QC to all validators, including L_{v+1} (line 25 of Alg. 1), ensuring that the leader receives a QC for view v by time $\tau_i^{qc} + \delta$. Recall that $i \neq L_{v+1}$.
- If i accepts the QC at line 31 of Alg. 1, it forwards the QC to the leader L_{v+1} (line 33 of Alg. 1), which again ensures that the leader receives a QC for view v by time $\tau_i^{qc} + \delta$.
- If i accepts the QC at line 5 of Alg. 2, it forwards the QC to L_{v+1} (line 15 of Alg. 2), and thus the same bound applies.

Therefore, in all cases, the leader L_{v+1} either receives or accepts a QC associated with view v by time $\tau_i^{qc} + \delta$. If it accepts such a QC by this time, the lemma follows. Therefore, suppose that L_{v+1} does not accept any QC for view v by time $\tau_i^{qc} + \delta$. In this case, it still receives the QC for view v by that time. Let $\tau_{L_{v+1}}^{qc} \leq \tau_i^{qc} + \delta \leq \tau(v+1) + \delta < \tau(v) + 3\delta + 2\delta$ denote the first time L_{v+1} receives a QC from view v. We now distinguish two possible cases concerning the time $\tau_{L_{v+1}}^{qc}$:

- Suppose the current view of L_{v+1} at time $\tau_{L_{v+1}}^{qc}$ is $v+1$. Since no TC for view v can be formed before time $\tau(v) + 3\delta + 2\delta > \tau_{L_{v+1}}^{qc}$, it would imply that L_{v+1} entered view $v+1$ upon accepting a QC from view v. However, this is impossible, as L_{v+1} does not accept any QC before time $\tau_{L_{v+1}}^{qc}$.
- Suppose that the current view of L_{v+1} at time $\tau_{L_{v+1}}^{qc}$ is greater than $v+1$. Hence, Lemma 12 proves that $\tau(v+2) \leq \tau_{L_{v+1}}^{qc} < \tau(v) + 3\delta + 2\delta$. Since no TC can be formed for view $v+1$ before time $\tau(v+1) + \Theta_{\text{view}} \geq \tau(v) + \Theta_{\text{view}} > \tau_{L_{v+1}}^{qc}$, any correct validator that has entered view $v+2$ must have done so due to a QC from view $v+1$. Moreover, as no correct validator issues a timeout message for view $v+1$ before $\tau(v+1) + \Theta_{\text{view}} > \tau_{L_{v+1}}^{qc}$, every correct validator contributing to the aforementioned QC from view $v+1$ must have done so after receiving a proposal from L_{v+1}. This implies that L_{v+1} must have entered view $v+1$ before $\tau_{L_{v+1}}^{qc}$, since this QC for view $v+1$ can only be formed after L_{v+1} has issued its proposal. Furthermore, L_{v+1} must have entered view $v+1$ upon accepting a QC for view v (as no TC associated with view v can be formed by this time), which contradicts the fact that L_{v+1} does not accept any QC associated with view v by time $\tau_{L_{v+1}}^{qc}$.

Since both cases are impossible, the current view of L_{v+1} upon receiving the first QC from view v at time $\tau_{L_{v+1}}^{qc} \leq \tau(v+1) + \delta$ must be smaller than $v+1$. Hence, the leader L_{v+1} accepts a QC at this time (line 37 of Alg. 1).

– *Case 2:* Suppose no correct validator accepts a QC for view v before time $\tau(v) + 3\delta + \delta$.
Given that no TC can be formed before time $\tau(v) + \Theta_{\text{view}} > \tau(v) + 3\delta + \delta$, we have that $\tau(v+1) \geq \tau(v) + 3\delta + \delta$. Moreover, by Lemma 20, all correct validators vote for the proposal issued by the leader of view v by that time. Consequently, the leader of view $v+1$ receives $2f+1$ votes from correct validators by time $\tau(v) + 3\delta + 2\delta$. Let $\tau_{L_{v+1}}^{qc} \leq \tau(v) + 3\delta + 2\delta$ denote the earliest time at which the leader L_{v+1} receives $2f+1$ votes for view v from correct validators. If validator i accepts a QC for view v before time $\tau_{L_{v+1}}^{qc}$, the lemma follows immediately. Otherwise, assume that validator i does not accept any QC for view v before time $\tau_{L_{v+1}}^{qc}$. Again, we consider two possibilities:

- Suppose the current view of L_{v+1} at time $\tau_{L_{v+1}}^{qc}$ is $v+1$. Since no TC for view v can be formed by time $\tau(v) + 3\delta + 2\delta \geq \tau_{L_{v+1}}^{qc}$, it follows that L_{v+1} would have accepted a QC for view v by that time, which contradicts the above assumption.
- Suppose that the current view of L_{v+1} at time $\tau_{L_{v+1}}^{qc}$ is greater than $v+1$. Hence, Lemma 12 proves that $\tau(v+2) \leq \tau_{L_{v+1}}^{qc} < \tau(v) + 3\delta + 2\delta$. Since no TC can be formed for view $v+1$ before time $\tau(v+1) + \Theta_{\text{view}} \geq \tau(v) + \Theta_{\text{view}} > \tau_{L_{v+1}}^{qc}$, any correct validator that has entered view $v+2$ must have done so due to a QC from view $v+1$. Moreover, as no correct validator issues a timeout message for view $v+1$ before $\tau(v+1) + \Theta_{\text{view}} > \tau_{L_{v+1}}^{qc}$, every correct validator contributing to the aforementioned QC from view $v+1$ must have done so after receiving a proposal from L_{v+1}. This implies that L_{v+1} must have entered view $v+1$ before $\tau_{L_{v+1}}^{qc}$, since this QC for view $v+1$ can only be formed after L_{v+1} has issued its proposal. Furthermore, L_{v+1} must have entered view $v+1$ upon accepting a QC for view v (as no TC associated with view v can be formed by this time), which contradicts the fact that L_{v+1} does not accept any QC associated with view v by time $\tau_{L_{v+1}}^{qc}$.

Since both cases are impossible, the current view of L_{v+1} at time $\tau^{qc}_{L_{v+1}}$ must be smaller than $v+1$. Therefore, at this moment, the leader L_{v+1} forms a QC for view v (line 20 of Alg. 1) and enters view $v+1$ upon accepting this QC (line 22 of Alg. 1).

Since the statement holds in both cases, the proof is complete. □

Next, we show that in any sequence of $3f+3$ consecutive views, there exists at least one group of three consecutive views in which each view has a correct leader. This is crucial, as the commitment rule of the MonadBFT protocol requires two consecutive QCs to commit a block. Hence, the existence of three consecutive views with correct leaders guarantees that these two consecutive QCs are both constructed (during the first two views) and subsequently disseminated (during the third view).

Lemma 22. *In any sequence of $3f+3$ consecutive views, there exists a group of three consecutive views such that each view has a correct leader.*

Proof. Consider any sequence of $3f+3$ consecutive views. This sequence contains $3f+3-2 = 3f+1$ distinct groups of three consecutive views. By the round-robin schedule, each of the $3f+1$ validators appears in exactly three of these groups. Since there are at most f faulty validators, the pigeonhole principle implies that at most $3f$ groups can include a view led by a faulty validator. Therefore, there exists at least one group in which all three views have correct leaders. □

Next, we prove that no TC can be constructed for any post-GST view with a correct leader.

Lemma 23. *Let $v > 1$ be any post-GST view with a correct leader. Then, no TC can be formed for view v. Furthermore, at least $f+1$ correct validators do not issue timeout messages associated with view v.*

Proof. Let us consider the two following cases:

- *Case 1:* At least $f+1$ correct validators enter views greater than v before time $\tau(v) + \Theta_{\text{view}}$.
  Since no correct validator can issue a timeout message before $\tau(v)+\Theta_{\text{view}}$ (by Lemma 14), those validators that advance to higher views before this time will never issue a timeout message for view v: they can neither fully nor prematurely time out from view v prior to time $\tau(v)+\Theta_{\text{view}}$. As at least $2f+1$ timeout messages are required to form a TC, no TC for view v can be constructed in this case, which proves the lemma.
- *Case 2:* At most f correct validators enter views greater than v before time $\tau(v) + \Theta_{\text{view}}$.
  In this case, we first prove that no correct validator can enter any view $> v+1$ before time $\tau(v)+\Theta_{\text{view}}$. Since $\tau(v+1) \geq \tau(v)$ (by Lemma 12) and no TC can be formed for view $v+1$ before time $\tau(v+1)+\Theta_{\text{view}} \geq \tau(v) + \Theta_{\text{view}}$ (by Lemma 15), the only way a correct validator could enter a view $> v+1$ before time $\tau(v) + \Theta_{\text{view}}$ is by accepting a QC from view $v+1$. Moreover, since no correct validator times out from view $v+1$ before $\tau(v+1)+\Theta_{\text{view}} \geq \tau(v)+\Theta_{\text{view}}$ (by Lemma 14), every correct validator that contributes to any QC from view $v+1$ formed before time $\tau(v) + \Theta_{\text{view}}$ must have done so upon processing the leader's proposal for view $v+1$ (line 15 of Alg. 1). As forming such a QC requires at least $f+1$ correct validators to have voted, this would imply that $f+1$ or more correct validators entered views greater than v before time $\tau(v) + \Theta_{\text{view}}$ (line 5 of Alg. 1), which contradicts the case assumption. Therefore, all correct validators that transition to future views before time $\tau(v) + \Theta_{\text{view}}$ do so only to view $v+1$.
  Let L denote the leader of view v. We further distinguish two scenarios:
  - Suppose that L disseminates a QC formed for view v to all validators before time $\tau(v) + \Theta_{\text{view}} - \Delta$ (by invoking BroadcastBackupQC at line 9 or line 25 or line 35 of Alg. 1 or line 13 of Alg. 2).
    In this case, every correct validator i receives from the leader L a QC formed for view v by time $\tau(v) + \Theta_{\text{view}}$. If the current view of validator i is $v+1$, then i has already entered view $v+1$ and therefore never issues a timeout message for view v. If the current view of validator i is $< v+1$, validator i accepts the received QC (line 31 of Alg. 1) and enters view v, which again implies that it will not issue a timeout message for that view. (Recall that the current view of validator i cannot exceed $v+1$.) Hence, in both cases, validator i never issues a timeout message for view v, and therefore no correct validator does so, implying that no TC can be formed for view v.

- Suppose that L does not disseminate a QC formed for view v to all validators before time $\tau(v) + \Theta_{\text{view}} - \Delta$ (by invoking BroadcastBackupQC at line 9 or line 25 or line 35 of Alg. 1 or line 13 of Alg. 2).
  For this scenario, we analyze two possible cases according to Lemma 20:
  * Suppose that some correct validator i accepts a QC for view v by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + \delta$. Note that this correct validator i cannot accept a QC for view v at line 19 of Alg. 2, since it is impossible to receive $2f + 1$ timeout messages for view v by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + \delta < \tau(v) + \Theta_{\text{view}}$ (as no correct validator issues a timeout message associated with view v before time $\tau(v) + \Theta_{\text{view}}$ according to Lemma 14). We now consider all possible points in the protocol where validator i could accept a QC for view v:
    · Suppose that validator i accepts a QC for view v at line 5 of Alg. 1. In this case, i forwards the accepted QC to the leader L (line 10 of Alg. 1). This implies that the leader L receives a QC for view v by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + 2\delta \leq \tau(v) + \Theta_{\text{view}} - \Delta$. Upon receiving this QC, if the current view of L is $v + 1$ (recall that it cannot be greater), then L has already disseminated a QC for view v (at line 9, line 25, line 35 of Alg. 1, or line 13 of Alg. 2). Otherwise, L processes the received QC and disseminates it at line 35 of Alg. 1. In both cases, we reach a contradiction with the assumption that L does not disseminate a QC formed in view v by time $\tau(v) + \Theta_{\text{view}} - \Delta$, making this case impossible.
    · Suppose that validator i accepts a QC for view v at line 22 of Alg. 1. In this case, i is either the leader of view v or the leader of view $v + 1$. If i is the leader of view v (i.e., $i = L$), then it disseminates the QC formed in view v at line 25 of Alg. 1, which contradicts the initial assumption that this does not occur.
      Otherwise, i is the leader of view $v + 1$, in which case it issues its proposal that is received by all correct validators by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + 2\delta < \tau(v) + \Theta_{\text{view}}$. Consequently, by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + 2\delta$, all correct validators have entered view $v + 1$, implying that no correct validator issues a timeout message for view v. Hence, the lemma follows in this case.
    · Suppose that validator i accepts a QC for view v at line 31 of Alg. 1. In this case, i must have received the QC from the leader L, which implies that L had previously disseminated the QC formed for view v (at line 9, line 25, line 35 of Alg. 1, or line 13 of Alg. 2). This contradicts the initial assumption, and therefore this case cannot occur.
    · Suppose that validator i accepts a QC for view v at line 37 of Alg. 1. In this case, i is the leader of view $v + 1$. Therefore, validator i issues its proposal that is received by all correct validators by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + 2\delta < \tau(v) + \Theta_{\text{view}}$. Consequently, by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + 2\delta$, all correct validators have entered view $v + 1$, implying that no correct validator issues a timeout message for view v. Hence, the lemma follows in this case.
    · Suppose that validator i accepts a QC for view v at line 5 of Alg. 2. In this case, i forwards the accepted QC to the leader L (line 14 of Alg. 2). This implies that the leader L receives a QC for view v by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + 2\delta \leq \tau(v) + \Theta_{\text{view}} - \Delta$. Upon receiving this QC, if the current view of L is $v + 1$ (recall that it cannot be greater), then L has already disseminated a QC for view v (at line 9, line 25, line 35 of Alg. 1, or line 13 of Alg. 2). Otherwise, L processes the received QC and disseminates it at line 35 of Alg. 1. In both cases, we reach a contradiction with the assumption that L does not disseminate a QC formed in view v by time $\tau(v) + \Theta_{\text{view}} - \Delta$, making this case impossible.

    Hence, in all possible scenarios, the lemma holds.
  * Suppose that no correct validator accepts a QC for view v by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + \delta$.
    By Lemma 20, all correct validators issue their votes by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + \delta$, which implies that the leader L receives votes from all correct validators by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + 2\delta \leq \tau(v) + \Theta_{\text{view}} - \Delta$. Upon receiving these votes, if the current view of L is $v + 1$ (recall that it cannot be greater), then L has already disseminated a QC for view v (at line 9, line 25, line 35 of Alg. 1, or line 13 of Alg. 2). Otherwise, L processes the received votes and disseminates the QC for view v (formed from these votes) at line 25 of Alg. 1. Hence, in both cases, we obtain a

contradiction with the assumption that L does not disseminate a QC formed in view v by time $\tau(v) + \Theta_{\text{view}} - \Delta$, making this case impossible.

As the statement of the lemma holds in both possible cases, the proof is concluded. □

Next, we show that no correct validator can (fully or prematurely) time out from a view with a correct leader before every correct validator receives a proposal from that leader.

Lemma 24. *Let $v > 1$ be any post-GST view with a correct leader. Then, no correct validator (fully or prematurely) times out from view v before every correct validator receives a proposal from the leader of view v.*

Proof. For the sake of contradiction, suppose that some correct validator i (either fully or prematurely) times out from view v before another correct validator j receives the proposal from the leader of view v. Observe that validator i cannot (fully or prematurely) time out in view v before time $\tau(v) + \Theta_{\text{view}}$ (by Lemma 14). By Lemma 19, the leader of view v issues its proposal by time $\tau(v) + 3\delta + \Theta_{\text{recovery}}$. Consequently, validator j receives this proposal by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + \delta < \tau(v) + \Theta_{\text{view}}$, which contradicts the assumption that validator i times out before validator j receives the leader's proposal. Hence, the claim follows. □

The following lemma shows that any QC formed in a post-GST view with a correct leader must correspond to the proposal issued in that same view.

Lemma 25. *Let $v > 1$ be any post-GST view with a correct leader, and let p denote the proposal issued by the leader in view v. If any QC qc is formed in view v, then $qc.\texttt{proposal_id} = p.\texttt{proposal_id}$.*

Proof. Assume for contradiction that qc does not point to p. Note that all correct validators that cast their votes in view v after receiving the leader's proposal do vote for the received proposal, and therefore must vote for p. Then, there must exist a correct validator contributing to qc by issuing a timeout message (upon fully or prematurely timing out from view v) including a vote for a proposal different from p. However, by Lemma 24, that correct validator must have previously received the leader's proposal p. Upon receiving proposal p, the correct validator must have voted for p (line 15 of Alg. 1), since, had its current view been greater than v (which would prevent the aforementioned vote), it would not have issued a timeout message associated with view v . Consequently, by Lemma 1, the timeout message must include a vote for p, which implies that qc points to p. □

The following lemma establishes that if a correct leader issues a proposal p in any post-GST view v, then, before the first TC for view $v + 1$ is formed (if any), at least $f + 1$ correct validators must have observed a QC formed in view v. This, in turn, implies that these validators update their local QC variable (last_qc) to this QC before the TC for view $v + 1$ is constructed.

Lemma 26. *Let $v > 1$ be any post-GST view with a correct leader. Then, before any TC for view $v + 1$ is formed (if any), at least $f + 1$ correct validators updated their* last_qc *variable to a QC from view v.*

Proof. Let tc be any TC such that $tc.\texttt{view} = v + 1$. Since at least $f + 1$ correct validators must have issued timeout messages for view $v+1$ for tc to exist, and since all timeout messages from correct validators are valid (by Lemma 11), each of them must include either a TC or a QC from the previous view v. By Lemma 23, no TC can be formed for view v, which implies that all correctly issued timeout messages must carry a QC from view v. Moreover, by the definition of the CreateTimeoutMessage function (line 26 of Alg. 6), this QC must be stored in the last_qc variable of each of these $f + 1$ correct validators. □

We now complement the previous lemma by proving that if there exists any post-GST view v with a correct leader, and some correct validator enters a view greater than $v + 1$, then at least $f + 1$ correct validators must have updated their last_qc variable to a QC formed in view v.

Lemma 27. *Let $v > 1$ be any post-GST view with a correct leader. Then, before any correct validator enters any view greater than $v+1$, at least $f+1$ correct validators updated their* last_qc *variable to a QC from view v.*

Proof. According to Lemma 12, for any view $> v+1$ to be entered, some correct validator must first enter view $v+2$. If this occurs as a result of a QC formed in view $v+1$, then at least $f+1$ correct validators must have voted for a proposal in that view. Such votes can be cast only after the validators have entered view $v+1$, regardless of whether the vote was issued upon receiving a proposal or upon timing out from view $v+1$. Upon entering view $v+1$—which can only happen via a QC from view v (by Lemma 23)—each of these $f+1$ correct validators update their last_qc variable to this QC (see the IncrementView function). Hence, the lemma holds in this case. Alternatively, if the transition to view $v+2$ occurs via a TC for view $v+1$, then the lemma holds due to Lemma 26. □

The following lemma establishes that any proposal (whether fresh or a reproposal) issued by a correct leader after GST is strictly extended by every proposal issued in subsequent views. This property is essential for ensuring liveness. Specifically, once a correct leader issues a proposal in a post-GST view, the lemma guarantees that all future proposals—regardless of whether they are issued by correct or Byzantine leaders—must extend this honest proposal. Consequently, once a group of three consecutive correct leaders occurs, the first leader in this group will strictly extend the honest proposal, and since the block proposed by this leader becomes committed by the end of the third view from the group, the original honest proposal is eventually committed as well.

Lemma 28. *Consider a proposal p issued by a correct leader in any post-GST view $v > 1$. Then, any valid proposal p^* issued in any view $v^* \geq v+1$ (i.e., $p^*.\texttt{view} = v^* \geq v+1$) must strictly extend p.*

Proof. We prove the claim by induction on the view number v'. Consider the following four induction hypotheses, $X(v')$, $Y(v')$, $Q(v')$, and $Z(v')$:

- $X(v')$: Any (valid) proposal p^* in view $v' \geq v+1$ strictly extends p and satisfies $p^*.\texttt{block.qc.view} \geq v$. Consequently, any (valid) tip T (of a fresh proposal) from view v' also strictly extends p and satisfies $T.\texttt{block_header.qc.view} \geq v$.
- $Y(v')$: Let tc be any TC with $tc.\texttt{view} = v' \geq v+1$ and $tc.\texttt{high_qc} \neq \bot$. Then, any proposal p_{tc} with $p_{tc}.\texttt{proposal_id} = tc.\texttt{high_qc.proposal_id}$ extends p and $tc.\texttt{high_qc.view} \geq v$.
- $Q(v')$: For any QC qc with $qc.\texttt{view} = v' \geq v+1$, any proposal p_{qc} with $p_{qc}.\texttt{proposal_id} = qc.\texttt{proposal_id}$ strictly extends proposal p.
- $Z(v')$: Let tc be any TC with $tc.\texttt{view} = v' \geq v+1$ and $tc.\texttt{high_tip} \neq \bot$. Then, the proposal whose tip is $tc.\texttt{high_tip}$ strictly extends p and $tc.\texttt{high_tip.block_header.qc.view} \geq v$.

We begin by proving the base case. Note that $X(v')$ is equivalent to the lemma's statement.

$X(v'=v+1)$: Let p^* be any proposal such that $p^*.\texttt{view} = v+1$. By Lemma 23, we have $p^*.\texttt{tc} = \bot$, which implies that $p^*.\texttt{block.qc.view} = v$. Due to Lemma 25, it follows that a parent of proposal p^* is p, thereby proving that $X(v'=v+1)$ holds.

$Y(v'=v+1)$: Let tc be any TC such that $tc.\texttt{view} = v+1$ and $tc.\texttt{high_qc} \neq \bot$. By Lemma 26, at least $f+1$ correct validators must have updated their last_qc variable to a QC for view v before tc was formed. Moreover, as $tc.\texttt{view} = v+1$, the preceding argument, together with the validity of tc, implies that $tc.\texttt{high_qc.view} = v$. Due to Lemma 25, $tc.\texttt{high_qc.proposal_id} = p.\texttt{proposal_id}$, thereby proving $Y(v'=v+1)$.

$Q(v'=v+1)$: Let qc be any QC with $qc.\texttt{view} = v' = v+1$. We distinguish between two cases:

- Suppose that at least one correct validator contributing to qc did so upon receiving a proposal in view v'. In this case, the received proposal strictly extends proposal p, since $X(v'=v+1)$ holds. Hence, $Q(v'=v+1)$ is satisfied.
- Suppose that no correct validator contributing to qc did so upon receiving a proposal in view v'. In this case, all correct votes issued in view v' are based on timeout messages, each reflecting the validator's local tip at the time of issuance. Now consider any correct validator i contributing to qc. Such a contribution must occur when validator i (either fully or prematurely) times out from view $v+1$. This implies that validator i had previously entered view $v+1$, which, by Lemma 25, can only happen after accepting a

QC from view v. Therefore, at the time of issuing the timeout message, validator i's last_qc variable must contain a QC from view v. The only circumstance under which validator i would issue a vote contributing to qc is if it updated its local tip in view $v+1$ based on a fresh proposal—since only in that case would local_tip.view > last_qc.view, prompting a vote. However, in this scenario, the condition $X(v' = v+1)$ implies that $Q(v' = v+1)$ holds.

Thus, $Q(v' = v+1)$ holds in both cases.

$\underline{Z(v' = v+1)}$: Let tc be any TC such that $tc.\texttt{view} = v+1$ and $tc.\texttt{high_tip} \neq \bot$. By Lemma 26, at least $f+1$ correct validators must have updated their last_qc variable to a QC for view v before tc was formed. As $tc.\texttt{view} = v+1$ and at least one correct validator contributing to tc has its last_qc variable containing a QC from view v, it follows that $tc.\texttt{high_tip.view} = v+1$. As $X(v' = v+1)$ holds, $tc.\texttt{high_tip}$ must strictly extend proposal p and $tc.\texttt{high_tip.block_header.qc.view} \geq v$, thereby proving $Z(v' = v+1)$.

We now prove that the inductive step holds.

$\underline{X(v' > v+1)}$: Let us consider all possible cases for a valid proposal p^* with $p^*.\texttt{view} = v' > v+1$:

- Suppose p^* is a fresh proposal. We further consider three possibilities:
  - Let $p^*.\texttt{block.qc.view} = v'-1 \geq v+1$. In this case, a parent of p^* strictly extends proposal p (because $Q(v'-1 \geq v+1)$ holds), which implies that p^* strictly extends proposal p. Thus, $X(v' > v+1)$ holds in this case.
  - Let $p^*.\texttt{nec} \neq \bot$. We first prove that $p^*.\texttt{nec.high_tip_qc_view} = p^*.\texttt{block.qc.view} \geq v$. In order for $p^*.\texttt{nec}$ to be formed, a correct validator i must have received an NE request accompanied by a TC tc with $tc.\texttt{view} = v'-1 \geq v+1$. Given that $Z(v'-1)$ holds, $tc.\texttt{high_tip}$ strictly extends proposal p and $tc.\texttt{high_tip.block_header.qc.view} \geq v$. Therefore, $p^*.\texttt{nec.high_tip_qc_view} = p^*.\texttt{block.qc.view} \geq v$. Hence, it is left to prove that p^* strictly extends proposal p. We now differentiate two cases:
    - Let $p^*.\texttt{block.qc.view} = v$. In this case, a parent of p^* must be p (by Lemma 25), which proves the statement of the invariant.
    - Let $p^*.\texttt{block.qc.view} > v$. In this case, the fact that $Q(p^*.\texttt{block.qc.view} \geq v+1)$ ensures that the statement of the invariant holds.
  - Let $p^*.\texttt{nec} = \bot$ and $p^*.\texttt{tc.high_qc} \neq \bot$. Note that $p^*.\texttt{tc.view} = v'-1 \geq v+1$. In this case, a parent of p^* extends p and $p^*.\texttt{tc.high_qc.view} \geq v$ (as ensured by $Y(v'-1 \geq v+1)$), which proves $X(v' > v+1)$ in this scenario.
- Suppose p^* is a reproposal. In this case, p^* is a reproposal of $p^*.\texttt{tc.high_tip}$; note that $p^*.\texttt{tc.view} = v'-1 \geq v+1$. Due to $Z(v'-1 \geq v+1)$, we have that p^* strictly extends proposal p (as $p^*.\texttt{tc.high_tip}$ does so) and $p^*.\texttt{tc.high_tip.block_header.qc.view} \geq v$. Thus, $X(v' > v+1)$ holds in this case as well.

As $X(v' > v+1)$ holds in all possible scenarios, $X(v' > v+1)$ is satisfied.

$Y(v' > v+1)$: Let tc be any TC such that $tc.\texttt{view} = v' > v+1$ and $tc.\texttt{high_qc} \neq \bot$. Since tc is formed and the first correct validator to issue a timeout message must do so upon fully timing out from view $v' > v+1$ (by Lemma 9), Lemma 27 ensures that at least $f+1$ correct validators must have updated their last_qc variable to a QC for view v before tc was formed. Hence, $tc.\texttt{high_qc.view} \geq v$. Moreover, due to the validity of tc, $tc.\texttt{high_qc.view} < v'$. We now consider two cases:

- If $tc.\texttt{high_qc.view} = v$, then by Lemma 25, $tc.\texttt{high_qc}$ points to proposal p, thereby establishing that $Y(v' > v+1)$ holds in this case.
- If $tc.\texttt{high_qc.view} > v$, then $Y(v' > v+1)$ follows from $Q(tc.\texttt{high_qc.view})$.

$\underline{Q(v' > v+1)}$: Let qc be any QC with $qc.\texttt{view} = v' > v+1$. We distinguish between two cases:

- Suppose that at least one correct validator contributing to qc did so upon receiving a proposal in view v'. In this case, the received proposal strictly extends proposal p, since $X(v' > v+1)$ holds. Hence, $Q(v' > v+1)$ is satisfied.

– Suppose that no correct validator contributing to qc did so upon receiving a proposal in view v'. In this case, all correct votes issued in view v' are based on timeout messages, each reflecting the validator's local tip at the time of issuance. By Lemma 27, at least $f+1$ correct validators must have updated their $\mathsf{last_qc}$ variable to a QC from view v before qc was formed. Let i be any such correct validator who both updated its $\mathsf{last_qc}$ to a QC from view v prior to the formation of qc and contributed a vote to qc. This implies that validator i's local tip T must belong to a view $\geq v+1$, as the vote is cast for that proposal. (Otherwise, i would have instead issued a timeout message containing its local QC and would not have voted.) Given that $X(T.\texttt{view} \in [v+1, v'])$ holds, it follows that qc must point to a proposal that strictly extends p.

Thus, $Q(v' > v+1)$ holds in both cases.

$Z(v' > v+1)$: Let tc be any TC with $tc.\texttt{view} = v' > v$ and $tc.\texttt{high_tip} \neq \bot$. Since tc is formed and the first correct validator to issue a timeout message must do so upon timing out from view $v' > v+1$, Lemma 27 ensures that at least $f+1$ correct validators must have updated their $\mathsf{last_qc}$ variable to a QC for view v before tc was formed. Hence, $tc.\texttt{high_tip.view} > v$; note that $tc.\texttt{high_tip.view} \leq v'$. Therefore, $Z(v' > v+1)$ holds due to $X(tc.\texttt{high_tip.view})$. □

Next, we derive an upper bound on the difference $\tau(v+1) - \tau(v)$ for a post-GST view v.

Lemma 29. *Let $v > 1$ be any post-GST view. Then, the following bound holds:*

$$\tau(v+1) - \tau(v) \leq 2\Theta_{\text{view}} + 2\delta.$$

Proof. By contradiction, suppose that $\tau(v+1) - \tau(v) > 2\Theta_{\text{view}} + 2\delta$. Let us now consider two different cases:

– Suppose that the leader of view v is correct. By Lemma 20, every correct validator issues its vote for a proposal from view v by time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + \delta < \tau(v) + 2\Theta_{\text{view}} + 2\delta$ (otherwise, it would hold that $\tau(v+1) < \tau(v) + 3\delta + \Theta_{\text{recovery}} + \delta$, contradicting our initial assumption). By time $\tau(v) + 3\delta + \Theta_{\text{recovery}} + 2\delta$, the leader receives votes from all correct validators. Once this occurs, the leader forms a QC (line 20 of Alg. 1) for view v and proceeds to enter view $v+1$ (line 22 of Alg. 1). Hence, $\tau(v+1) - \tau(v) \leq 3\delta + \Theta_{\text{recovery}} + 2\delta$, which again contradicts our starting assumption. This implies that the leader of v cannot be correct.
– Suppose that the leader of view v is faulty. First, observe that a correct validator i that entered view v at time $\tau(v)$ issues a valid timeout message at time $\tau(v) + \Theta_{\text{view}}$, which carries either a QC or a TC from view $v-1$. (Otherwise, validator i would have already advanced to a future view, contradicting the initial assumption.) Consequently, all correct validators enter view v by time $\tau(v) + \Theta_{\text{view}} + \delta$, since upon receiving any valid timeout message associated with view v, a correct validator immediately enters that view (line 5 of Alg. 2). Subsequently, each correct validator issues its own timeout message by time $\tau(v) + 2\Theta_{\text{view}} + \delta$ (as none have progressed to later views by this point), and all such messages are received by every correct validator by time $\tau(v) + 2\Theta_{\text{view}} + 2\delta$. At this stage, a correct validator can form a QC or a TC associated with view v (line 17 of Alg. 2), enabling it to move to the next view $v+1$ (line 19 or line 24 of Alg. 2). Thus, we once again reach a contradiction with the initial assumption.

Since neither of the two cases can occur, our initial assumption must be false, thereby completing the proof of the lemma. □

Next, we show that every correct validator enters a post-GST view within $(f+1) \cdot (2\Theta_{\text{view}} + 2\delta) + 4\delta + \Theta_{\text{recovery}}$ time after GST.

Lemma 30. *Let $v_{\max}$ denote the highest view that is not a post-GST view. Then, every correct validator enters a view $> v_{\max}$ by time*

$$GST + (f+1) \cdot (2\Theta_{\text{view}} + 2\delta) + 4\delta + \Theta_{\text{recovery}}.$$

Proof. Since $v_{\max}$ is the highest non-post-GST view, it follows that all views $> v_{\max}$ are post-GST views. Consider any correct validator i. Among the views $\{v_{\max}+1, v_{\max}+2, \ldots, v_{\max}+f+1\}$, there exists a view v_i with a correct leader. By Lemma 18, the leader of view v_i enters view v_i. Then, by Lemma 19, it issues its proposal by time $\tau(v_i) + 3\delta + \Theta_{\text{recovery}}$, which is received by validator i no later than time $\tau(v_i) + 4\delta + \Theta_{\text{recovery}}$. Upon receiving this proposal, if $\mathsf{cur_view}$ at validator i is less than v_i, then validator i enters view $v_i > v_{\max}$. Otherwise, validator i has already entered a view higher than $v_{\max}$. Thus, it remains to bound $\tau(v_i) + 4\delta + \Theta_{\text{recovery}} - \text{GST}$, which we do by first bounding $\tau(v_{\max}+1) - \text{GST}$.

Bounding $\tau(v_{\max}+1) - GST$. Assume, for the sake of contradiction, that

$$\tau(v_{\max}+1) - \text{GST} > 2\Theta_{\text{view}} + 2\delta.$$

Let i be a correct validator that enters view $v_{\max}$ at time $\tau(v_{\max})$. Validator i issues a valid timeout message associated with view $v_{\max}$ by time $\text{GST} + \Theta_{\text{view}}$, carrying either a QC or a TC from view $v_{\max} - 1$. (Otherwise, validator i would have already progressed to a future view, contradicting the initial assumption.) Consequently, all correct validators enter view $v_{\max}$ by time $\text{GST} + \Theta_{\text{view}} + \delta$, since upon receiving any valid timeout message for view $v_{\max}$, a correct validator immediately enters that view (if it has not already done so) at line 5 of Alg. 2. Each correct validator then issues its own timeout message for view $v_{\max}$ by time $\text{GST} + 2\Theta_{\text{view}} + \delta$ (as none have advanced to any future view), and these messages are received by every correct validator by time $\text{GST} + 2\Theta_{\text{view}} + 2\delta$. At this point, a correct validator can form a QC or a TC associated with view $v_{\max}$ (line 17 of Alg. 2), enabling it to move to view $v_{\max}+1$. Hence,

$$\tau(v_{\max}+1) - \text{GST} \leq 2\Theta_{\text{view}} + 2\delta.$$

Bounding $\tau(v_i) + 4\delta + \Theta_{\text{recovery}} - GST$. Recall that $\tau(v_{\max}+f+1) \geq \tau(v_{\max}+1)$ by Lemma 12. Finally, we bound the time by which validator i enters its leader view v_i and issues its proposal:

$$\begin{aligned}
\tau(v_i) + 4\delta + \Theta_{\text{recovery}} - \text{GST} \;&\leq \underbrace{\big(\tau(v_{\max}+1) - \text{GST}\big)}_{\leq 2\Theta_{\text{view}}+2\delta} + \underbrace{\big(\tau(v_{\max}+2) - \tau(v_{\max}+1)\big)}_{\leq 2\Theta_{\text{view}}+2\delta} \\
&\quad + \cdots + \underbrace{\big(\tau(v_{\max}+f+1) - \tau(v_{\max}+f)\big)}_{\leq 2\Theta_{\text{view}}+2\delta} + 4\delta + \Theta_{\text{recovery}} \\
&\leq (f+1)\cdot(2\Theta_{\text{view}} + 2\delta) + 4\delta + \Theta_{\text{recovery}}.
\end{aligned}$$

This establishes the desired bound and completes the proof. □

We are now ready to prove that MonadBFT satisfies the liveness property. Specifically, we show that any block issued by a correct validator after time

$$\text{GST} + (f+1)\cdot(2\Theta_{\text{view}} + 2\delta) + 4\delta + \Theta_{\text{recovery}}$$

is eventually committed. The need for this additional delay beyond GST arises from the instability of the network immediately after GST, as established in Lemma 30. Prior to this point, the network may still exhibit pre-GST behavior, which can prevent correct proposals from being supported. For instance, consider a scenario where a correct validator is the leader of a pre-GST view v. If this validator issues its proposal exactly at time GST, it is possible that all other correct validators have already advanced to a higher view due to earlier timeouts. As a result, the proposal for view v cannot gather sufficient votes to be committed, despite being correctly formed. This delay thus ensures that the system has fully transitioned out of any pre-GST instability before requiring commitment guarantees. One could interpret this as a semantic adjustment: the "effective GST" is the model-defined GST plus this bounded delay, accounting for residual view divergence.

Theorem 3 (Liveness). *After time* $\text{GST} + (f+1)\cdot(2\Theta_{\text{view}} + 2\delta) + 4\delta + \Theta_{\text{recovery}}$*, every block issued by a correct validator is committed by every correct validator.*

Proof. Consider any correct validator i that issues a proposal p after time

$$\mathrm{GST} + (f+1) \cdot (2\Theta_{\mathrm{view}} + 2\delta) + 4\delta + \Theta_{\mathrm{recovery}}.$$

By Lemma 30, validator i must have entered a post-GST view by that time. Let v denote the post-GST view in which the validator i issues its proposal p. By Lemma 22, within the sequence of views from $v+1$ to $v+3f+3$, there exists a group of three consecutive views

$$S = [v^*,\, v^*+1,\, v^*+2],$$

all of which have correct leaders.

Since the leader of view v^* is correct, it issues a valid proposal p^* that strictly extends proposal p (by Lemma 28). By Lemma 18, the leader of view v^*+1 enters view $v+1$. Furthermore, by Lemma 23, this leader enters view v^*+1 only after accepting a QC from view v^*. By Lemma 25, this implies that the proposal issued by the leader of view v^*+1 must contain a QC for p^*. Furthermore, the leader of view v^*+2 issues a proposal that contains a QC for the proposal from view v^*+1 (by Lemma 21), which itself includes a QC for p^*. Consequently, every correct validator j receives the proposal from the leader of view v^*+2 by time $\tau(v^*+2)+2\delta$. We now distinguish two cases:

- If the current view of validator j is $\leq v^*+2$ upon receiving this proposal, then validator j commits proposal p (as the proposal from view v^*+2 is processed) at line 11 of Alg. 1, which proves the theorem in this case.
- Otherwise, validator j has already entered some view $v^{**} > v^*+2$. Since no TC can be formed for any view between v^*+2 and $v^{**}-1$ by time $\tau(v^*+2)+2\delta < \tau(v^*+2)+\Theta_{\mathrm{view}}$, validator j must have entered view v^{**} upon accepting a QC from view $v^{**}-1 \geq v^*+2$. This QC necessarily "extends" the proposal from view v^* through a chain of QCs (with no TCs), ensuring that there exist at least two consecutive QCs extending the proposal from view v^*. Hence, p is committed in this case as well, since the CommitAndSpecCommit function is invoked after every QC acceptance. □

A.4 Optimistic Responsiveness

In this subsection, we prove that MonadBFT satisfies optimistic responsiveness: if all validators are correct, then during any post-GST time interval of duration d, each correct validator commits at least $\Omega(d/\delta)$ blocks, where δ denotes the actual network delay. To this end, we first determine when the first post-GST view is entered by the first correct validator relative to GST.

Lemma 31. *Let $v > 1$ be the smallest post-GST view. Then, the following bound holds:*

$$\tau(v) - GST \leq 2\Theta_{\mathrm{view}} + 2\delta.$$

Proof. (This statement was already established in the first part of the proof of Lemma 30. For completeness, we provide a self-contained proof here.) Suppose, for the sake of contradiction, that $\tau(v) - \mathrm{GST} > 2\Theta_{\mathrm{view}} + 2\delta$. By Lemma 12, no correct validator enters any view greater than $v-1$ by time $\mathrm{GST} + 2\Theta_{\mathrm{view}} + 2\delta$.

Note first that $v-1$ cannot be a post-GST view, since v is the smallest post-GST view. Hence, $\tau(v-1) < \mathrm{GST}$. Let i be a correct validator that enters view $v-1$ at time $\tau(v-1)$. Validator i issues its timeout message for view $v-1$ by time $\mathrm{GST} + \Theta_{\mathrm{view}}$, as it does not enter any view greater than $v-1$ by that time. By Lemma 11, this timeout message must carry either a QC or a TC from view $v-2$. Upon receiving this timeout message (by time $\mathrm{GST} + \Theta_{\mathrm{view}} + \delta$), each correct validator enters view $v-1$, unless it has already done so.

Subsequently, every correct validator (either fully or prematurely) times out from view $v-1$ by time $\mathrm{GST} + \Theta_{\mathrm{view}} + \delta + \Theta_{\mathrm{view}}$, since no correct validator enters any view higher than $v-1$ before that time. Finally, by time $\mathrm{GST} + 2\Theta_{\mathrm{view}} + 2\delta$, every correct validator has received timeout messages from all other correct validators and forms either a QC or a TC for view $v-1$ (line 17 of Alg. 2), thereby entering view v (line 19 or line 24 of Alg. 2). This contradicts our initial assumption, completing the proof. □

We are now ready to establish the optimistic responsiveness property.

Theorem 4 (Optimistic responsiveness). *If all validators are correct, then during any post-GST time interval* $[\tau_1, \tau_2]$ *such that* $\tau_1 > GST + 4\Theta_{\text{view}} + 4\delta$ *and* $\tau_2 - \tau_1 \in \omega(\delta)$*, each correct validator commits* $\Omega\left(\frac{\tau_2 - \tau_1}{\delta}\right)$ *blocks.*

Proof. Let $v_{\max}$ denote the largest post-GST view such that $\tau(v_{\max}) < \tau_1$. Note that $v_{\max}$ is well defined by Lemma 31. Next, let $\mathcal{V}$ denote the set of views that "start" during the time interval $[\tau_1, \tau_2]$:

$$\mathcal{V} = \{\, v \mid \tau(v) \geq \tau_1 \text{ and } \tau(v) + 2\delta \leq \tau_2 \,\}.$$

First, we show that $\tau(v_{\max} + 1) < \tau_1 + 5\delta$, i.e., that the smallest view of $\mathcal{V}$ starts at most 5δ time after τ_1. Let v_0 denote the smallest post-GST view. By Lemma 31, $\tau(v_0) \leq \text{GST} + 2\Theta_{\text{view}} + 2\delta$. Moreover, Lemma 29 proves that $\tau(v_0 + 1) \leq \tau(v_0) + 2\Theta_{\text{view}} + 2\delta \leq \text{GST} + 4\Theta_{\text{view}} + 4\delta < \tau_1$. Hence, $v_{\max} \geq v_0 + 1$, which implies that view $v_{\max} - 1$ is a post-GST view; as all validators are correct, its leader is correct. By Lemma 23, no TC can be formed for view $v_{\max} - 1$, and thus the leader of view $v_{\max}$ entered view $v_{\max}$ upon accepting a QC for view $v_{\max} - 1$. Therefore, Lemma 21 applies to the pair of views $v_{\max}$ and $v_{\max} + 1$: the leader of view $v_{\max} + 1$ enters view $v_{\max} + 1$ upon accepting a QC for view $v_{\max}$ by time $\tau(v_{\max}) + 3\delta + 2\delta$. Consequently, $\tau(v_{\max} + 1) \leq \tau(v_{\max}) + 5\delta < \tau_1 + 5\delta$; recall that $\tau(v_{\max} + 1) \geq \tau_1$ by the definition of $v_{\max}$. As $\tau_2 - \tau_1 \in \omega(\delta)$, the smallest view v' of $\mathcal{V}$ is indeed $v' = v_{\max} + 1$. Note that the leader of view v' enters view v' upon accepting a QC for view $v' - 1 = v_{\max}$.

By Lemma 21, the number of views in $\mathcal{V}$ satisfies

$$|\mathcal{V}| \in \Omega\left(\frac{\tau_2 - \tau_1}{\delta}\right).$$

Specifically, the smallest view of $\mathcal{V}$ starts by time $\tau_1 + 5\delta$ and, for each view $v \in \mathcal{V} \setminus \{\text{the largest view of } \mathcal{V}\}$, Lemma 21 establishes that the time difference between consecutive views is bounded, i.e., $\tau(v + 1) - \tau(v) \in O(\delta)$; recall that, by Lemma 21, the leader of each view of $\mathcal{V}$ enters that view upon accepting a QC for the preceding view, which ensures that Lemma 21 indeed applies to every pair of consecutive views of $\mathcal{V}$.

Let v^* denote the largest view in $\mathcal{V}$. By Lemma 21 (and the assumption that all validators are correct), all correct validators receive the proposal p^* issued by the leader of view v^* by time $\tau(v^*) + 2\delta \leq \tau_2$. Furthermore, by Lemma 23, all proposals issued in views within $\mathcal{V}$ (including v^*) are "connected" through QCs, since no TC can be formed for any view in $\mathcal{V}$.

We now distinguish two cases depending on the state of any correct validator j upon receiving the proposal p^* from the leader of view v^* (which occurs by time τ_2):

- If the current view of validator j is at most v^* when the proposal is received, then j commits the grandparent of p^* (as the proposal from view v^* is processed) at line 11 of Alg. 1. Therefore, all proposals issued in the views of $\mathcal{V}$—except for the last two—are committed. Since $|\mathcal{V}| \in \Omega\left(\frac{\tau_2 - \tau_1}{\delta}\right)$ and we commit $|\mathcal{V}| - 2$ blocks, the number of committed blocks is $|\mathcal{V}| - 2 \in \Omega\left(\frac{\tau_2 - \tau_1}{\delta}\right) - 2 = \Omega\left(\frac{\tau_2 - \tau_1}{\delta}\right)$, where the last equality holds because $\tau_2 - \tau_1 \in \omega(\delta)$.
- Otherwise, validator j has already entered some view $v^{**} > v^*$. Since no TC can be formed for any view greater than or equal to v^*, validator j must have entered view v^{**} upon accepting a QC from view $v^{**} - 1 \geq v^*$. This QC necessarily "extends" the proposal from view v^* through a continuous chain of QCs (without any TCs). Hence, the grandparent of p^* is committed in this case as well, since the CommitAndSpecCommit function is invoked after every QC acceptance. This, in turn, implies that all proposals issued in the views of $\mathcal{V}$—except for the last one—are committed. By the same reasoning as in the previous case, the number of committed blocks is $|\mathcal{V}| - 1 \in \Omega\left(\frac{\tau_2 - \tau_1}{\delta}\right)$. □

As shown, Theorem 4 captures the case where all validators are correct. However, we can state a stronger guarantee. Assume there is a sequence $G = [v, v + 1, \ldots, v + x - 1]$ of x consecutive post-GST views, each led by a correct validator. Then, by the time a correct validator receives the proposal issued by the leader of view $v + x - 1$, it commits all blocks proposed in the views of G, except possibly those from the last two

views $v + x - 2$ and $v + x - 1$. Moreover, this occurs within $O(x\delta)$ time from the moment the first correct validator enters view $v+1$. Crucially, if the leader of view v enters view v upon accepting a QC (rather than a TC), the same guarantee is achieved within $O(x\delta)$ time from the moment the first correct validator enters view v. Note that this guarantee does not require all validators to be correct.

Theorem 5 (Optimistic responsiveness; stronger guarantee). *Let G be a sequence of x consecutive post-GST views $[v, v+1, \ldots, v+x-1]$, each led by a correct validator. Consider any correct validator i, and let τ_i^{v+x-1} denote the time at which i receives the proposal issued by the leader of view $v + x - 1$. Then, by time τ_i^{v+x-1}, validator i commits all blocks proposed in the views of G, except possibly those from the last two views $v+x-2$ and $v+x-1$. Furthermore, the time difference $\tau_i^{v+x-1} - \tau(v+1)$ lies in $O(x\delta)$. Finally, if the leader of view v enters view v upon accepting a QC associated with view $v-1$, then $\tau_i^{v+x-1} - \tau(v)$ also lies in $O(x\delta)$.*

Proof. First, observe that the leader of view $v + 1$ enters view $v + 1$ upon accepting a QC for view v. By Lemma 18, the leader of view $v + 1$ indeed enters view $v + 1$. Since a validator can only enter a new view upon accepting either a QC or a TC from the previous view, and Lemma 23 ensures that no TC can be formed for view v, it follows that the leader of view $v + 1$ must have entered view $v + 1$ upon accepting a QC for view v.

Next, for each view $v' \in [v+1, v+2, \ldots, v+x-2]$, Lemma 21 shows that the time gap between consecutive views is bounded, i.e., $\tau(v'+1) - \tau(v') \in O(\delta)$. Consequently, $\tau(v+x-1) - \tau(v+1) \in O(x\delta)$. Furthermore, if the leader of view v enters view v upon accepting a QC associated with view $v-1$, then Lemma 21 also proves that $\tau(v+1) - \tau(v) \in O(\delta)$, which implies that $\tau(v+x-1) - \tau(v) \in O(x\delta)$.

Let $v^* = v + x - 1$ denote the largest view in G. By Lemma 21 (and the assumption that all leaders of views from G are correct), each correct validator i receives the proposal p^* issued by the leader of view v^* by time $\tau(v^*) + 2\delta$. (Note that $\tau(v^*) + 2\delta - \tau(v+1) \in O(x\delta)$ and, if the leader of view v enters view v upon accepting a QC associated with view $v-1$, $\tau(v^*) + 2\delta - \tau(v) \in O(x\delta)$.) Furthermore, by Lemma 23, all proposals issued in views within G (including v^*) are "connected" through QCs, since no TC can be formed for any view in G.

We now distinguish two cases depending on the state of validator i upon receiving the proposal p^* from the leader of view v^* (which occurs by time $\tau(v^*) + 2\delta$):

- If the current view of validator i is at most v^* when the proposal is received, then i commits the grandparent of p^* (as the proposal from view v^* is processed) at line 11 of Alg. 1. Therefore, all proposals issued in the views of G—except for the last two—are committed.
- Otherwise, validator i has already entered some view $v^{**} > v^*$. Since no TC can be formed for any view $\geq v^*$ by time $\tau(v^*) + 2\delta$ (by Lemma 15), validator i must have entered view v^{**} upon accepting a QC from view $v^{**} - 1 \geq v^*$. This QC necessarily "extends" the proposal from view v^* through a continuous chain of QCs (without any TCs). Hence, the grandparent of p^* is committed in this case as well, since the CommitAndSpecCommit function is invoked after every QC acceptance, which also proves that all proposals issued in the views of G—except for the last two—are committed. □

A.5 Leader Fault Isolation

In this subsection, we prove that MonadBFT satisfies the leader fault isolation property, which states that, after GST, if a view v led by a Byzantine leader is "sandwiched" between two views led by correct leaders, then the duration of view v is at most $O(\delta) + \Theta_{\text{view}}$. To this end, we first show that all correct validators "complete" any post-GST view led by a correct leader within $O(\delta)$ time of one another.

Lemma 32. *Let $v > 1$ be any post-GST view with a correct leader. Then, (1) all correct validators enter a view greater than v by time $\tau(v+1) + 2\delta$, or (2) $\tau(v+2) \leq \tau(v+1) + 2\delta$.*

Proof. Let L denote the leader of view v. By Lemma 23, no TC can be formed for view v, and at least $f + 1$ correct validators do not issue timeout messages associated with this view. Hence, any correct validator i

that enters view $v+1$ at time $\tau(v+1)$ must have done so upon accepting a QC associated with view v. Observe that, since at least $f+1$ correct validators did not emit timeout messages for view v, validator i could not have accepted a QC for view v at line 19 of Alg. 2, as this requires $2f+1$ timeout messages. To establish the lemma, we now examine all possible circumstances under which validator i could have accepted a QC for view v:

- Suppose that i accepts a QC for view v at line 5 of Alg. 1.
  In this case, validator i forwards the accepted QC to the leader L (line 10 of Alg. 1), which is received by L by time $\tau(v+1)+\delta$. Upon receiving this QC, if the current view of L is $\leq v$, the leader disseminates the QC to all correct validators (line 35 of Alg. 1), that receive it by time $\tau(v+1)+2\delta$, thereby establishing the lemma in this case. Otherwise, we consider two different cases:
  - Suppose that the current view of L is $v+1$. Then, the leader has already accepted a QC associated with view v and disseminated it by time $\tau(v+1)+\delta$, which again ensures that the lemma holds.
  - Suppose that that the current view of L is $> v+1$. In this case, Lemma 12 proves that $\tau(v+2) \leq \tau(v+1)+2\delta$.
- Suppose that i accepts a QC for view v at line 22 of Alg. 1.
  In this case, validator i is either the leader of view v or of view $v+1$. If i is the leader of view v, it disseminates the accepted QC (line 25 of Alg. 1), which all correct validators receive by time $\tau(v+1)+\delta$, thereby establishing the lemma. Otherwise, if i is the leader of view $v+1$, it disseminates its proposal for view $v+1$ (line 27 of Alg. 1), which is likewise received by all correct validators by time $\tau(v+1)+\delta$, thus confirming the lemma in this case as well.
- Suppose that i accepts a QC for view v at line 31 of Alg. 1.
  In this case, the leader L has already disseminated this QC, implying that all correct validators receive it by time $\tau(v+1)+\delta$, thereby establishing the lemma.
- Suppose that i accepts a QC for view v at line 37 of Alg. 1.
  In this case, i is the leader of view $v+1$. Hence, it disseminates its proposal for view $v+1$ (line 39 of Alg. 1), which is received by all correct validators by time $\tau(v+1)+\delta$, thus confirming the lemma in this case as well.
- Suppose that i accepts a QC for view v at line 5 of Alg. 2.
  In this case, validator i forwards the accepted QC to the leader L (line 14 of Alg. 2), which is received by L by time $\tau(v+1)+\delta$. Upon receiving this QC, if the current view of L is $\leq v$, the leader disseminates the QC to all correct validators (line 35 of Alg. 1), that receive it by time $\tau(v+1)+2\delta$, thereby establishing the lemma in this case. Otherwise, we consider two different cases:
  - Suppose that the current view of L is $v+1$. Then, the leader has already accepted a QC associated with view v and disseminated it by time $\tau(v+1)+\delta$, which again ensures that the lemma holds.
  - Suppose that that the current view of L is $> v+1$. In this case, Lemma 12 proves that $\tau(v+2) \leq \tau(v+1)+2\delta$.

Since the lemma holds in all possible cases, the proof is complete. □

We are ready to prove that MonadBFT satisfies the leader fault isolation property.

Theorem 6 (Leader fault isolation). *Consider any three consecutive post-GST views v, $v+1$, and $v+2$ such that the leaders of views v and $v+2$ are correct. Then, the following bound holds:*

$$\big(\textit{the earliest time at which all correct validators enter a view } > v+1\big) - \tau(v+1) \leq O(\delta) + \Theta_{\text{view}}.$$

Proof. By Lemma 32, one of the following holds: (1) all correct validators enter view $v+1$ by time $\tau(v+1)+2\delta$, or (2) $\tau(v+2) \leq \tau(v+1)+2\delta$. We first establish that $\tau(v+2)-\tau(v+1) \leq O(\delta)+\Theta_{\text{view}}$. This bound follows immediately in the second case. Consider now the first case. Assume, for the sake of contradiction, that no correct validator enters view $v+2$ (or any higher view) by time $\tau(v+1)+2\delta+\Theta_{\text{view}}+\delta$. (Otherwise, if some correct validator does enter a view $\geq v+2$ by that time, Lemma 12 guarantees that $\tau(v+2)-\tau(v+1) \leq O(\delta)+\Theta_{\text{view}}$.) Since all correct validators enter view $v+1$ by time $\tau(v+1)+2\delta$, and no correct validator enters any view higher than $v+1$ by time $\tau(v+1)+2\delta+\Theta_{\text{view}}+\delta$, each correct validator must issue a timeout

message associated with view $v+1$ by time $\tau(v+1)+2\delta+\Theta_{\text{view}}$—either due to a full or premature timeout. Consequently, by time $\tau(v+1)+2\delta+\Theta_{\text{view}}+\delta$, every correct validator has received timeout messages from all other correct validators and thus forms either a QC or a TC associated with view $v+1$ (line 17 of Alg. 2), entering view $v+2$ at line 19 or line 24 of Alg. 2 (unless it has already entered a view greater than view $v+1$). Hence, in all possible cases, we conclude that $\tau(v+2)-\tau(v+1) \leq O(\delta)+\Theta_{\text{view}}$.

Having established that $\tau(v+2)-\tau(v+1) \leq O(\delta)+\Theta_{\text{view}}$, Lemma 18 proves that the leader of view $v+2$ enters view $v+2$ by time $\tau(v+2)+3\delta$. Therefore, the leader of view $v+2$ disseminates either a proposal, a proposal request, or an NE request by time $\tau(v+2)+3\delta$. This implies that all correct validators receive the proposal, proposal request, or NE request by time $\tau(v+2)+4\delta$. Consequently, all correct validators transition to a view higher than $v+1$ by time $\tau(v+2)+4\delta \leq \tau(v+2)+O(\delta)$. Hence, the statement of the theorem follows. □